

СЕМЕНОВ А.С.

**ТЕОРИЯ АВТОМАТОВ И ФОРМАЛЬНЫХ ЯЗЫКОВ:
ТЕОРИЯ ПЕРЕВОДА**

Москва 2025

СОДЕРЖАНИЕ

Введение	
Глава 5. Промежуточные формы представления программ	
5.1. Польская запись	
5.2. Тетрады	
5.3. Триады	
5.4. Байт-коды JVM	
Глава 6. Методы синтаксически управляемого перевода	
6.1. Перевод и семантика	
6.2. СУ-схемы	
6.3. Транслирующие грамматики	
6.4. Атрибутные транслирующие грамматики	
6.5. Методика разработки описания перевода.	
6.6. Пример разработки АТ грамматики	
6.7. Трансляция класса C++ в класс Python на основе АТ-грамматики.	
Глава 7. Языковые процессоры	
7.1. L-атрибутивные и S-атрибутивные транслирующие грамматики	
7.2. Форма простого присваивания	
7.3. Атрибутивный перевод для LL(1) грамматик.	
7.4. S-атрибутивный ДМП-процессор	
Приложение А. Порядок выполнения задач	
Библиографический список	

Введение

Системы программирования - это компьютерные программы предназначены для разработки программного обеспечения. К ним относят: ассемблеры осуществляющие преобразование программы в форме исходного текста на языке ассемблера в машинные команды в виде объектного кода, трансляторы программ, компиляторы переводящие текст программы на языке высокого уровня, в эквивалентную программу на машинном языке, интерпретаторы анализирующие команды или операторы программы и тут же выполняющие их;

Пособие направлено на освоение и применение моделей формальных языков для разработки систем программирования. При этом решаются следующие задачи: определение типа модели формального языка, доказательство правильности сделанного выбора модели, построение цепочек вывода и дерева разбора для модели, нахождения эквивалентных моделей, описание языка и построения модели распознавателя, программная реализация модели, перевод языка из одной модели в другую формальную модель. Отличительной особенностью пособия является практическое содержание и решение задач с применением программной платформы, реализующей основополагающие алгоритмы формальных моделей, которые используются для проведения практических и курсовых работ. Пособие служит документацией к базовым алгоритмам платформы. Алгоритмы представлены в виде теоретико-множественной нотации.

Пособие состоит из семи глав и приложения, в котором приведены основные шаги выполнения практических работ. Теоретический материал и практические задания организованы по темам, в соответствии с классификацией моделей формальных языков, приведенной в главе 1. Основной задачей классификации является изучение порождающих и распознающих моделей формальных языков [1-5].

Пособие организовано в порядке возрастания сложности: предшествующий материал используется в последующих главах. В процессе выполнения практических заданий развивается умение разрабатывать модели формальных языков, проводить сравнительный анализ моделей, определять их вид в соответствии с классификацией, а также применять объектно-ориентированный подход для реализации моделей на языке программирования C#.

Во второй главе рассматривается класс автоматных грамматик и конечных автоматов, которые широко применяются в различных программных приложениях объектно-ориентированного подхода и искусственного интеллекта.

В третьей главе рассмотрен класс контекстно-свободных грамматик и МП-автоматов, позволяющий строить оптимальным образом синтаксические анализаторы для языков программирования.

В четвертой главе рассмотрены грамматики общего вида и машины Тьюринга, приведены соотношения между языками и грамматикам.

В пятой и шестой главах рассматриваются методы перевода между различными формальными моделями.

Книга полезна для студентов, аспирантов, ученых и специалистов, занимающихся разработкой формальных моделей языков, формальными лингвистическими моделями и математическим моделированием программных систем.

Глава 5. Промежуточные формы представления программ

Математически перевод определяется как множество пар, причём первый элемент принадлежит множеству объектов, которые требуется перевести, а второй элемент - множеству объектов, являющихся результатом перевода.

Компилятор в целом определяет перевод исходной программы, представленной в виде цепочки символов на языке, в объектный код. Если компиляцию рассматривать как многоэтапный процесс, то можно говорить о переводе, присущем каждому этапу компиляции. Например, в процессе лексического анализа исходная программа преобразуется из цепочки символов в цепочку лексических единиц (лексем), а на этапе синтаксического анализа цепочка лексем преобразуется в некоторую *промежуточную форму*, являющуюся линейной формой записи синтаксического дерева программы. Такой подход имеет определённые преимущества:

- перевод в промежуточную форму позволяет не учитывать особенности целевого языка, которые значительно усложняют перевод;
- упрощается перенос на другие целевые машины, для чего потребуется только реализовать преобразование промежуточной формы в язык новой машины;
- к программе в промежуточной форме можно применять алгоритмы машинно-независимой оптимизации.

Различные формы представления промежуточной программы отличаются друг от друга, в основном, способом соединения операторов и операндов. Общепринятыми промежуточными формами представления программы являются [2, 11, 28–30, 34, 45]:

- **польская инверсная запись;**
- **тетрады;**
- **триады;**
- **связные списочные структуры.**

В настоящее время ряд компиляторов в качестве промежуточной формы программы генерирует файлы, содержащие байт-код (Byte-code), представляющий собой инструкции для виртуальной машины Java (JVM – Java Virtual Machine). Поскольку ядро виртуальной машины Java реализовано практически для любого типа компьютеров, то файлы байт-кодов можно рассматривать как независимые от платформы приложения.

5.1. Польская запись

Одной из основных конструкций языков программирования является выражение, которое:

- может быть описано только рекурсивной самовложенной КС-грамматикой, т. к. оно включает в себя парные символы (скобки), т. е. по своей природе рекурсивно;

- содержит операции, имеющие различное число операндов и выполняющиеся в определенном порядке (приоритет операций);
- может содержать операнды различного типа, что требует дополнительного анализа при переводе.

Перечисленные особенности, с одной стороны, усложняют анализ и перевод выражений, требуют тщательного проектирования описания перевода, а с другой стороны, позволяют использовать выражения в качестве модели при рассмотрении проблем перевода.

5.1.1. Вычисление выражений

Порядок вычисления значения выражения определяется приоритетом операций и наличием в выражении скобок. Для простоты рассмотрим вычисление полноскобочного выражения, т. е. выражения, в котором при помощи дополнительных скобок явно определен порядок выполнения операций. Например выражение

$$A + B - C + D * E$$

в полноскобочной форме может быть записано как

$$(((A + B) - C) + (D * E)).$$

При выполнении вычислений необходимо найти самую внутреннюю пару скобок, выполнить операцию, удалить использованную пару скобок, заменив её вычисленным значением, и повторять этот процесс, пока есть скобки.

Замечание

Если каждой левой скобке придать вес +1, а каждой правой – вес -1, то процесс нахождения выполняемой на данном шаге операции упростится: нужно будет просмотреть выражение и найти максимум веса, который соответствует самой внутренней левой скобке.

Существуют более простые рекурсивные алгоритмы вычисления выражений [2, 11, 28].

Польский математик Ян Лукашевич обнаружил, что при использовании функциональной записи выражения, в которой операция предшествует своим операндам (а не записывается между ними), скобки становятся излишними.

Пример 5.1

Бинарная операция сложения обычно записывается в виде $A + B$. Её функциональная запись – это $+(A, B)$. При рассмотрении выражений, содержащих только бинарные операции скобки и запятую в записи можно опустить, тогда сложение можно записать как $+AB$.

5.1.2 Префиксная форма

Префиксная форма (или польская запись) формально может быть определена следующим образом:

1. Всякая переменная или константа есть выражение.

2. Если θ_1 – знак унарной (одноместной) операции, а α – выражение, то $\theta_1\alpha$ является выражением.
3. Если θ_2 – знак бинарной (двухместной) операции, а α_1 и α_2 – выражения, то $\theta_2\alpha_1\alpha_2$ является выражением.
4. Если θ_n – знак n -местной операции, а $\alpha_1, \alpha_2, \dots, \alpha_n$ являются выражениями, то $\theta_n\alpha_1\alpha_2 \dots \alpha_n$ – выражение.
5. Других выражений не существует.

Пример 5.2

Выражение языка ALGOL-60 $(A + B) * C \uparrow (D / (E + F))$, где $\uparrow$ – обозначение операции возведения в степень, можно представить в префиксной форме как $* +AB \uparrow C / D + EF$.

Префиксная форма не нарушает порядка следования операндов в выражении:

$A + B$ записывается как $+AB$

$B + A$ записывается как $+BA$

т. е. эта форма может быть использована как для записи коммутативных, так и некоммутативных операций. Кроме того, в отличие от обычной алгебраической записи выражений префиксная форма однозначно определяется порядок выполнения операций. Например, выражение $A + B + C + D$ можно интерпретировать по-разному (см. первый столбец табл. 10.1), а префиксная запись выражения всегда однозначно определяет порядок выполнения операций (см. второй столбец табл. 10.1).

Таблица 5.1. Полноскобочная и префиксная формы выражения

Полноскобочная форма выражения	Префиксная форма выражения
$((A + B) + C) + D$	$+ + +ABCD$
$((A + B) + (C + D))$	$+ + AB + CD$
$((A + (B + C)) + D)$	$+ + A + BCD$
$(A + ((B + C) + D))$	$+A + +BCD$
$(A + (B + (C + D)))$	$+A + B + CD$

Замечание Обычно в языках программирования операция сложения выполняется слева направо.

Префиксная форма довольно редко используется при проектировании трансляторов это связано с тем, что преобразование выражения в префиксную запись требует его просмотра справа налево, а в большинстве трансляторов просмотр и интерпретация входной программы выполняется слева направо.

5.1.3. Постфиксная форма

В компиляторах чаще всего используют постфиксную форму, в котором операции записываются справа от операндов. Постфиксная форма (польская инверсная запись – ПОЛИЗ) определяется следующими правилами:

1. Всякая переменная или константа есть выражение.
2. Если θ_1 – знак унарной (одноместной) операции, а α – выражение, то $\alpha\theta_1$ является выражением.
3. Если θ_2 – знак бинарной (двухместной) операции, а α_1 и α_2 – выражения, то $\alpha_1\alpha_2\theta_2$ является выражением.
4. Если θ_n – знак n -местной операции, а $\alpha_1, \alpha_2, \dots, \alpha_n$ являются выражениями, то $\alpha_1\alpha_2 \dots \alpha_n\theta_n$ – выражение.
5. Других выражений не существует.

Замечание Изображение операции “унарный минус” совпадает с изображением операции “бинарный минус”, то унарный минус можно представлять двумя способами: либо записывать его как бинарный оператор, т. е. $-B$ представлять в виде $0 - B$, либо для унарного минуса ввести новый символ операции, например символ ‘!’.

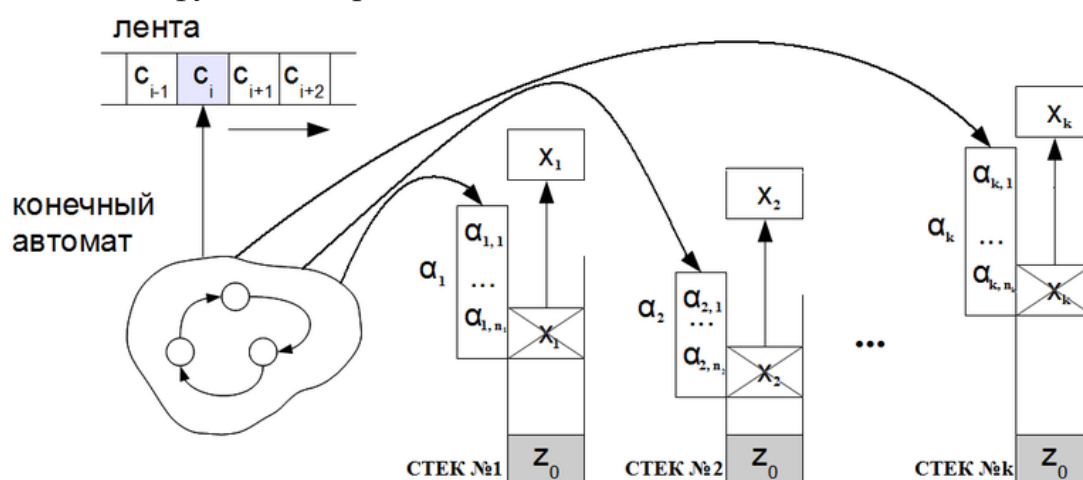
Пример 5.3

Выражение языка ALGOL-60 $(A + B) * C \uparrow (D / (E + F))$, где $\uparrow$ – обозначение операции возведения в степень, можно представить в польской инверсной записи как $AB + CDEF + / \uparrow *$.

Определение. k -стековой машиной называется набор

$A = \langle \Sigma, \Gamma, Q, s \in Q, T \subset Q, z_0 \in \Gamma, \delta: Q \times \Sigma \cup \{\varepsilon\} \times \Gamma^k \rightarrow P(Q \times (\Gamma^*)^k) \rangle$, где

- Σ — входной алфавит на ленте;
- Γ — стековый алфавит;
- Q — множество состояний автомата;
- s — стартовое состояние автомата;
- T — множество допускающих состояний автомата;
- z_0 — маркер дна стека;
- δ — функция переходов.



Пример 5.4

3-стековый автомат для перевода на основе алгоритма
Выражение для перевода находится на входной ленте

$A = \langle \Sigma, \Gamma, Q, s, T, z_0, \delta \rangle$, где
 $\Sigma = \{ '(', ')', +, -, *, /, 1, \dots, n \}$,
 $\Gamma = \{ '(', ')', +, -, *, /, 1, \dots, n, \perp \}$,
 $Q = \{ q_1, q_2, q_3, q_4 \}$,
 $s = q_1$,
 $T = \{ q_4 \}$,
 $z_0 = \perp$,
 $\delta = \{$
 1. $\delta(q_1, a, \varepsilon, \varepsilon, \varepsilon) = \{ q_1, \varepsilon, a, \varepsilon \}$, $a \in \Sigma$,
 2. $\delta(q_1, \varepsilon, \varepsilon, \varepsilon, \varepsilon) = \{ q_2, \varepsilon, \varepsilon, \varepsilon \}$,
 3. $\delta(q_2, \varepsilon, \varepsilon, a, \varepsilon) = \{ q_2, a, \varepsilon, \varepsilon \}$, $a \in \Gamma / \{ \perp \}$,
 4. $\delta(q_2, \varepsilon, \varepsilon, \perp, \varepsilon) = \{ q_3, \varepsilon, \perp, \varepsilon \}$,
 5. $\delta(q_3, \varepsilon, a, \varepsilon, \varepsilon) = \{ q_3, \varepsilon, a, \varepsilon \}$, $a \in \{ 1, \dots, n \}$,
 6. $\delta(q_3, \varepsilon, a, \varepsilon, b) = \{ q_3, a, b, \varepsilon \}$, $a \in \{ +, - \}$, $b \in \{ +, -, *, / \}$,
 7. $\delta(q_3, \varepsilon, a, \varepsilon, b) = \{ q_3, a, b, \varepsilon \}$, $a \in \{ *, / \}$, $b \in \{ *, / \}$,
 8. $\delta(q_3, \varepsilon, a, \varepsilon, \varepsilon) = \{ q_3, \varepsilon, \varepsilon, a \}$, $a \in \{ +, -, *, / \}$,
 9. $\delta(q_3, \varepsilon, '(', \varepsilon, \varepsilon) = \{ q_3, \varepsilon, \varepsilon, '(' \}$,
 10. $\delta(q_3, \varepsilon, ')', \varepsilon, a) = \{ q_3, ')', a, \varepsilon \}$, $a \in \Gamma / \{ '(' \}$,
 11. $\delta(q_3, \varepsilon, ')', \varepsilon, '(') = \{ q_3, \varepsilon, \varepsilon, \varepsilon \}$,
 12. $\delta(q_3, \varepsilon, \perp, \varepsilon, a) = \{ q_3, \perp, a, \varepsilon \}$, $a \in \{ +, -, *, / \}$,
 13. $\delta(q_3, \varepsilon, \perp, \varepsilon, \perp) = \{ q_4, \perp, \varepsilon, \perp \}$,
 14. $\delta(q_4, \varepsilon, \varepsilon, a, \varepsilon) = \{ q_4, \varepsilon, \varepsilon, a \}$, $a \in \Gamma / \{ \perp \}$
 $\}$

В начале все символы с ленты перекладываются в один из стеков, ещё один стек выступает в качестве вспомогательного, третий стэк заменяет выходную очередь алгоритма. Т.к. автомат работает именно с символами, то перевести в постфиксную запись он способен и примеры с переменными вместо чисел.

Выражение: $A + B - C + D * E$

Таблица 5.2. Процесс перевода

Лента	Стек 1	Стек 2	Стек 3	δ
$A + B - C + D * E$	$\perp$	$\perp$	$\perp$	1
Перекладывание в стек...				
ε	$\perp$	$E * D + C - B + A \perp$	$\perp$	3
Реверс стека...				
ε	$A + B - C + D * E \perp$	$\perp$	$\perp$	4
ε	$A + B - C + D * E \perp$	$\perp$	$\perp$	5
ε	$+ B - C + D * E \perp$	$A \perp$	$\perp$	8
ε	$B - C + D * E \perp$	$A \perp$	$+ \perp$	5
ε	$- C + D * E \perp$	$B A \perp$	$+ \perp$	6
ε	$- C + D * E \perp$	$+ B A \perp$	$\perp$	8
ε	$C + D * E \perp$	$+ B A \perp$	$- \perp$	5

ε	+ D * E ⊥	C + B A ⊥	- ⊥	6
ε	+ D * E ⊥	- C + B A ⊥	⊥	8
ε	D * E ⊥	- C + B A ⊥	+ ⊥	5
ε	* E ⊥	D - C + B A ⊥	+ ⊥	8
ε	E ⊥	D - C + B A ⊥	* + ⊥	5
ε	⊥	E D - C + B A ⊥	* + ⊥	12
ε	⊥	* E D - C + B A ⊥	+ ⊥	12
ε	⊥	+ * E D - C + B A ⊥	⊥	13
Реверс стека...				
ε	⊥	⊥	A B + C - D E * + ⊥	

В постфиксной записи: $A B + C - D E * +$

Польскую инверсную запись можно рассматривать как программу работы стековой машины, содержащую команды двух типов:

1. Чтение операнда из выражения вызывает запись его в магазин.
2. Чтение операции влечет за собой выполнение следующих действий:
 - выполнение операции, операнды для которой читаются из верхушки магазина;
 - запись результата операции в магазин.

При вычислении выражения, представленного в постфиксной записи, его элементы читаются слева направо.

Например, выражение $7\ 5\ 2\ * \ +$ соответствует последовательности действий приведённой в табл. 10.2 (рассматриваемый элемент выражения выделен полужирным шрифтом).

Таблица 5.3. Порядок вычисления выражения $7\ 5\ 2\ * \ +$

Выражение	Тип элемента	Действие	Магазин
7 5 2 * +	операнд	Записать в магазин операнд (число 7)	7 ⊥
7 5 2 * +	операнд	Записать в магазин операнд (число 5)	5 7 ⊥
7 5 2 * +	операнд	Записать в магазин операнд (число 2)	2 5 7 ⊥
7 5 2 * +	операция (*)	Прочитать из магазина операнд (число 2)	2 5 7 ⊥
		Прочитать из магазина операнд (число 5)	5 7 ⊥
		Выполнить операцию $2 * 5$	7 ⊥
		Записать в магазин результат операции (число 10)	10 7 ⊥
7 5 2 * +	операция (+)	Прочитать из магазина операнд (число 10)	10 7 ⊥
		Прочитать из магазина операнд число (7)	7 ⊥
		Выполнить операцию $10 + 7$	⊥
		Записать в магазин результат операции (число 17)	17 ⊥

Пример 5.5 Стековая машина для вычисления выражения в постфиксной записи.

Вычисление выражения в постфиксной записи проводится следующим образом: операнды кладутся в стек, при встрече оператора он применяется к двум верхним операндам стека, результат кладётся в стек.

$A = \langle \Sigma, \Gamma, Q, s, T, z_0, \delta, P \rangle$:

$Q = \{q, q_+, q_-, q_*, q_{/}\}, \Sigma = \{-n, \dots, -1, 0, 1, \dots, n, +, -, *, /\},$

$\Gamma = \{\perp, -n, \dots, -1, 0, 1, \dots, n\},$

$s = q,$

$z_0 = \perp,$

$T = \{q\},$

$\delta = \{$

1. $\delta(q, +, b a) = P(q_+, b a), \forall a, b \in \Gamma$

2. $\delta(q, -, b a) = P(q_-, a b), \forall a, b \in \Gamma$

3. $\delta(q, *, b a) = P(q_*, a b), \forall a, b \in \Gamma$

4. $\delta(q, /, b a) = P(q_{/}, a b), \forall a, b \in \Gamma$

5. $\delta(q, a, \varepsilon) = \{q, a\}, \forall a \in \Sigma / \{+, -, *, /\}$

$\}$

P можно задать таблично, например для q_+

+	1	2	3	...
1	{q, 2}	{q, 3}	{q, 4}	
2	{q, 3}	{q, 4}	{q, 5}	
3	{q, 4}	{q, 5}	{q, 6}	
...				...

Пример работы автомата: $1 + 2 - 3 + 4 * 5$

В ПОЛИЗ: $1\ 2\ +\ 3\ -\ 4\ 5\ *\ +$

Пусть $n = 30$:

$(q, 1\ 2\ +\ 3\ -\ 4\ 5\ *\ +, \perp) \vdash^5 (q, 2\ +\ 3\ -\ 4\ 5\ *\ +, 1\ \perp) \vdash^5 (q, +\ 3\ -\ 4\ 5\ *\ +, 2\ 1\ \perp) \vdash^1 (q, 3\ -\ 4\ 5\ *\ +, 3\ \perp) \vdash^5 (q, -\ 4\ 5\ *\ +, 3\ 3\ \perp) \vdash^2 (q, 4\ 5\ *\ +, 0\ \perp) \vdash^5 (q, 5\ *\ +, 4\ 0\ \perp) \vdash^5 (q, *\ +, 5\ 4\ 0\ \perp) \vdash^3 (q, +, 20\ 0\ \perp) \vdash^1 (q, \varepsilon, 20\ \perp)$

В результате работы автомата на вершине стека оказывается результат вычислений

Пример разработки транслирующей грамматики для трансляции алгоритма "устранение левой рекурсии" в форме конструктора множеств с оператором вывода " $\Rightarrow$ " в алгоритм с операторами типа foreach.

Алгоритм устранения левой рекурсии, заданный в виде алгоритма типа foreach:

```

P' = ∅
Pdel = ∅
V1 = V
Vr = ∅
foreach (Ai ∈ V | 1 ≤ i ≤ n)
    foreach (Aj ∈ V | 1 ≤ j ≤ i)
        foreach (p ∈ P | Ai → Ajγ)
            foreach (p ∈ P | {Aj → α1 | ... | αk} ∉ Pdel)
                P = P ∪ Ai → xmγ
                Pdel = Pdel ∪ Ai → Ajγ
            end
        end
    end
    foreach (p ∈ P | Ai → Aiγ)
        Vr = Vr ∪ Ai
    end
    foreach (Ai | Ai ∈ Vr)
        V1 = V1 ∪ Ai'
    end
    foreach (Ai → Aiα1 | ... | A1αk ∈ P | Ai → Aiα1 | ... | αk ∉ Pdel)
        P' = P' ∪ Ai' → α1 | ... | αk | α1Ai' | ... | αkAi'
        Pdel = Pdel ∪ Ai → Aiα1 | ... | Aiα1
    end
    foreach (Ai → β1 | ... | βm ∈ P | Ai → βi | ... | βm ∉ Pdel)
        P' = P' ∪ Ai → β1 | ... | βm | β1Ai' | ... | βmAi'
        P = P ∪ Ai → β1 | ... | βm | β1Ai' | ... | βmAi'
    end
    foreach (Ai ∉ Vr | R → xm ∈ { Ai → α1 | ... | αk }, R → xi ∉ Pdel)
        P' = P' ∪ xm
    end
end
end

```

Разработаем конструктор множеств для алгоритма устранения левой рекурсии, опираясь на определение конструктора множеств и описание алгоритма типа foreach:

$$\begin{aligned}
 &\{P' | P' = \emptyset, P_{del} = \emptyset, V_1 = V, V_r = \emptyset\} = \\
 &\{A_i \in V | 1 \leq i \leq n\} \Rightarrow \\
 &\{\{\{A_j \in V | 1 \leq j \leq i\} \Rightarrow \{p \in P | A_i \rightarrow A_j \gamma\} \Rightarrow \{p \in P | \{A_j \rightarrow \alpha_1 | \dots | \alpha_k\} \notin P_{del}\} \Rightarrow \{P \\
 &= P \cup A_i \rightarrow x_m \gamma, P_{del} = P_{del} \cup A_i \rightarrow A_j \gamma\}\}\}, \{p \in P | A_i \rightarrow A_i \gamma\} \Rightarrow \{V_r = V_r \cup \\
 &A_i\}\}, \{A_i | A_i \in V_r\} \Rightarrow \{V_1 = V_1 \cup A_i'\}\}, \{A_i \rightarrow A_i \alpha_1 | \dots | A_1 \alpha_k \in P | A_i \rightarrow A_i \alpha_1 | \dots | \alpha_k \\
 &\notin P_{del}\} \Rightarrow \{P' = P' \cup A_i' \rightarrow \alpha_1 | \dots | \alpha_k | \alpha_1 A_i' | \dots | \alpha_k A_i', P_{del} = P_{del} \cup A_i \rightarrow \\
 &A_i \alpha_1 | \dots | A_i \alpha_1\}\}, \{A_i \rightarrow \beta_1 | \dots | \beta_m \in P | A_i \rightarrow \beta_i | \dots | \beta_m \notin P_{del}\} \Rightarrow \{P' = P' \cup A_i \rightarrow \\
 &\beta_1 | \dots | \beta_m | \beta_1 A_i' | \dots | \beta_m A_i', P = P \cup A_i \rightarrow \beta_1 | \dots | \beta_m | \beta_1 A_i' | \dots | \beta_m A_i'\}\}, \{A_i \notin V_r | R \rightarrow x_m \in \{ \\
 &A_i \rightarrow \alpha_1 | \dots | \alpha_k\}, R \rightarrow x_i \notin P_{del}\} \Rightarrow \{P' = P' \cup x_m\}\}
 \end{aligned}$$

Анализ выполняемого перевода позволяет сделать вывод о том, что выходная цепочка включает в себя объекты (сущности) двух видов:

- Объекты, не изменяющиеся в процессе перевода (множество терминальных символов выходного языка.

- Объекты, генерируемые во время перевода {foreach, \n, \t, end}.

Так как объекты, включаемые в выходную цепочку, не передаются, то можно сделать вывод, что атрибуты в данном контексте не имеют смысла.

Обозначим терминальные строки, не изменяющиеся в процессе перевода, символами @i:

$$@_1 := P' = \emptyset$$

$$@_2 := P_{del} = \emptyset$$

$$@_3 := V_1 = V$$

$$@_4 := V_r = \emptyset$$

$$@_5 := A_i \in V \mid 1 \leq i \leq n$$

$$@_6 := A_j \in V \mid 1 \leq j \leq i$$

$$@_7 := p \in P \mid A_i \rightarrow A_j \gamma$$

$$@_8 := p \in P \mid \{A_j \rightarrow \alpha_1 \dots \alpha_k\} \notin P_{del}$$

$$@_9 := P = P \cup A_i \rightarrow x m \gamma$$

$$@_{10} := P_{del} = P_{del} \cup A_i \rightarrow A_j \gamma$$

$$@_{11} := p \in P \mid A_i \rightarrow A_i \gamma$$

$$@_{12} := V_r = V_r \cup A_i$$

$$@_{13} := A_i \mid A_i \in V_r$$

$$@_{14} := V_1 = V_1 \cup A'_i$$

$$@_{15} := A_i \rightarrow A_i \alpha_1 \dots \alpha_k \in P \mid A_i \rightarrow A_i \alpha_1 \dots \alpha_k \notin P_{del}$$

$$@_{16} := P' = P' \cup A'_i \rightarrow \alpha_1 \dots \alpha_k \alpha_1 A'_i \dots \alpha_k A'_i$$

$$@_{17} := P_{del} = P_{del} \cup A_i \rightarrow A_i \alpha_1 \dots \alpha_k$$

$$@_{18} := A_i \rightarrow \beta_1 \dots \beta_m \in P \mid A_i \rightarrow \beta_1 \dots \beta_m \notin P_{del}$$

$$@_{19} := P' = P' \cup A_i \rightarrow \beta_1 \dots \beta_m \beta_1 A'_i \dots \beta_m A'_i$$

$$@_{20} := P = P \cup A_i \rightarrow \beta_1 \dots \beta_m \beta_1 A'_i \dots \beta_m A'_i$$

$$@_{21} := A_i \notin V_r \mid R \rightarrow x_m \in \{A_i \rightarrow \alpha_1 \dots \alpha_k\}, R \rightarrow x_i \notin P_{del}$$

$$@_{22} := P' = P' \cup x_m$$

Транслирующей грамматикой (Т-грамматикой) называется КС-грамматика, множество терминальных символов которой разбито на множество входных символов и множество выходных символов, которые называются также символами действия.

Составим БНФ, которая описывает синтаксис заданного конструктора множеств:

(1)<алгоритм> ::= {<множество>|<терминальная строка>, <терминальная строка>, <терминальная строка>, <терминальная строка>} = {<терминальная строка>} => {<вывод конечного множества>}

(2)<вывод конечного множества> ::= {<добавление правил вида $A_i \rightarrow x \gamma$ и неиспользуемых правил>, {<добавление нетерминалов, имеющих левую рекурсию>, {<добавление дополнительных нетерминалов со штрихами>, {<добавление правил вида $A_i \rightarrow A_i \alpha_1 \dots \alpha_k$ >, {<добавление правил вида $A_i \rightarrow$

- $\beta_1|\dots|\beta_m\rangle$, {<добавление правил, если не было рекурсии>}
- (3)<добавление правил вида $A_i \rightarrow x\gamma$ и неиспользуемых правил> ::= {<терминальная строка>} => {<множество правил вида $A_i \rightarrow A_j\gamma$ >}
- (4)<множество правил вида $A_i \rightarrow A_j\gamma$ > ::= {<терминальная строка>} => {<множество правил вида $A_j \rightarrow \alpha_1|\dots|\alpha_k$ >}
- (5)<множество правил вида $A_j \rightarrow \alpha_1|\dots|\alpha_k$ > ::= {<терминальная строка>} => {<терминальная строка>, <терминальная строка>}
- (6)<добавление нетерминалов, имеющих левую рекурсию> ::= {<терминальная строка>} => {<терминальная строка>}
- (7)<добавление дополнительных нетерминалов со штрихами> ::= {<терминальная строка>} => {<терминальная строка>}
- (8)<добавление правил вида $A_i \rightarrow A_i\alpha_1|\dots|A_1\alpha_k$ > ::= {<терминальная строка>} => {<терминальная строка>, <терминальная строка>}
- (9)<добавление правил вида $A_i \rightarrow \beta_1|\dots|\beta_m$ > ::= {<терминальная строка>} => {<терминальная строка>, <терминальная строка>}
- (10)<добавление правил, если не было рекурсии> ::= {<терминальная строка>} => {<терминальная строка>}

Составим входную КС-грамматику

Введём обозначения: S - <алгоритм>, A - <вывод конечного множества>, B - <добавление правил вида $A_i \rightarrow x\gamma$ и неиспользуемых правил>, C - <добавление нетерминалов, имеющих левую рекурсию>, D - <добавление дополнительных нетерминалов со штрихами>, E - <добавление правил вида $A_i \rightarrow A_i\alpha_1|\dots|A_1\alpha_k$ >, F - <добавление правил вида $A_i \rightarrow \beta_1|\dots|\beta_m$ >, G - <добавление правил, если не было рекурсии>. Тогда правила вывода КС-грамматики, описывающей синтаксис входного языка, имеют вид:

1. $S \rightarrow \{P' | @_1, @_2, @_3, @_4\} = \{ @_5 \} \Rightarrow \{A\}$
2. $A \rightarrow \{B\}, \{C\}, \{D\}, \{E\}, \{F\}, \{G\}$
3. $B \rightarrow \{ @_6 \} \Rightarrow \{H\}$
4. $H \rightarrow \{ @_7 \} \Rightarrow \{I\}$
5. $I \rightarrow \{ @_8 \} \Rightarrow \{ @_9, @_{10} \}$
6. $C \rightarrow \{ @_{11} \} \Rightarrow \{ @_{12} \}$
7. $D \rightarrow \{ @_{13} \} \Rightarrow \{ @_{14} \}$
8. $E \rightarrow \{ @_{15} \} \Rightarrow \{ @_{16}, @_{17} \}$
9. $F \rightarrow \{ @_{18} \} \Rightarrow \{ @_{19}, @_{20} \}$
10. $G \rightarrow \{ @_{21} \} \Rightarrow \{ @_{22} \}$

Из каждой Т-грамматики можно получить две обычных грамматики, одна из которых позволяет строить входные цепочки, а другая - выходные. Правила построения таких грамматик можно сформулировать следующим образом:

Если из правил транслирующей грамматики G_T удалить выходные символы, то получится входная грамматика $G_{Твх}$ для заданной грамматики. Если из правил заданной транслирующей грамматики удалить входные символы, то получится выходная грамматика $G_{Твых}$ для заданной транслирующей грамматики G_T . Язык, порождаемый грамматикой $G_{Твх}$, называется входным языком заданной

транслирующей грамматики, а язык порождаемый $G_{\text{ТВЫХ}}$ называется выходным языком заданной транслирующей грамматики $G_{\text{Т}}$.

Построим СУ-схему для заданного перевода

	Правило входной грамматики	Правило выходной грамматики
(1)	$S \rightarrow \{P' @_1, @_2, @_3, @_4\} = \{ @_5 \} \Rightarrow \{A\}$	$S \rightarrow @_1 \backslash n @_2 \backslash n @_3 \backslash n @_4 \backslash n \text{foreach}(@_5) \backslash n \backslash t A \backslash n \text{end}$
(2)	$A \rightarrow \{B\}, \{C\}, \{D\}, \{E\}, \{F\}, \{G\}$	$A \rightarrow \text{foreach}(B) \backslash n \backslash t C \backslash n \backslash t D \backslash n \backslash t E \backslash n \backslash t F \backslash n \backslash t G \backslash n \backslash t \text{end}$
(3)	$B \rightarrow \{ @_6 \} \Rightarrow \{H\}$	$B \rightarrow \text{foreach}(@_6) \backslash n \backslash t H \backslash n \backslash t \text{end}$
(4)	$H \rightarrow \{ @_7 \} \Rightarrow \{I\}$	$H \rightarrow \text{foreach}(@_7) \backslash n \backslash t \backslash t I \backslash n \backslash t \backslash t \text{end}$
(5)	$I \rightarrow \{ @_8 \} \Rightarrow \{ @_9, @_{10} \}$	$I = \text{foreach}(@_8) \backslash n \backslash t \backslash t \backslash t @_9 \backslash n \backslash t \backslash t \backslash t @_{10} \backslash n \backslash t \backslash t \backslash t \text{end}$
(6)	$C \rightarrow \{ @_{11} \} \Rightarrow \{ @_{12} \}$	$C = \text{foreach}(@_{11}) \backslash n \backslash t \backslash t @_{12} \backslash n \backslash t \text{end}$
(7)	$D \rightarrow \{ @_{13} \} \Rightarrow \{ @_{14} \}$	$D = \text{foreach}(@_{13}) \backslash n \backslash t \backslash t @_{14} \backslash n \backslash t \text{end}$
(8)	$E \rightarrow \{ @_{15} \} \Rightarrow \{ @_{16}, @_{17} \}$	$E = \text{foreach}(@_{15}) \backslash n \backslash t \backslash t @_{16} \backslash n \backslash t \backslash t @_{17} \backslash n \backslash t \text{end}$
(9)	$F \rightarrow \{ @_{18} \} \Rightarrow \{ @_{19}, @_{20} \}$	$F = \text{foreach}(@_{18}) \backslash n \backslash t \backslash t @_{19} \backslash n \backslash t \backslash t @_{20} \backslash n \backslash t \text{end}$
(10)	$G \rightarrow \{ @_{21} \} \Rightarrow \{ @_{22} \}$	$G = \text{foreach}(@_{21}) \backslash n \backslash t \backslash t @_{22} \backslash n \backslash t \text{end}$

Построим Т-Граматику, опираясь на данную СУ-схему:

В начале по заданной СУ-схеме создается транслирующая грамматика Г. Это всегда можно сделать, поскольку заданная СУ-схема должна быть простой. Если входная грамматика заданной СУ-схемы относится к классу LL(1) -грамматик, то и входная грамматика транслирующей грамматики также будет относиться к этому классу.

Контекстно-свободная грамматика, не содержащая аннулирующих правил, называется разделенной или простой, если выполняются следующие два условия:

1. Правая часть каждого правила начинается терминалом.
2. Если два правила имеют одинаковые левые части, то правые части этих правил должны начинаться различными терминальными символами.

Входная КС-грамматика удовлетворяет этим условиям, значит, она является LL(1) грамматикой, следовательно, Т-грамматика также.

Зададим транслирующую грамматику $G_{\text{Т}}$ по заданной СУ-схеме (в квадратных скобках входная часть, в фигурных выходная):

1. $S \rightarrow [\{P' | @_1, @_2, @_3, @_4\} = \{ @_5 \} \Rightarrow \{ @_1 \backslash n @_2 \backslash n @_3 \backslash n @_4 \backslash n \text{foreach}(@_5) \backslash n \backslash t A [] \} \backslash n \text{end}]$

2. $A \rightarrow [{}]<\text{foreach}(>B[],\{<\text{>}\backslash\text{n}\text{>}C[],\{<\text{>}\backslash\text{n}\text{>}D[],\{<\text{>}\backslash\text{n}\text{>}E[],\{<\text{>}\backslash\text{n}\text{>}F[],\{<\text{>}\backslash\text{n}\text{>}G[]\}<\text{n}\text{>}\text{end}>$
3. $B \rightarrow [{}@_6\} \Rightarrow \{<\text{foreach}(@_6)\backslash\text{n}\text{>}H[]\}<\text{n}\text{>}\text{end}>$
4. $H \rightarrow [{}@_7\} \Rightarrow \{<\text{foreach}(@_7)\backslash\text{n}\text{>}I[]\}<\text{n}\text{>}\text{end}>$
5. $I \rightarrow [{}@_8\} \Rightarrow \{<@_9, @_{10}\}<\text{foreach}(@_8)\backslash\text{n}\text{>}\text{t}\text{>}@_9\backslash\text{n}\text{>}\text{t}\text{>}@_{10}\backslash\text{n}\text{>}\text{t}\text{>}\text{end}>$
6. $C \rightarrow [{}@_{11}\} \Rightarrow \{<@_{12}\}<\text{foreach}(@_{11})\backslash\text{n}\text{>}\text{t}\text{>}@_{12}\backslash\text{n}\text{>}\text{end}>$
7. $D \rightarrow [{}@_{13}\} \Rightarrow \{<@_{14}\}<\text{foreach}(@_{13})\backslash\text{n}\text{>}\text{t}\text{>}@_{14}\backslash\text{n}\text{>}\text{end}>$
8. $E \rightarrow [{}@_{15}\} \Rightarrow \{<@_{16}, @_{17}\}<\text{foreach}(@_{15})\backslash\text{n}\text{>}\text{t}\text{>}@_{16}\backslash\text{n}\text{>}\text{t}\text{>}@_{17}\backslash\text{n}\text{>}\text{end}>$
9. $F \rightarrow [{}@_{18}\} \Rightarrow \{<@_{19}, @_{20}\}<\text{foreach}(@_{18})\backslash\text{n}\text{>}\text{t}\text{>}@_{19}\backslash\text{n}\text{>}\text{t}\text{>}@_{20}\backslash\text{n}\text{>}\text{end}>$
10. $G \rightarrow [{}@_{21}\} \Rightarrow \{<@_{22}\}<\text{foreach}(@_{21})\backslash\text{n}\text{>}\text{t}\text{>}@_{22}\backslash\text{n}\text{>}\text{end}>$

Обозначим:

$a := [{}P' | @_1, @_2, @_3, @_4\} = \{<@_5\} \Rightarrow \{<]; <a> := <@_1\backslash\text{n}\text{>}@_2\backslash\text{n}\text{>}@_3\backslash\text{n}\text{>}@_4\backslash\text{n}\text{>}\text{foreach}(@_5)\backslash\text{n}\text{>}; b := [{}]; := <\text{n}\text{>}\text{end}>; c := [{}]; <c> := <\text{foreach}(>; d := [{}]; <d> := <\text{n}\text{>}; e := [{}]; <e> := <\text{n}\text{>}\text{end}>; f := [{}@_6\} \Rightarrow \{<; <f> := <@_6\)\backslash\text{n}\text{>}\text{t}>; g := [{}@_7\} \Rightarrow \{<; <g> := <\text{foreach}(@_7)\backslash\text{n}\text{>}\text{t}\text{>}; h := [{}]; <h> := <\text{n}\text{>}\text{t}\text{>}\text{end}>; i := [{}@_8\} \Rightarrow \{<@_9, @_{10}\}<; <i> := <\text{foreach}(@_8)\backslash\text{n}\text{>}\text{t}\text{>}@_9\backslash\text{n}\text{>}\text{t}\text{>}@_{10}\backslash\text{n}\text{>}\text{t}\text{>}\text{end}>; j := [{}@_{11}\} \Rightarrow \{<@_{12}\}<; <j> := <\text{foreach}(@_{11})\backslash\text{n}\text{>}\text{t}\text{>}@_{12}\backslash\text{n}\text{>}\text{end}>; k := [{}@_{13}\} \Rightarrow \{<@_{14}\}<; <k> := <\text{foreach}(@_{13})\backslash\text{n}\text{>}\text{t}\text{>}@_{14}\backslash\text{n}\text{>}\text{end}>; l := [{}@_{15}\} \Rightarrow \{<@_{16}, @_{17}\}<; <l> := <\text{foreach}(@_{15})\backslash\text{n}\text{>}\text{t}\text{>}@_{16}\backslash\text{n}\text{>}\text{t}\text{>}@_{17}\backslash\text{n}\text{>}\text{end}>; m := [{}@_{18}\} \Rightarrow \{<@_{19}, @_{20}\}<; <m> := <\text{foreach}(@_{18})\backslash\text{n}\text{>}\text{t}\text{>}@_{19}\backslash\text{n}\text{>}\text{t}\text{>}@_{20}\backslash\text{n}\text{>}\text{end}>; n := [{}@_{21}\} \Rightarrow \{<@_{22}\}<; <n> := <\text{foreach}(@_{21})\backslash\text{n}\text{>}\text{t}\text{>}@_{22}\backslash\text{n}\text{>}\text{end}>; o := [{}]; <o> := <\text{n}\text{>}$

С учетом вышеперечисленных обозначений Т-грамматика будет выглядеть так:

1. $S \rightarrow a<a>Ab$
2. $A \rightarrow c<c>Bd<d>Co<o>Do<o>Eo<o>Fo<o>Ge<e>$
3. $B \rightarrow f<f>Hh<h>$
4. $H \rightarrow g<g>Ie<e>$
5. $I \rightarrow i<i>$
6. $C \rightarrow j<j>$
7. $D \rightarrow k<k>$
8. $E \rightarrow l<l>$
9. $F \rightarrow m<m>$
10. $G \rightarrow n<n>$

Построение транслирующей грамматики по СУ-схеме

ВХОД: $T = \{V_{\text{ТВХ}}, V_{\text{ТВЫХ}}, V_A, Q, <I>\}$

ВЫХОД: $\Gamma_T = \{V'_{\text{ТВХ}}, V'_{\text{ТВЫХ}}, V'_A, R, <I>\}$

foreach($A \rightarrow \alpha, \beta \in Q, \alpha = x_0 \langle A_0 \rangle \dots x_n \langle A_n \rangle, \beta = y_0 \langle A_0 \rangle \dots y_n \langle A_n \rangle$)
 $A \rightarrow x_0 \{y_0\} \langle A_0 \rangle \dots x_n \{y_n\} \langle A_n \rangle$
end

Рассмотрим построение выходной цепочки для входа, представляющего алгоритм устранения левой рекурсии в виде конструктора множеств:

$\{P' | @_1, @_2, @_3, @_4\} = \{ @_5 \} \Rightarrow \{ \{ @_6 \} \Rightarrow \{ \{ @_7 \} \Rightarrow \{ \{ @_8 \} \Rightarrow \{ @_9, @_{10} \} \} \} \}, \{ \{ @_{11} \} \Rightarrow \{ @_{12} \} \}, \{ \{ @_{13} \} \Rightarrow \{ @_{14} \} \}, \{ \{ @_{15} \} \Rightarrow \{ @_{16}, @_{17} \} \}, \{ \{ @_{18} \} \Rightarrow \{ @_{19}, @_{20} \} \}, \{ \{ @_{21} \} \Rightarrow \{ @_{22} \} \} \}$

С учетом обозначений выше эта цепочка будет выглядеть так:

acfgiehjdjokolomoneb

Преобразователем с магазинной памятью (МП) называется совокупность восьми объектов:

$МП = \{P, S, s_0, H, h_0, F, W, \phi\},$

где P - входной алфавит, состоящий из символов, записываемых на входную ленту; W - выходной алфавит, содержащий символы, записываемые на выходную ленту; H - магазинный алфавит, содержащий символы, записываемые и считываемые из магазина;

h_0 - маркер дна магазина, $h_0 \in H$;

S - множество состояний преобразователя; s_0 - начальное состояние из множества S ;

F - множество конечных состояний, представляющих собой подмножество S ;

ϕ - функция переходов преобразователя, которая задает отображение,

$\phi: S \times \{P \cup \{\epsilon\} \times H \cup S \times H^* \times W^*.$

Она может быть записана в функциональном виде:

$\phi(s, p, h) = (s', \beta, \gamma),$

где $h \in H, p \in P, \beta \in H^*, \gamma \in W^*$ и $s, s' \in S$.

Поскольку преобразователь должен в процессе формирования выхода осуществлять распознавание входной цепочки, он строится на основе транслирующей грамматики с использованием правил построения распознавателя. Преобразователь должен выполнять левый вывод входной цепочки в магазине и удалять терминальные символы, находящиеся в вершине, при совпадении их с очередным символом на входной ленте. Поскольку в магазине будут находиться и выходные символы, содержащиеся в правилах Т-грамматики дополним правила построения преобразователя следующим правилом: при появлении выходного символа в вершине магазина он передается на выход независимо от символа, находящегося под входной головкой.

Определим ы МП следующим образом:

$$S = \{s_0\}, P = V_{\text{ТВХ}}, H = \{V_{\text{ТВХ}}, V_A, \{h_0\}, V_{\text{ТВЫХ}}\}, F = \{s_0\}, W = V_{\text{ТВЫХ}}$$

Определим функции переходов МП:

(1) Для всех правил вида $A \rightarrow b\alpha$, где $b \in V_{\text{ТВХ}}$ и $\alpha \in (V_{\text{ТВХ}}, V_{\text{ТВЫХ}}, V_A)^*$, строятся команды:

$$\phi(s_0, b, A) = (s_0, \alpha', \$),$$

где α' - зеркальное отображение цепочки α .

(2) Для всех правил вида $A \rightarrow B\alpha$, где $B \in V_A$ и $\alpha \in (V_{\text{ТВХ}}, V_{\text{ТВЫХ}}, V_A)^*$, строятся команды:

$$\phi^*(s_0, u, A) = (s_0, \alpha'B, \$),$$

где $u \in \text{ВЫБОР}(A \rightarrow B\alpha_{\text{ВХ}})$ и $\alpha_{\text{ВХ}}$ - цепочка, полученная из α путем удаления из нее всех выходных символов.

(3) Для всех правил вида $A \rightarrow \$$ строятся команды:

$$\phi^*(s_0, u, A) = (s_0, \$, \$),$$

где $u \in \text{ВЫБОР}(A \rightarrow \$)$.

(4) Для всех символов b , принадлежащих $V_{\text{ТВХ}}$, стоящих не только на первом месте в правой части правил транслирующей грамматики, т.е. символов, заносимых в магазин, строятся команды:

$$\phi(s_0, b, b) = (s_0, \$, \$).$$

(5) Для всех выходных символов $\langle u \rangle$, таких что $u \in V_{\text{ТВЫХ}}$, строятся команды:

$$\phi^*(s_0, z, \langle u \rangle) = (s_0, \$, u),$$

где $z \in V_{\text{ТВХ}} \setminus \{\varepsilon\}$

Т.е. команды строятся для сочетаний $\langle u \rangle z$, таких, что z может следовать за $\langle u \rangle$ в цепочках, выводимых в заданной грамматике.

(6) Заключительная команда имеет вид:

$$\phi^*(s_0, \varepsilon, h_0) = (s_0, \$, \$).$$

Задание правил конфигурации МП по ТГ

Вход: $\Gamma_T = \{V_{TBX}, V_{TBYX}, V_A, R, <I>\}$

Выход: ϕ - множество правил МП

$\phi = \emptyset$

foreach ($A \rightarrow b\alpha \in R, b \in V_{TBX}, \alpha \in (V_{TBX} \cup V_{TBYX} \cup V_A)^*$)

$\phi = \phi \cup (\phi(s_0, b, A) = (s_0, \alpha', \$))$

end

foreach ($A \rightarrow B\alpha \in R, B \in V_A, \alpha \in (V_{TBX} \cup V_{TBYX} \cup V_A)^*$)

$\phi = \phi \cup (\phi^*(s_0, u, A) = (s_0, \alpha'B, \$))$

end

foreach ($A \rightarrow \$ \in R$)

$\phi = \phi \cup (\phi^*(s_0, u, A) = (s_0, \$, \$))$

end

foreach ($b \in V_{TBX} \mid A \rightarrow \beta b \gamma$)

$\phi = \phi \cup (\phi(s_0, b, b) = (s_0, \$, \$))$

end

foreach ($\{u\} \mid u \in V_{TBYX}$)

$\phi = \phi \cup (\phi^*(s_0, z, <u>) = (s_0, \$, u))$

end

$\phi = \phi \cup (\phi^*(s_0, \varepsilon, h_0) = (s_0, \$, \$))$

Используя правило построения команд преобразователя (1) для правил грамматики, начинающихся входным терминальным символом, получаем команды преобразователя МП₁:

1. $\phi(s_0, a, S) = (s_0, bA<a>, \$)$
2. $\phi(s_0, c, A) = (s_0, <e>eG<o>oF<o>oE<o>oD<o>oC<d>dB<c>, \$)$
3. $\phi(s_0, f, B) = (s_0, <h>hH<f>, \$)$
4. $\phi(s_0, g, H) = (s_0, <e>eI<g>, \$)$
5. $\phi(s_0, i, I) = (s_0, <i>, \$)$
6. $\phi(s_0, j, C) = (s_0, <j>, \$)$
7. $\phi(s_0, k, D) = (s_0, <k>, \$)$
8. $\phi(s_0, l, E) = (s_0, <l>, \$)$
9. $\phi(s_0, m, F) = (s_0, <m>, \$)$
10. $\phi(s_0, n, G) = (s_0, <n>, \$)$

Правила построения команд вида (2),(3) к заданной грамматике неприменимы, поэтому с помощью правил (4) и (5) построим команды, обеспечивающие передачу выходных символов на выход. Эти команды имеют вид:

11. $\phi^*(s_0, c, <a>) = (s_0, \$, <a>)$
12. $\phi^*(s_0, f, <c>) = (s_0, \$, <c>)$
13. $\phi^*(s_0, g, <f>) = (s_0, \$, <f>)$
14. $\phi^*(s_0, i, <g>) = (s_0, \$, <g>)$

15. $\phi^*(s_0, e, \langle i \rangle) = (s_0, \$, \langle i \rangle)$
16. $\phi^*(s_0, e, e) = (s_0, \$, \$)$
17. $\phi^*(s_0, h, \langle e \rangle) = (s_0, \$, \langle e \rangle)$
18. $\phi^*(s_0, h, h) = (s_0, \$, \$)$
19. $\phi^*(s_0, d, \langle h \rangle) = (s_0, \$, \langle h \rangle)$
20. $\phi^*(s_0, d, d) = (s_0, \$, \$)$
21. $\phi^*(s_0, j, \langle d \rangle) = (s_0, \$, \langle d \rangle)$
22. $\phi^*(s_0, o, \langle j \rangle) = (s_0, \$, \langle j \rangle)$
23. $\phi^*(s_0, o, o) = (s_0, \$, \$)$
24. $\phi^*(s_0, k, \langle o \rangle) = (s_0, \$, \langle o \rangle)$
25. $\phi^*(s_0, o, \langle k \rangle) = (s_0, \$, \langle k \rangle)$
26. $\phi^*(s_0, l, \langle o \rangle) = (s_0, \$, \langle o \rangle)$
27. $\phi^*(s_0, o, \langle l \rangle) = (s_0, \$, \langle l \rangle)$
28. $\phi^*(s_0, m, \langle o \rangle) = (s_0, \$, \langle o \rangle)$
29. $\phi^*(s_0, o, \langle m \rangle) = (s_0, \$, \langle m \rangle)$
30. $\phi^*(s_0, n, \langle o \rangle) = (s_0, \$, \langle o \rangle)$
31. $\phi^*(s_0, e, \langle n \rangle) = (s_0, \$, \langle n \rangle)$
32. $\phi^*(s_0, e, e) = (s_0, \$, \$)$
33. $\phi^*(s_0, b, \langle e \rangle) = (s_0, \$, \langle e \rangle)$
34. $\phi^*(s_0, b, b) = (s_0, \$, \$)$
35. $\phi^*(s_0, \varepsilon, \langle b \rangle) = (s_0, \$, \langle b \rangle)$

Для перехода в заключительное состояние s_1 в соответствии с правилом (6) построим команду:

$$36. \phi^*(s_0, \varepsilon, h_0) = (s_0, \$, \$)$$

Алгоритм трансляции

```

foreach ( $a_i \in P$ )
    while ( $a_i \neq h_0$ )
        if ( $a_i = \varepsilon$ )
             $a_i = h_0$ 
        end
        if ( $a_i \in V_{TBX}, h \in V_A, \rho h \in H^*$ )
             $h = \alpha'$ 
        end

```

```

    if ( $a_i \in V_{\text{ТВХ}} \cup \{\varepsilon\}$ ,  $h = \langle u \rangle$ ,  $u \in V_{\text{ТВЫХ}}$ )
         $W = W \cup \langle u \rangle$ 
    end
    if ( $a_i = h$ )
         $\rho h = \rho$ 
    end
end
end
end

```

Последовательность конфигураций, получаемых с помощью команд преобразователя, имеет вид:

```

( $s_0$ , acfgiehdjokolomoneb $\varepsilon$ ,  $h_0S$ ,  $\$$ )  $\vdash^1$ 
( $s_0$ , cfgiehdjokolomoneb $\varepsilon$ ,  $h_0\langle b \rangle bA\langle a \rangle$ ,  $\$$ )  $\vdash^{11}$ 
( $s_0$ , cfgiehdjokolomoneb $\varepsilon$ ,  $h_0\langle b \rangle bA$ ,  $\langle a \rangle$ )  $\vdash^2$ 
( $s_0$ , fgiehdjokolomoneb $\varepsilon$ ,  $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle oC\langle d \rangle dB\langle c \rangle$ ,  $\langle a \rangle$ )
 $\vdash^{12}$ 
( $s_0$ , fgiehdjokolomoneb $\varepsilon$ ,  $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle oC\langle d \rangle dB$ ,  $\langle a \rangle \langle c \rangle$ )
 $\vdash^3$ 
( $s_0$ , giehdjokolomoneb $\varepsilon$ ,  $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle oC\langle d \rangle d\langle h \rangle hH\langle f \rangle$ ,
 $\langle a \rangle \langle c \rangle$ )  $\vdash^{13}$ 
( $s_0$ , giehdjokolomoneb $\varepsilon$ ,  $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle oC\langle d \rangle d\langle h \rangle hH$ ,
 $\langle a \rangle \langle c \rangle \langle f \rangle$ )  $\vdash^4$ 
( $s_0$ , iehdjokolomoneb $\varepsilon$ ,
 $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle oC\langle d \rangle d\langle h \rangle h\langle e \rangle eI\langle g \rangle$ ,  $\langle a \rangle \langle c \rangle \langle f \rangle$ )  $\vdash^{14}$ 
( $s_0$ , iehdjokolomoneb $\varepsilon$ ,  $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle oC\langle d \rangle d\langle h \rangle h\langle e \rangle eI$ ,
 $\langle a \rangle \langle c \rangle \langle f \rangle \langle g \rangle$ )  $\vdash^5$ 
( $s_0$ , ehdjokolomoneb $\varepsilon$ ,  $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle oC\langle d \rangle d\langle h \rangle h\langle e \rangle e\langle i \rangle$ ,
 $\langle a \rangle \langle c \rangle \langle f \rangle \langle g \rangle$ )  $\vdash^{15}$ 
( $s_0$ , ehdjokolomoneb $\varepsilon$ ,  $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle oC\langle d \rangle d\langle h \rangle h\langle e \rangle e$ ,
 $\langle a \rangle \langle c \rangle \langle f \rangle \langle g \rangle \langle i \rangle$ )  $\vdash^{16}$ 
( $s_0$ , hdjokolomoneb $\varepsilon$ ,  $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle oC\langle d \rangle d\langle h \rangle h\langle e \rangle$ ,
 $\langle a \rangle \langle c \rangle \langle f \rangle \langle g \rangle \langle i \rangle$ )  $\vdash^{17}$ 
( $s_0$ , hdjokolomoneb $\varepsilon$ ,  $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle oC\langle d \rangle d\langle h \rangle h$ ,
 $\langle a \rangle \langle c \rangle \langle f \rangle \langle g \rangle \langle i \rangle \langle e \rangle$ )  $\vdash^{18}$ 
( $s_0$ , djokolomoneb $\varepsilon$ ,  $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle oC\langle d \rangle d\langle h \rangle$ ,
 $\langle a \rangle \langle c \rangle \langle f \rangle \langle g \rangle \langle i \rangle \langle e \rangle$ )  $\vdash^{19}$ 
( $s_0$ , djokolomoneb $\varepsilon$ ,  $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle oC\langle d \rangle d$ ,
 $\langle a \rangle \langle c \rangle \langle f \rangle \langle g \rangle \langle i \rangle \langle e \rangle \langle h \rangle$ )  $\vdash^{20}$ 
( $s_0$ , jokolomoneb $\varepsilon$ ,  $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle oC\langle d \rangle$ ,
 $\langle a \rangle \langle c \rangle \langle f \rangle \langle g \rangle \langle i \rangle \langle e \rangle \langle h \rangle$ )  $\vdash^{21}$ 
( $s_0$ , jokolomoneb $\varepsilon$ ,  $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle oC$ ,
 $\langle a \rangle \langle c \rangle \langle f \rangle \langle g \rangle \langle i \rangle \langle e \rangle \langle h \rangle \langle d \rangle$ )  $\vdash^6$ 
( $s_0$ , okolomoneb $\varepsilon$ ,  $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle o\langle j \rangle$ ,
 $\langle a \rangle \langle c \rangle \langle f \rangle \langle g \rangle \langle i \rangle \langle e \rangle \langle h \rangle \langle d \rangle$ )  $\vdash^{22}$ 
( $s_0$ , okolomoneb $\varepsilon$ ,  $h_0\langle b \rangle b\langle e \rangle eG\langle o \rangle oF\langle o \rangle oE\langle o \rangle oD\langle o \rangle o$ ,
 $\langle a \rangle \langle c \rangle \langle f \rangle \langle g \rangle \langle i \rangle \langle e \rangle \langle h \rangle \langle d \rangle \langle j \rangle$ )  $\vdash^{23}$ 

```

5.1.3.1. Свойства ПОЛИЗ

Польская инверсная запись обладает следующими свойствами:

- однозначно определяет порядок выполнения операций;
- имеет простую синтаксическую структуру.

Свяжем с каждым элементом e_i польской инверсной записи выражения вес p в соответствии со следующими правилами:

- если элемент e_i – переменная или константа, то $p(e_i) = 1$;
- если элемент e_i – знак унарной операции, то $p(e_i) = 0$;
- если элемент e_i – знак бинарной операции, то $p(e_i) = -1$;
- если элемент e_i – знак n -местной операции, то $p(e_i) = -(n - 1)$;

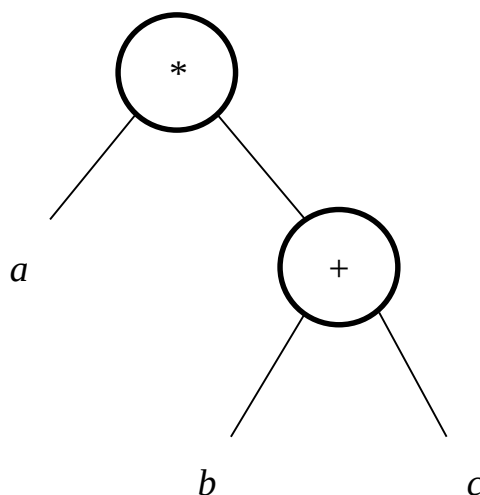
и определим функцию $P(k) = \sum p(i)$. Тогда для каждого выражения справедливы утверждения:

- $P(i) > 0$ для $1 < i < k$;
- $P(n) = 1$ для правильного выражения из n элементов.

5.1.3.2. Графическое представление выражений

Выражение можно изобразить в виде дерева. Пусть узлы дерева представляют собой операции, а ветви – операнды. Для бинарных операций левая ветвь соответствует левому операнду, а правая ветвь – правому операнду, причём в каждой ветви дерева узлы, которые лежат ниже, соответствуют операциям, которые выполняются раньше.

Например, выражения $a * (b + c)$ соответствует дерево, изображенное на рис.



10.1

Рис. 10.1. Дерево выражения $a * (b + c)$

Префиксная форма этого выражения равна $* a + bc$, а постфиксная – $abc + *$.

Интересно, что все три формы представления выражения (инфиксная, префиксная и постфиксная) являются, по существу, различными формами представления или обхода построенного бинарного дерева. Можно убедиться, что префиксная польская запись выражения $* + abc$ – это левое скобочное

представление дерева, из которого удалены скобки, или *прямой обход* дерева. Аналогично, постфиксная запись $abc + *$ – это правое скобочное представление дерева, из которого удалены все скобки, или *обратный обход* дерева.

5.1.4. ПОЛИЗ как промежуточный язык

Польская инверсная запись выражений имеет два важных свойства, которые позволяют использовать ее не только для представления выражений, но и в качестве промежуточного языка для языковых процессов:

1. Действия, описываемые в ПОЛИЗ, можно выполнять (или программировать) в процессе ее одностороннего безвозвратного просмотра слева направо, т. к. операнды в ПОЛИЗ всегда предшествуют знаку операции.
2. Операнды ПОЛИЗ расположены в том же порядке, что в исходном (инфиксном) выражении. Перевод выражения в ПОЛИЗ сводится только к изменению порядка следования знаков операций.

Расширение ПОЛИЗ для представления других конструкций языков программирования (переменных с индексами, указателей функций, условных выражений, основных операторов языка программирования) выполняется очень просто: нужно придерживаться одного правила – *знак оператора должен следовать непосредственно за соответствующими ему операндами*.

Замечание “вычисление выражения” означает вычисление численного значения выражения в трансляторах интерпретирующего типа, так и *процесс анализа выражения и построения для него объектного кода* для компиляторов.

5.1.4.1. Элементы массивов

Пусть требуется вычислить выражение:

$$(a + b[i, j + 1]) * c + d,$$

в котором $b[i, j + 1]$ – элемент двухмерного массива.

Для представления в постфиксной записи элементов массива необходимо определить дополнительную операцию ПОЛИЗ, вычисляющую, *адрес элемента массива* (код операции SUBS). Операндами операции SUBS являются имя массива и значения индексов. Число операндов этой операции переменное. Оно зависит от размерности массива и определяется по формуле $k = n + 1$, где n – размерность массива. Операнды операции SUBS должны располагаться в определенном порядке: имя массива, значения индексных выражений и константа k (целое без знака), определяющая число операндов. С учётом представления элементов массива ПОЛИЗ рассматриваемого выражения имеет вид:

$$a \ b \ i \ j \ 1 + 3 \ SUBS + c * d +.$$

Графическое представление этого выражения приведено на рис. 10.2.

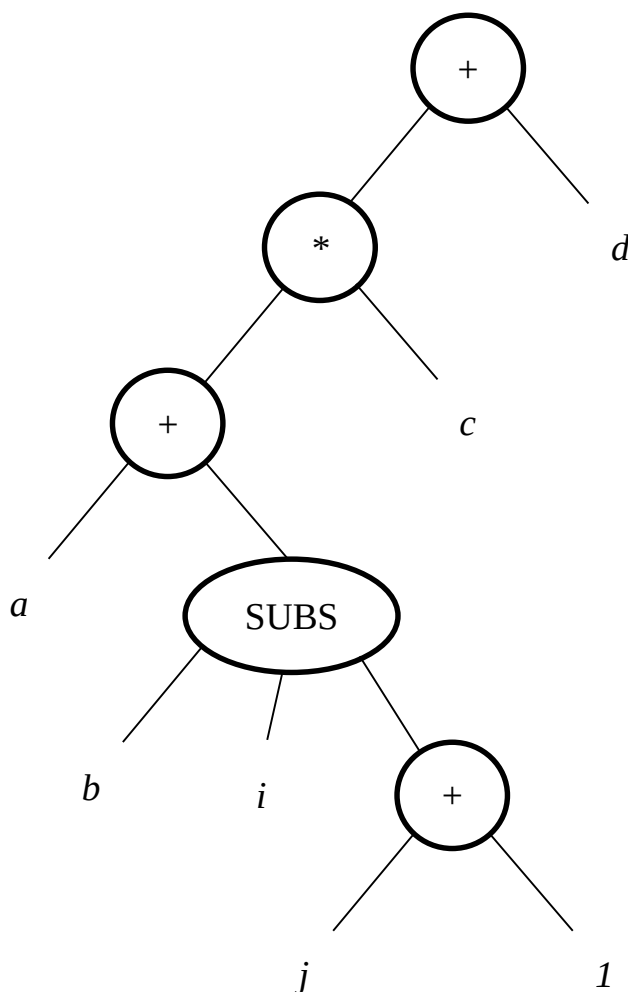


Рис. 10.2. Дерево выражения $(a + b[i, j + 1]) * c + d$

Замечание

При представлении в ПОЛИЗ элементов массивов можно явно не указывать число операндов операции SUBS. В этом случае имя массива должно располагаться непосредственно перед кодом операции, а информация о размерности массива, определяющая число операндов операции, должна храниться в другом месте, например, в таблице идентификаторов языкового процессора.

5.1.4.2. Указатели функций

С точки зрения представления в польской инверсной записи указатель функции (оператор обращения к функции) мало чем отличается от переменной с индексом. Для представления указателя функции введем в ПОЛИЗ дополнительную операцию CALL, имеющую переменное число операндов, зависящее от числа аргументов в вызове функции. Количество операндов операции CALL определяется выражением:

$$k = n + 1,$$

где n – число аргументов функции.

Например, ПОЛИЗ выражения $a + f(i, j + 1)$ имеет вид $a f i j 1 + 3 \text{ CALL} +$ (рис. 10.3).

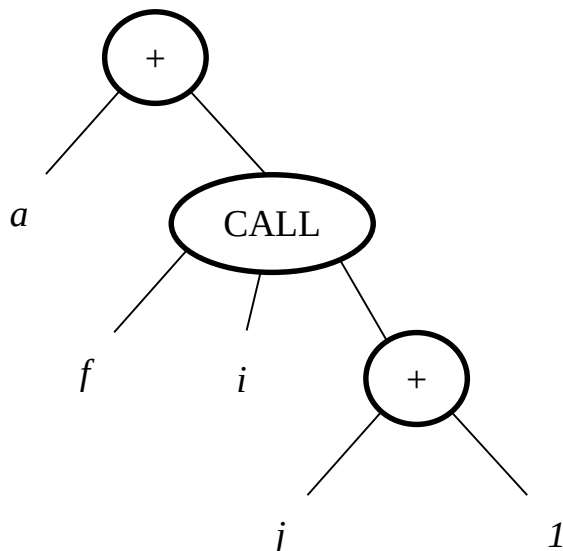


Рис. 10.3. Дерево выражения $a + f(i, j + 1)$

5.1.4.3. Условные выражения

В рассмотренных ранее выражениях порядок выполнения операций определялся приоритетом операций и скобками. В некоторых языках программирования имеются *условные выражения*, в которых порядок выполнения операций и значения операндов определяются *динамически* в зависимости от некоторых условий. Например, в языке C++ выражения $a > b ? 1 : 2$ принимает значение 1, если $a > b$, и значение 2 в противном случае. Условное арифметическое выражение $x + (\text{if } a > b \text{ then } 1 \text{ else } 2)$ языка ALGOL-60 принимает значение $x + 1$, если $a > b$, и значение $x + 2$ в противном случае.

В приведенных примерах значения операндов при вычислении выражения изменяются динамически в зависимости от значения условия $a > b$.

Для обеспечения возможности представления условных выражений добавим в польскую инверсную запись следующие объекты:

- метки, которые могут помечать некоторые элементы ПОЛИЗ и использоваться для динамического изменения хода вычислений;
- дополнительные операции:
 - BF – бинарная операция “Условный переход по значению ложь”. Выражение $a \text{ } t \text{ } \text{BF}$ означает, что при a равном значению “ложь” необходимо перейти к рассмотрению элемента ПОЛИЗ, помеченного меткой t , в противном случае – перейти к рассмотрению очередного элемента ПОЛИЗ;
 - BRL – унарная операция “Безусловный переход”, операндом которой является метка элемента, который будет рассматриваться следующим;

- DEFL – унарная операция “Определить метку”, операндом которой является метка, которую необходимо поместить перед некоторым элементом ПОЛИЗ.

Используя введенные допущения, условное выражение $x + (\text{if } a > b \text{ then } 1 \text{ else } 2)$ можно представить в виде:

$x \ a \ b > m_1 \text{ BF } 1 \ m_2 \text{ BRL } m_1 \text{ DEFL } 2 \ m_2 \text{ DEFL } +$

На рис 10.4 приведён порядок использования элементов ПОЛИЗ при вычислении рассмотренного условного выражения: верхняя стрелка соответствует случаю $a > b$, а нижние стрелки – $a \leq b$.

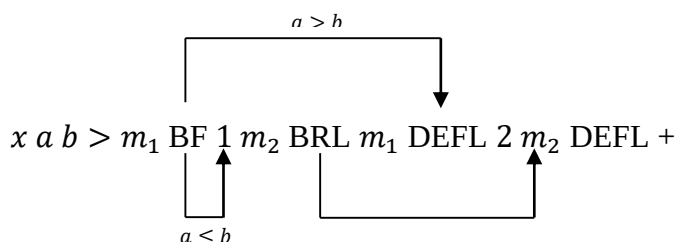


Рис. 5.4. ПОЛИЗ выражения $x + (\text{if } a > b \text{ then } 1 \text{ else } 2)$

Для представления условного выражения с помощью дерева требуется ввести пустые узлы, метки и дополнительные операции (BF, BRL, DEFL). Пустые узлы соответствуют разветвлению вычислительного процесса и вводятся для того, чтобы число ветвей, исходящих из узла, было равно числу операндов операции. На рис 10.5 приведено дерево условного выражения $x + (\text{if } a > b \text{ then } 1 \text{ else } 2)$, при обратном обходе которого получается приведенное ранее представление выражения в ПОЛИЗ.

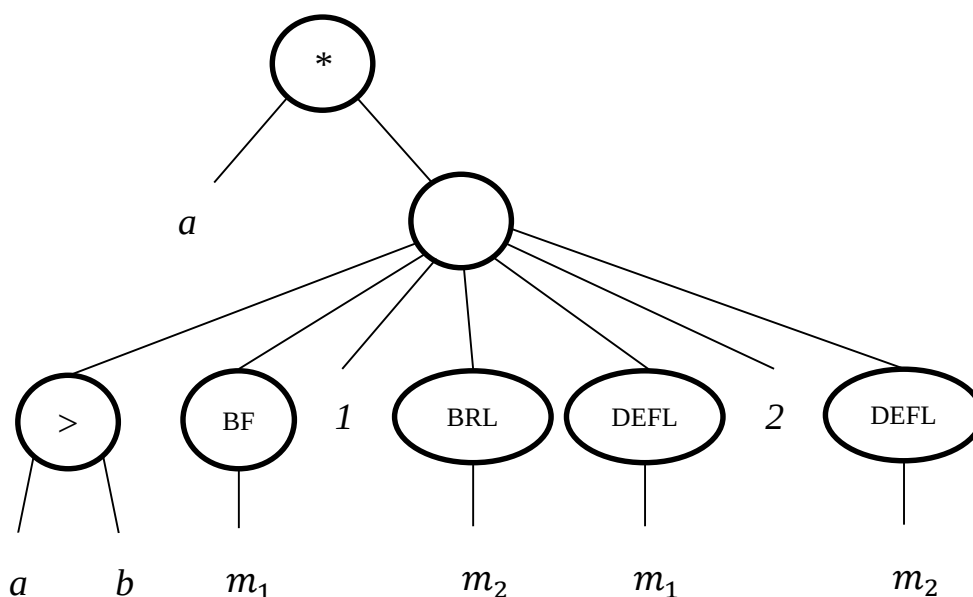


Рис. 10.5. Дерево условного выражения $x + (\text{if } a > b \text{ then } 1 \text{ else } 2)$

Замечание

Включение дополнительных объектов, отражающих динамические свойства условного выражения, усложняет построение дерева и его интерпретацию.

5.1.4.4. Оператор присваивания

Для представления оператора присваивания в польской инверсной записи требуется ввести бинарную операцию “Присвоить значение” ($:=$). Левый операнд операции определяет объект, которому нужно присвоить значение, а правый – вычисленное значение. Например, оператор присваивания языка Паскаль $a[j, k + 1] := \text{sqr}(m[j]) / 2$ может быть записан в ПОЛИЗ в виде:

$a\ j\ k\ 1\ +\ 3\ \text{SUBS}\ \text{sqr}\ m\ j\ 2\ \text{SUBS}\ 2\ \text{CALL}\ 2\ /\ :=.$

5.1.4.5. Условный оператор

Используя результаты, полученные при рассмотрении условного выражения, условный оператор языка C++, имеющий формат:

if (<выражение>) <оператор1> **else** <оператор2> ,

может быть представлен в польской инверсной записи следующим образом:

<выражение> m_1 BF <оператор1> m_2 BRL m_1 DEFL <оператор2> m_2
DEFL

В табл. 10.3 приведено представление основных операций, включенных в большинство языков программирования, в польской инверсной записи. Используя эти операции, можно разработать представление, других, более сложных операторов языков программирования. При необходимости перечисленные операции могут быть модифицированы, а их список расширен.

Таблица 5.3. Представление основных операций в ПОЛИЗ

Оператор	Код операции	Операнды	Семантика оператора
Вычисление адреса элемента массива	SUBS	$a, i_1, \dots, i_n, k$	Вычисление адреса элемента массива a ; $i_1, \dots, i_n$ – индексные выражения; $k = n + 1$ – число операндов операции
Вызов функции	CALL	$f, x_1, \dots, x_n, k$	Вызов функции f с аргументами $x_1, \dots, x_n$; $k = n + 1$ – число операндов операции
Преобразование типа целый $\rightarrow$ вещественный	I2A	E	Преобразование типа значения выражение E
вещественный $\rightarrow$ целый	A2I	E	
Оператор присваивания	$:=$	a, b	Оператор присваивания $b := a$

Определение метки	DEFL	m	Определение метки m
Безусловный переход на метку	BRL	m	Переход на метку m (m – идентификатор метки)
Безусловный переход	BR	n	Безусловный переход на n (n – номер элемента ПОЛИЗ)

Таблица 10.3 (окончание)

Оператор	Код операции	Операнды	Семантика оператора
Условный переход по значению “ложь”	BF	r, m	Переход на метку m , если $r = \text{false}$
Переход по условию: равно нулю больше нуля меньше нуля не меньше нуля не больше нуля	BZ BP BM BPZ BMZ	m, r	Переход на m (m – метка или номер элемента ПОЛИЗ) при выполнении соответствующего условия для выражения r
Начало блока	BLBEG	—	Отмечает начало блока
Конец блока	BLEND	—	Отмечает конец блока

Рассмотрим далее разработку представления в ПОЛИЗ конструкций языков программирования, отсутствующих в табл. 10.3, на примере оператор цикла.

5.1.4.6. Оператор цикла

Разработку представления оператора цикла в ПОЛИЗ можно выполнить за два шага. Рассмотрим, например, оператор цикла языка Pascal, имеющего формат:

for $i := \langle \text{Выражение_1} \rangle$ **to** $\langle \text{Выражение_2} \rangle$ **do** $\langle \text{Оператор} \rangle$

Сначала опишем порядок выполнения оператора **for**, используя условный оператор и оператор перехода:

```

 $i := \langle \text{Выражение\_1} \rangle;$ 
 $m_1:$  if ( $i \leq \langle \text{Выражение\_2} \rangle$ )
then

```

```

        <Операторы>
        i := i + 1;
        goto m1
    endif;

```

*m*₄:

Теперь, зная представление в ПОЛИЗ условного оператора и оператора перехода, можно получить ПОЛИЗ оператора **for**:

```

i <Выражение_1> := m1 DEFL i <Выражение_2> – m4 BP <Оператор> i 1 + m1
BRL m4 DEFL

```

Недостатки этого представления оператора цикла становятся очевидными при разработке описания перевода. Это связано с тем, что часть операторов ПОЛИЗ, формируемая при просмотре заголовка цикла, должна следовать в ней *после* тела цикла, т. е. порядок следования операндов в ПОЛИЗе и в исходной программе не совпадает.

Опишем выполнение оператора цикла другим способом:

```

        i := <Выражение1>;
m1:   if i ≤ <Выражение2>
        then goto m2
        else goto m4
        endif
m3:   i := i + 1;
        goto m1;
m2:   <Операторы>;
        goto m3;

```

*m*₄:

В этом случае польская инверсная запись имеет вид:

```

i <Выражение_1> := m1 DEFL i <Выражение_2> ≤ m4 BF m2 BRL m3 DEFL
i i 1+ := m1 BRL m2 DEFL <Оператор> m3 BRL m4 DEFL

```

При таком представлении следование операндов в исходном тексте и в ПОЛИЗ совпадает, что упрощает реализацию перевода.

Возможны и другие способы представления. Например, в [26] за счет введение другого набора операций, польская инверсная запись операторов цикла значительно упрощена.

Дополним ПОЛИЗ следующим операциями:

- *m*₁ *m*₂ *a* BRET – безусловный переход с возвратом. Эта операция имеет три операнда:
 - *m*₁ – метка, на которую должен быть выполнен безусловный переход;
 - *m*₂ – метка, на которую должен быть выполнен возврат по операции RET;
 - *a* – вспомогательная переменная, куда должна быть записана метка возврата;
- *a* RET – операция возврата, которая выполняет операцию перехода на метку, записанную в переменной *a*.

Пример 5.4

Рассмотрим выполнение польской инверсной записи:

m_1 DEFL <Операторы_1> m_2 m_1 a BRET <Операторы_2> m_2 DEFL <Операторы_3> a RET

при условии, что представление элементов <Операторы_1>, <Операторы_2>, <Операторы_3> в ПОЛИЗ известно:

- выполняются <Операторы_1>;
- выполняется оператор BRET: значение метки m_1 записывается во вспомогательную переменную a , выполняется переход к элементу ПОЛИЗ, помеченному меткой m_2 ;
- выполняются <Операторы_3>;
- операция возврата RET выполняет переход к элементу ПОЛИЗ, метка которого записана в переменной a , т. е. к элементу, помеченному меткой m_1 (рис. 10.6), и процесс повторяется.

Заметим, что <Операторы_2> никогда не выполняются.

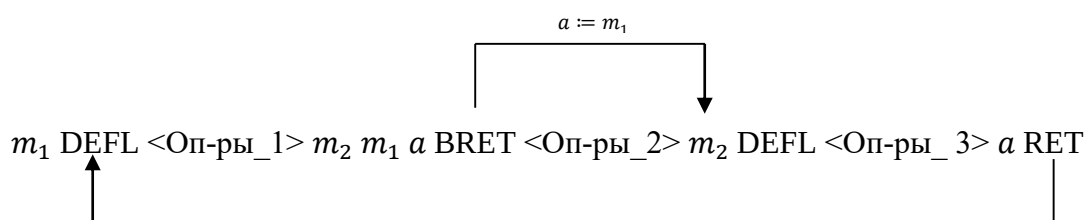


Рис. 5.6. Порядок выполнения ПОЛИЗ из примера 10.4

Используя введенные операции, рассматриваемый оператор цикла

for $i := \langle \text{Выражения}_1 \rangle$ **to** $\langle \text{Выражения}_2 \rangle$ **do** <Оператор>

Можно представить в ПОЛИЗ следующим образом:

$i \langle \text{Выражения}_1 \rangle := m_1$ $i \langle \text{Выражения}_2 \rangle \leq m_4$ BF m_2 m_1 a BRET m_2 DEFL <Оператор> a RET m_4 DEFL.

Порядок выполнения ПОЛИЗ для этого оператора цикла приведен на рис. 10.7.

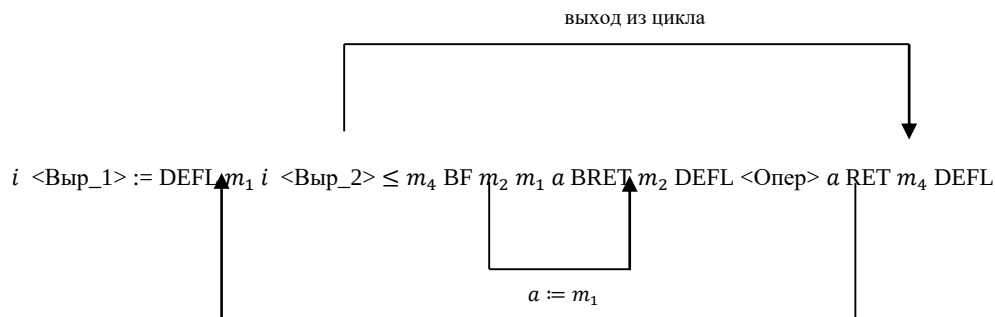


Рис. 5.7. Порядок выполнения ПОЛИЗ для оператора цикла **for**
Возможны и другие способы представления оператора цикла.

Замечание

Разработка представления конструкций языка программирования в ПОЛИЗ – это довольно сложная работа, при выполнении которой необходимо

учитывать особенности языка программирования, выбранных методов анализа и целевой машины.

5.2. Тетрады

Для бинарных операций удобной формой представления программы после синтаксического анализа являются тетрады. Формат тетрад:

<код операции>, <операнд_1>, <операнд_2>, <результат>,

где <операнд_1> и <операнд_2> специфицируются аргументы, а <результат> – временное имя для хранения результата выполнения операции (переменная из рабочей области). В качестве операндов тетрад наряду с переменными и константами, определенными в исходной программе, могут выступать результаты ранее выполненных тетрад. Например, выражение: $a * b + c * d$ представляется в виде последовательности следующих тетрад:

$*, a, b, t_1$

$*, c, d, t_2$

$+, t_1, t_2, t_3$.

Замечание В тетрадах запрещены встроенные в операнды выражения.

Последовательность тетрад представляет собой программу, инструкции которой обрабатываются последовательно. Операнды одной тетрады должны быть одинакового типа. Для преобразования типа операнда можно использовать тетрады преобразования типа с кодами операций IA2 (целый — в вещественный) и A2I (вещественный — в целый). Поскольку операция преобразования типа одноместная, она записывается с пустым вторым операндом, например,

A2I, a , , t

Для представления унарного минуса в тетрадах также можно не использовать второй операнд. Тетрада

$-, a$, , t

интерпретируется как присвоение временной переменной t значения $-a$.

Множество очевидных (для представления выражений) операций может быть при необходимости расширено.

5.2.1 Переменные с индексами

Для представления переменных с индексами можно использовать тетраду

SUBS, a , i , t ,

которая интерпретируется как операция доступа $a[i]$ и позволяет выбрать элемент одномерного массива.

Представление элементов многомерных массивов с помощью тетрад основана на приведении многомерного массива к одномерному (см. разд. 1.7.5.2). Так, если двумерный массив a , имеющий n строк и m столбцов, отображается в памяти по строкам, нумерация индексов начинается с единицы и размер элемента массива – одно слово, то обращение к элементу массива $a[i, j]$ можно записать следующим образом:

$-, i, 1, t_1$

$*, t_1, i, t_2$

$+, t_2, j, t_3$

$-, t_3, 1, t_4$

SUBS, a , t_4, t_5

Первые четыре тетрады необходимы для вычисления значения смещения элемента массива $a[i, j]$ от начала массива по формуле: $(i - 1) * m + j - 1$.

5.2.2. Указатели функций

Обращение к функции записывается с помощью одной или нескольких тетрад в зависимости от числа аргументов функции. Так обращение к функции одной переменной $abs(x)$ записывается в виде одной тетрады

CALL, abs, x, t ,

Возможна и другая реализация обращения к функции с использованием тетрад. Так обращение к функции двух переменных $f(x, y)$ можно записать в виде:

PARAM, $x, ,$

PARAM, $y, ,$

CALL $, f, 2$.

Здесь тетрады с кодом PARAM используются для определения параметров функции, а в тетраде CALL указывается имя функции и число её параметров.

5.2.3. Операторы

Формат тетрад для представления оператора присваивания и операторов безусловного и условного переходов приведен в табл. 10.4.

Таблица 5.4. Формат тетрад для представления основных операторов

Тетрада				Семантика тетрады
$:=$	b		a	$a := b$
DEFL	L			Определение метки l
BRL	I			Безусловный переход к тетраде с меткой I
BF	I	E		Переход к тетраде с меткой I , если значение выражения E равно “ложь”
BR	m			Безусловный переход на тетраду с номером m
BZL (BPL, BML)	m	E		Переход к тетраде с меткой m , если значение выражения E равно нулю (больше нуля, меньше нуля)
BZ (BP, BM)	m	E		Переход на тетраду с номером m , если значение выражения E равно нулю (больше нуля, меньше нуля)

Таблица 10.4. (окончание)

Тетрада				Семантика тетрады
BE (BP, BM)	m	e		Переход на тетраду с номером m , если значение выражения E равно нулю (больше нуля, меньше нуля)
BE (BL, BG)	m	e_1	e_2	Переход на тетраду с номером m , если значение выражения E_1 равно (меньше, больше) значения выражения E_2

В табл. 10.5. приведено представление оператора условного перехода и операторов цикла с помощью тетрад из табл. 10.4.

Замечание В табл. 10.5 функция `signum` служит для проверки знака выражения (первый операнд задает значение проверяемого выражения, а второй операнд возвращает знак выражения).

Тетрады удобно использовать для выполнения машинно-независимой оптимизации, в частности исключения лишних операций. Основным недостатком тетрад является большой объем памяти, необходимый для их хранения. Несмотря на то, что во многих тетрадах имеются свободные поля, результат выполнения операции всегда записывается в четвертое поле. Внутреннее представление тетрады – запись, состоящая из четырех лексем. При этом коды операций тетрад, пустые поля, номера тетрад и временные переменные t_i являются лексемами специального типа.

Таблица 5.5. Представление оператора условного перехода и операторов цикла с помощью тетрад из табл. 5.4.

Оператор	Формат оператора			
	Код операции	Операнд_1	Операнд_2	Результат
if <выражение> then <операторы_1> else <операторы_2> end	<выражение> (результат в t)			
	BF	<i>Lelse</i>	t	
	<операторы_1>			
	BRL	<i>Lend</i>		
	DEFL	<i>Lelse</i>		
	<операторы_2>			
	DEFL	<i>Lend</i>		

Таблица 10.5. (окончание)

Оператор	Формат оператора			
	Код операции	Операнд_1	Операнд_2	Результат
for $j :=$ <выражение_1> step <выражение_2> until <выражение_3> do <оператор>	<выражение_1> (результат в t_1)			
	$:=$	t_1		j
	<выражение_2> (результат в t_2) <выражение_3> (результат в t_3)			
	DEFL	<i>Lbegin</i>		
	–	j	t_3	t_4
	CALL	<code>signum</code>	t_2	t_5
	*	t_4	t_5	t_6
	BPL	<i>Lend</i>	t_6	
	<оператор>			
	+	j	t_2	j
	BRL	<i>Lbegin</i>		

	DEFL	<i>Lend</i>		
while <выражение> do <оператор>	DEFL	<i>Lbegin</i>		
	<выражение_1> (результат в <i>t</i>)			
	BF	<i>t</i>	<i>Lend</i>	
	<оператор>			
	BRL DEFL	<i>Lbegin</i> <i>Lend</i>		

5.3. Триады

Триады так же, как и тетрады, являются удобной промежуточной формой представления бинарных операций. Формат триады имеет вид:

<код операции>, <операнд_1>, <операнд_2>.

В качестве кодов операций триад можно использовать коды операций тетрад, за исключением операций условного перехода BE (BL, BM), которые для триад не используются.

В триадах отсутствует поле для записи результата. Если в качестве операнда некоторой триады нужно использовать результат выполнения другой (ранее выполненной) триады, то в соответствующее поле триады записывается ссылка на триаду, в которой этот результат был получен. Результаты выполнения триад хранятся отдельно. Например, запись в виде последовательности триады выражения

$$a * b - c / d$$

будет иметь следующий вид (номера триад указаны в скобках):

- (1) *, *a*, *b*
- (2) /, *c*, *d*
- (3) −, (1), (2)

Количество триад и тетрад, необходимое для представления одних и тех же конструкций языка, является одним и тем же, но каждая триада занимает в памяти меньше места. Экономия памяти при использовании триад в качестве промежуточной формы программы по сравнению с тетрадами достигается также за счет того, что после выполнения триады, в которой одним из операндов является результат выполнения *i*-ой триады, ее результат может быть уничтожен.

При выполнении машинно-независимой оптимизации некоторые операции могут удаляться из промежуточного представления программы или переставляться на другое место. При использовании тетрад такие преобразования не вызывают затруднений. В случае же использования триад могут возникнуть осложнения из-за того, что триады ссылаются друг на друга. Поэтому, если машинно-независимая оптимизация будет выполняться, необходимо либо хранить все результаты выполнения триад, либо использовать *косвенные триады*.

Косвенные триады представляются двумя списками. В первом списке хранятся сами триады, причем одинаковые триады в список заносятся только один раз. Второй список содержит номера триад в порядке их выполнения. При использовании косвенных триад перестановка и исключение идентичных операций во время машинно-независимой оптимизации производятся путем преобразования списка, содержащего номера триад. При этом вид триад и их количество в списке не изменяются. Использование косвенных триад приводит к дополнительному сокращению объема памяти, необходимого для хранения промежуточной формы программы, если исходная программа содержит большое число идентичных операций (обычно это имеет место при наличии в программе большого числа индексированных переменных)

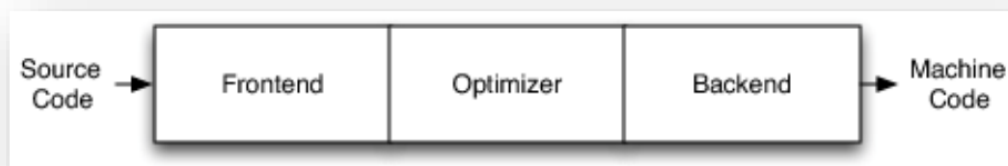
Пример представления последовательности операторов присваивания языка Паскаль с помощью косвенных триад приведены в табл. 5.6.

Таблица 5.6. Представление последовательности операторов присваивания языка Паскаль с помощью косвенных триад

Оператор	Список триад			Порядок выполнения триад
$A := B[j] + B[j + 1];$	SUBS	B	j	(1), (2), (3), (4), (5), (1), (2), (3), (6), (7), (1), (2), (3), (8), (9), (10)
$C := B[j] - B[j + 1];$	+	j	1	
$D := B[j] + 2 * B[j + 1]$	SUBS	B	(2)	
	+	(1)	(3)	
	:=	(4)	A	
	•	(1)	(3)	
	:=	(6)	C	
	*	2	(3)	
	+	(1)	(8)	
	:=	(9)	D	

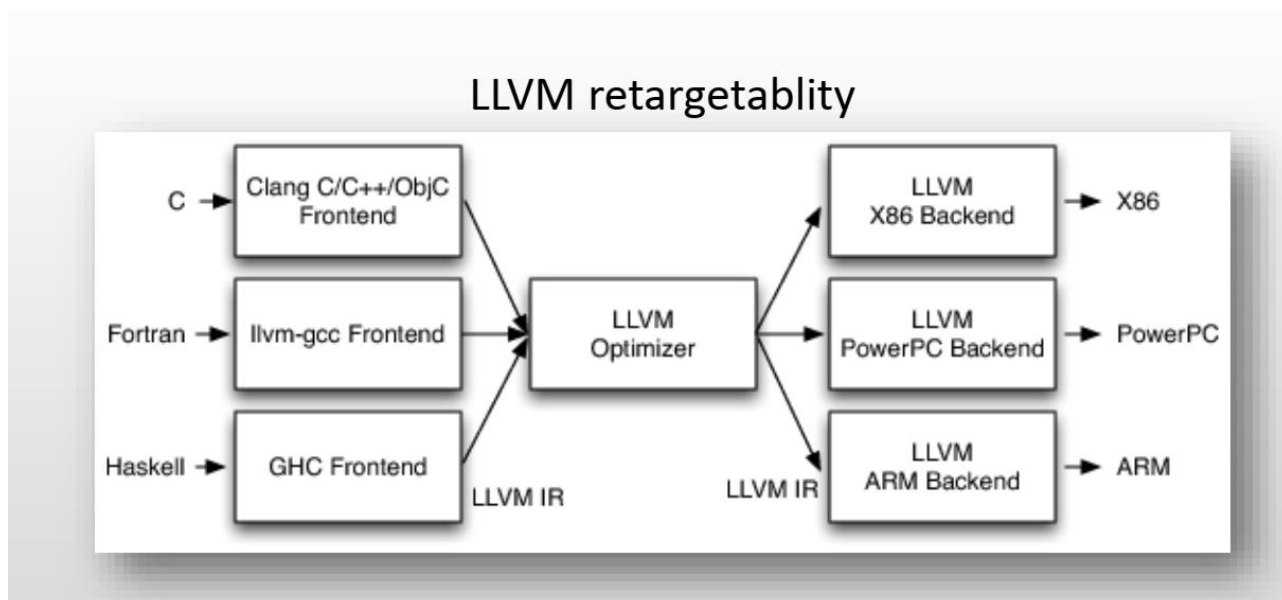
5.6. LLVM

Traditional Three-Phase Compiler design



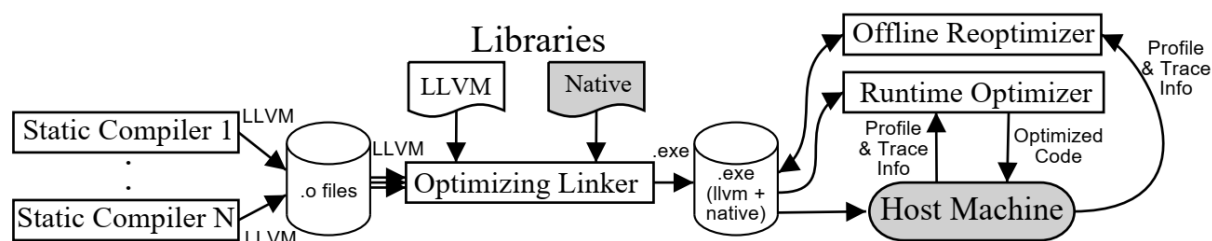
LLVM (ранее Low Level Virtual Machine[7]) — проект программной инфраструктуры для создания компиляторов и сопутствующих им утилит.

Состоит из набора компиляторов из языков высокого уровня (так называемых «фронтендов»), системы оптимизации, интерпретации и компиляции в машинный код. В основе инфраструктуры используется RISC-подобная платформонезависимая система кодирования машинных инструкций (байткод LLVM IR), которая представляет собой высокоуровневый ассемблер, с которым работают различные преобразования.



В основе LLVM лежит промежуточное представление кода (Intermediate Representation, IR), над которым можно производить трансформации во время компиляции, компоновки и выполнения. Из этого представления генерируется оптимизированный машинный код для целого ряда платформ, как статически, так и динамически (JIT-компиляция). LLVM 9.0.0 поддерживает статическую генерацию кода для x86, x86-64, ARM, PowerPC, SPARC, MIPS, RISC-V, Qualcomm Hexagon, NVPTX, SystemZ, Xcore. JIT-компиляция (генерация машинного кода во время исполнения) поддерживается для архитектур x86, x86_64, PowerPC, MIPS, SystemZ, и частично ARM[12].

LLVM написана на C++ и портирована на большинство Unix-подобных систем и Windows. Система имеет модульную структуру, отдельные её модули могут быть встроены в различные программные комплексы, она может расширяться дополнительными алгоритмами трансформации и кодогенераторами для новых аппаратных платформ. LLVM включена обёртка API для OCaml.



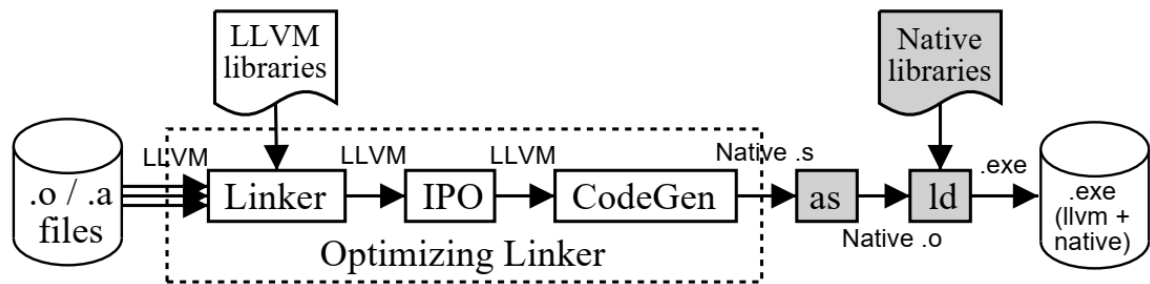


Figure 2: LLVM Optimizing Linker

How to build the LLVM

- <http://llvm.org/docs/GettingStarted.html#getting-started>
- Download llvm 3.2, clang., Compiler-RT from <http://llvm.org/releases/download.html#3.2>

```

$ tar xzf llvm-3.2.src.tar.gz
$ cd llvm-3.2.src/tool
$ tar xzf clang-3.2.src.tar.gz
$ mv clang-3.2.src.tar.gz clang
$ cd llvm-3.2.src/projects
$ tar xzf compiler-rt-3.2.src.tar.gz
$ mv compiler-rt-3.2.src compiler-rt
$ cd where-you-want-to-build-llvm
$ ../llvm/configure
$ make

```

5.5. Байт-коды JVM

Виртуальная машина Java базируется на понятии виртуального компьютера с логическим набором инструкций (псевдокодов), определяющих операции, которые может выполнить этот компьютер. *Интерпретатор JVM* – это программа, которая понимает семантику байт-кодов JVM и преобразует их в инструкции объектной машины. JVM базируется на концепции *стековой машины*.

Стековая машина не использует никакие физические регистры для передачи информации от одной инструкции к другой. Вместо этого используется стек, в котором производятся вычисления. В JVM имеется единственный регистр, называемый *регистром PC*, который содержит *адрес* текущей выполняемой инструкции в потоке байт-кодов. Инструкции байт-кода выполняются последовательно. Для изменения порядка выполнения байт-кодов имеются специальные команды, изменяющие содержимое регистра PC.

Большинство семантических процедур, реализующих инструкции байт-кода, получают свои операнды из стека и помещают результаты обратно в стек. Инструкции могут иметь также аргументы, которые следуют в потоке байт-кодов непосредственно за самой инструкцией. В табл. 10.7 приводятся наиболее часто используемые инструкции байт-кода, упорядоченные по коду операции. Полный список всех инструкций JVM можно найти, например, в [8].

Пример 5.5

С использованием операций из табл. 10.7. байт-код фрагмента программы на Java

```
x = 0; a = 1; b = 2; c = 5;
x = a + b * c;
```

имеет следующий вид

03	Записать в стек 0
3C	Прочитать содержимое стека в x
04	Записать в стек 1
3D	Прочитать содержимое стека в a
10 02	Записать в стек 2
3E	Прочитать содержимое стека в b
10 05	Записать в стек 5
36 04	Прочитать содержимое стека в c
1C	Поместить в стек a
1B	Поместить в стек b
15 04	Поместить в стек c
68	Поместить в стек bc*
60	Поместить в стек a+
3C	Прочитать содержимое стека в x

Символьные вычисления

Символьные вычисления — это преобразования и работа с математическими равенствами и формулами как с последовательностью символов. Они отличаются от численных расчётов, которые оперируют приближёнными численными значениями, стоящими за математическими выражениями. Системы символьных вычислений (их так же называют системами компьютерной алгебры) могут быть использованы для символьного интегрирования и дифференцирования, подстановки одних выражений в другие, упрощения формул и т. д. В общем случае символьное вычисление заданного выражения производится по следующей схеме. Сначала

входное выражение преобразуется из инфиксной формы в дерево (для этого обычно используется какой-либо вариант алгоритма Дейкстры преобразования инфиксной записи в польскую постфиксную запись). В вершине дерева, представляющего выражение, находится функция, которую мы хотим вычислить. Система символьных вычислений должна располагать набором правил вычисления данной функции. Эти правила последовательно просматриваются и, если какое-либо правило применимо к данному дереву, то оно применяется к нему. В результате получаем новое дерево и вновь пытаемся применить к нему существующие правила. Применение правил к дереву выражений обычно называется переписыванием терма (term rewriting). Когда система не находит правила, применимого к данному дереву, оно считается результирующим. После этого результирующее дерево преобразуется в инфиксную запись и выводится на устройство вывода. В качестве примера рассмотрим дифференцирование полиномов. Полиномы будем представлять сразу в префиксной форме. Например, полином $3x^2+4$ примет вид $(+ (* 3 (^ x 2)) 4)$.

В следующей таблицы приведены примеры символьных вычислений

Исходное выражение	Результат
$(3x^2 + 4)'$	$6x$
$\frac{x^2 - 3x - 4}{x - 4} + 2x - 5$	$3x - 4$
$\sin 5x$	$16 \sin x \cos x^4 - 12 \sin x \cos x^2 + \sin x$

Опишем основные этапы символьного вычисления на примере дифференцирования:

1. Порождающая грамматика арифметического выражения
2. Преобразование инфиксной формы записи выражения в постфиксную
3. Построения по постфиксной форме префиксной, затем построение дерева выражений и применения к нему необходимых операций
4. Упрощения результата и приведение его обратно к инфиксной форме

Синтаксис арифметических выражений (с учетом *приоритета* операций) будем задавать следующей *порождающей грамматикой*:

$G = (T, V, P, S_0)$, где

$T = \{x, \sin x, \cos x, \operatorname{tg} x, \operatorname{ctg} x, +, -, \backslash, *, (,), 0 \dots 9\}$;

$V = \{S, A, B, C, D, E, J\}$;

$S_0 = S$.

P:

$p_1: S \rightarrow +A \mid -A \mid A$

$p_2: A \rightarrow B+A \mid B-A \mid B$

$p_3: B \rightarrow C*B \mid C \backslash B \mid C$

$p_4: C \rightarrow D \mid (S)$

$p_5: D \rightarrow x \mid \sin x \mid \cos x \mid \operatorname{tg} x \mid \operatorname{ctg} x \mid E \mid E*x \mid E*\sin x \dots$

$p_6: E \rightarrow J$

$p_7: E \rightarrow EJ$

$p_8: \mathbb{J} \rightarrow 0 \mid \dots \mid 9$

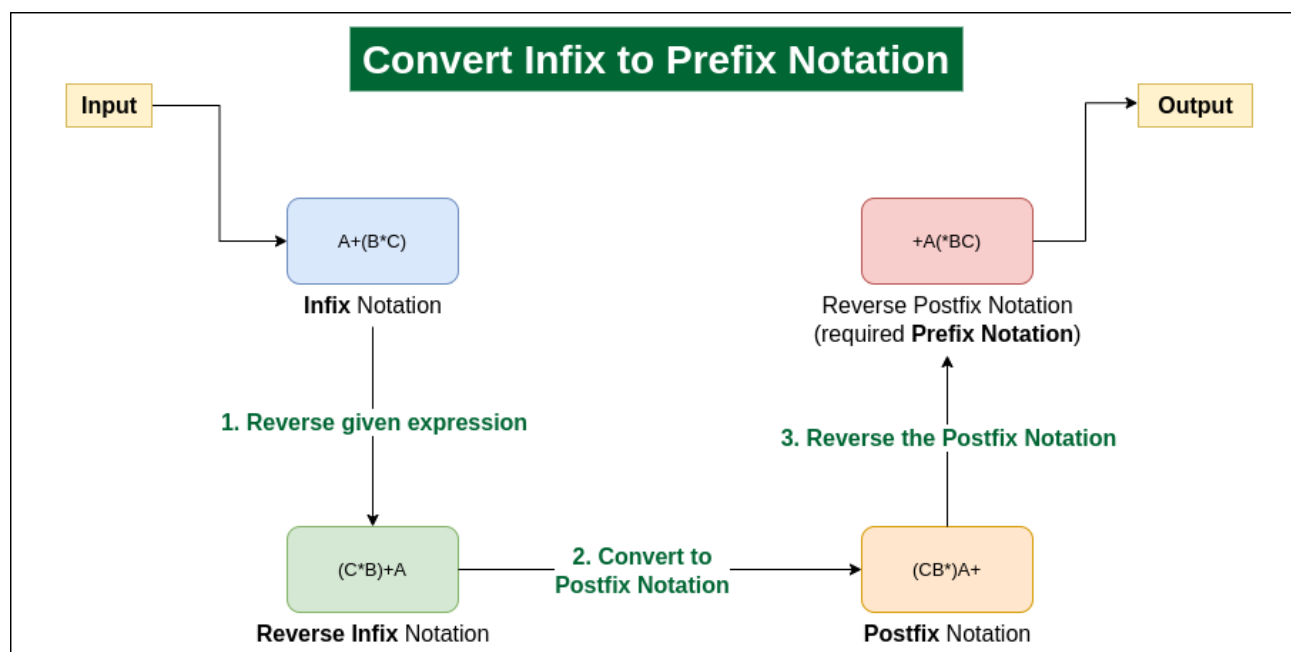
Пример цепочек:

$S \Rightarrow^1 A \Rightarrow^2 B+A \Rightarrow^3 C*B+A \Rightarrow^4 D*B+A \Rightarrow^5 E*B+A \Rightarrow^6 3*B+A \Rightarrow^7 3*C*B+A \Rightarrow^8 3*D*B+A \Rightarrow^9 3*x*B+A \Rightarrow^{10} 3*x*C+A \Rightarrow^{11} 3*x*D+A \Rightarrow^{12} 3*x*x+A \Rightarrow^{13-18} 3*x*x+4$

Теперь получив выражение в инфиксной форме, преобразуем его в постфиксную форму, используя алгоритм Дейкстры.

Принцип работы алгоритма Дейкстры:

- Проходим исходную строку;
- При нахождении **числа**, заносим его в выходную строку;
- При нахождении **оператора**, заносим его в стек;
- Выталкиваем в выходную строку из стека все операторы, имеющие приоритет выше рассматриваемого;
- При нахождении открывающейся скобки, заносим её в стек;
- При нахождении закрывающейся скобки, выталкиваем из стека все операторы до открывающейся скобки, а открывающуюся скобку удаляем из стека.



полином $3*x^2+4$ примет вид $(+*3^x24)$. Апостроф здесь обязателен, поскольку необходимо понять, что нужно именно дифференцировать. У нас есть функция, которая по полиному p и переменной x вычислит производную $p'(x)$, которую мы обозначим как $D(p, x)$. Например, производная полинома $3*x^2+4$ равна $6*x$, поэтому мы хотели бы получить результат $*6 x$.

Выпишем систему правил дифференцирования:

$$D[x, x]=1$$

$$D[c, x]=0 \text{ (} c \text{ – атом, отличный от } x \text{)}$$

$$D[u+v, x]=D[u, x]+D[v, x]$$

$$D[u-v, x]=D[u, x]-D[v, x]$$

$$D[uv, x] = uD[v, x] + vD[u, x]$$

$$D[u^n, x] = nu^{(n-1)}D[u, x]$$

$$D[\sin x, x] = \cos x$$

$$D[\cos x, x] = -\sin x$$

...

$$D[\operatorname{ctgx}, x] = -1/\sin x^2$$

Очевидно, что после вычисления производной необходимо произвести упрощение полученного выражения. В системах символьных вычислений процедура упрощения результата обычно вызывается автоматически перед его преобразованием в инфиксную запись. Она имеет собственную систему правил, которая в нашем случае может выглядеть так (t означает любое выражение):

Исходное выражение	Результат
*0t	0
*1t	t
+0t	t
+t0	t
*t1	t
-t0	t
^t0	1
^t1	t
^0t	0
^1t	1
a*b*t	c*t, c=a*b

Функция упрощения рекурсивно применяет приведенные в таблице правила к дереву полинома, начиная с корня. Однако, легко проверить, что она не гарантирует окончательного упрощения полинома. Проблема в том, что по мере прохождения дерева мы не можем вернуться на его верхние уровни. Простейшим решением проблемы является написание еще одной функции, которая будет применять функцию упрощения к полиному до тех пор, пока выходное дерево не совпадет с входным (т. е. остановится, когда дальнейшие упрощения невозможны). В результате чего на выходе мы получим как раз *6x. После чего данное выражение нужно привести в инфиксную форму

Алгоритм преобразования префикса в инфикс:

- Прочитайте выражение префикса в обратном порядке (справа налево)
- Если символ является операндом, то поместите его в стек
- Если символ является оператором, то извлеките два операнда из стека
Создайте строку путем объединения двух операндов и оператора между ними.

строка = (операнд 1 + оператор + операнд 2)

И помещаем результирующую строку обратно в Stack

- Повторяйте вышеуказанные шаги до конца выражения префикса.

- В конечном стеке будет только 1 строка, т. е. результирующая строка

Пример цепочек: // 3x-4

$S \Rightarrow^1 A \Rightarrow^2 B-A \Rightarrow^3 C*B-A \Rightarrow^4 D*B-A \Rightarrow^5 E*B-A \Rightarrow^6 J*B-A \Rightarrow^7 3*B-A \Rightarrow^8 3*C-A \Rightarrow^9 3*D-A \Rightarrow^{10} 3*x-A \Rightarrow^{11} 3*x-B \Rightarrow^{12} 3*x-C \Rightarrow^{13} 3*x-D \Rightarrow^{14} 3*x-E \Rightarrow^{15} 3*x-J \Rightarrow^{16} 3*x-4$

Контрольные вопросы

1. Почему исходная программа переводится сначала в промежуточное представление, а затем в объектный код?
2. Как арифметическое выражение записывается в ПОЛИЗ? Как в ПОЛИЗ записывается унарный минус?
3. Почему в компиляторах используется для представления выражений польская инверсная запись?
4. Как производится вычисление выражения, представленного в польской инверсной записи?
5. Как графически представляется польская инверсная запись?
6. Как можно представить в тетрадах переменную с индексами, указатель функции?
7. Чем различаются триады и косвенные триады?
8. Что представляет собой виртуальная машина Java?
9. Как обрабатываются инструкции байт-кода?

Упражнения

1. Представьте графически и в польской инверсной записи выражения:
 - 1.1. $|\cos x + \sin y - z|$.
 - 1.2. $\sqrt{b + |a|}$.
 - 1.3. $1 + a_{i,j} - \operatorname{ctg} \frac{x}{\sqrt{x^2+1}}$.
2. Представьте в ПОЛИЗ, тетрадах и триадах операторы
 - 2.1 $a := b^2 + 4ac$.
 - 2.2 $\text{if } (b \ \&\& \ c < d) \ a + +; \text{else } a += 2$.
 - 2.3 $\text{for } j := n \text{ downto } 1 \text{ do } a[j + 1]$.

Глава 6. Методы синтаксически управляемого перевода

Грамматики описывают только синтаксические аспекты формальных языков.

Синтаксически управляемая трансляция (англ. Syntax-directed translation, SDT) - метод реализации компилятора, при котором перевод исходного языка в выходной язык полностью выполняется анализатором.

6.1. Перевод и семантика

Определение. Отношение между входными цепочками языка и выходными цепочками языка называют трансляцией или переводом.

Существуют два подхода к описанию перевода:

1. использование системы, аналогичной формальной грамматике или являющейся её расширением, которая порождает перевод (точнее пару цепочек, принадлежавших переводу);

Для перевода могут быть применены два метода:

1. схема трансляции — описание грамматики с встроенными семантическими операциями $A \rightarrow B + C \{+\}$;
2. семантические правила с атрибутами: грамматика может быть применена для выполнения семантических правил над атрибутами символов.

правило продукций	семантические правила
$A \rightarrow B + C$	$A.val \rightarrow B.val + C.val$

Атрибуты могут включать в себя тип переменной, значение выражения, и т.п. Для символа A с атрибутом val обращение к атрибуту выглядит как $A.val$.

2. применение автомата, распознающего входную цепочку и выдающего на выходе перевод этой цепочки.

Перейдём к рассмотрению простейших способов описания перевода.

Определение. Пусть Σ — входной алфавит и Δ — выходной алфавит. Переводом с языка $L_1 \subseteq \Sigma^*$ на язык $L_2 \subseteq \Delta^*$ называется отношение τ из $\Sigma^* \times \Delta^*$, для которого L_1 — область определения, а L_2 — множество значений.

Если $(x, y) \in \tau$, то цепочка y называется выходом для цепочки x . В общем случае в переводе τ для одной входной цепочки может быть задано более одной выходной цепочки.

Для языков программирования перевод должен быть функцией, т. е. для каждой входной программы должно быть не более одной выходной программы.

Простейшие переводы можно задать при помощи гомоморфизма.

Пример 5.1. Пусть требуется перевести символы, образующие целые числа со знаком, в и названия. В этом случае гомоморфизм h можно определить следующим образом:

$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, -, +\};$

$h(a) = a$, если $a \in \Sigma$;

Для $a \in \Sigma$ гомоморфизм $h(a)$ определяется в соответствии с табл. 5.1.

Таблица 5.1. Определение гомоморфизма $h(a)$

a	h(a)
1	один
2	два
3	три
4	четыре
5	пять
6	шесть
7	семь
8	восемь
9	девять
0	нуль
+	плюс
-	минус

Используя данное определение, перевод входной цепочки "-15", содержащей запись числа -15, будет иметь вид: "минус один пять".

Гомоморфизм позволяет описывать только простейшие переводы.

Нельзя задать при помощи гомоморфизма даже такие простые переводы, как $\tau_1 = \{(x, y) \mid x \text{ — инфиксное выражение и } y \text{ — префиксная запись выражения } x\}$ и $\tau_2 = \{(x, y) \mid x \text{ — инфиксное выражение и } y \text{ — постфиксная запись выражения } x\}$. Нужны более мощные формализмы.

6.2 СУ-схемы

Схема синтаксически управляемого (СУ-схема) перевода представляет собой систему, порождающую пары цепочек, принадлежавших переводу. Неформально схема синтаксически управляемого перевода представляет собой грамматику, в которой каждому правилу приписан элемент перевода. При порождении цепочки каждый раз, когда правило участвует в этом процессе, элемент перевода генерирует часть выходной цепочки, соответствующей части входной цепочки, порождённой этим правилом.

Рассмотрим схему синтаксически управляемого перевода, описывающего перевод скобочного арифметического выражения, порождаемого грамматикой G_0 , в соответствующую польскую инверсную запись G_1 . Схема перевода представлена в табл. 5.2.

Таблица 5.2. СУ-схема перевода

N	Правила грамматики G_0	Элемент перевода G_1
1	$E \rightarrow E + T$	$E \rightarrow \{\text{new Symbol}\} E \{\text{new FA}\} T +$
2	$E \rightarrow T$	$E \rightarrow T$
3	$T \rightarrow T * P$	$T \rightarrow TP *$

4	$T \rightarrow P$	$T \rightarrow P$
5	$P \rightarrow (E)$	$P \rightarrow E$
6	$P \rightarrow i$	$P \rightarrow i$

Правилу грамматики $E \rightarrow E + T$ соответствует элемент перевода $E \rightarrow ET+$. Используя СУ-схему, приведённую в табл. 5.2, определим перевод цепочки $i * (i + i)$.

Определим левый вывод входной цепочки:

$E \Rightarrow^2 T \Rightarrow^3 T * P \Rightarrow^4 P * P \Rightarrow^6 i * P \Rightarrow^5 i * (E) \Rightarrow^1 i * (E + T) \Rightarrow^2 i * (T + T) \Rightarrow^4 i * (P + T) \Rightarrow^6 i * (i + T) \Rightarrow^4 i * (i + P) \Rightarrow^6 i * (i + i)$

Полученный вывод 23465124646 определяет последовательность использования элементов перевода при порождении выходной цепочки, поэтому последовательность выводимых пар цепочек имеет следующий вид:

$(E, E) \Rightarrow^2 (T, T) \Rightarrow^3 (T * P, TP *) \Rightarrow^4 (P * P, PP *) \Rightarrow^6 (i * P, iP *) \Rightarrow^5 (i * (E), i * E) \Rightarrow^1 (i * (E + T), i * ET +) \Rightarrow^2 (i * (T + T), i * TT +) \Rightarrow^4 (i * (P + T), i * PT +) \Rightarrow^6 (i * (i + T), i * iT +) \Rightarrow^4 (i * (i + P), i * iP +) \Rightarrow^6 (i * (i + i), i * ii +)$

Каждая выходная цепочка при порождении части входной цепочки получается путем замены (подходящего) не терминала выходной цепочки значением соответствующего ему элемента перевода.

Рассмотренная схема перевода относится к классу схем, называемых *схемами синтаксически управляемого перевода*.

Определение. Схемой синтаксически управляемого перевода называется пятёрка $Tb = (V, \Sigma, \Delta, P, S)$, где:

V — конечное множество нетерминальных символов;

Σ — конечный входной алфавит;

Δ — конечный выходной алфавит;

P — конечное множество правил вида $A \rightarrow \alpha, \beta$, где $\alpha \in (V \cup \Sigma)^*$, $\beta \in (V \cup \Delta)^*$ и вхождения не терминалов в цепочку β образуют перестановку вхождений не терминалов в цепочку α ;

S — начальный символ ($S \in V$).

По определению, каждому вхождению определённого не терминала в цепочку α соответствует некоторое вхождение этого не терминала в цепочку β . Если некоторый не терминал входит в цепочку α более одного раза, то может возникнуть неоднозначность. Для исключения этого будем использовать верхний целочисленный индекс, например в правиле $A \rightarrow B^1CB^2, B^2B^1C$ первой, второй и третьей позиции цепочки B^1CB^2 соответствуют вторая, третья и первая позиции цепочки B^2B^1C .

Примечание: выводимая пара цепочек Tb СУ-схемы определяется рекурсивно следующим образом:

1. (S, S) — выводимая пара, в которой символы S соответствуют друг

другу.

2. Если $(\alpha A \beta, \alpha' A \beta')$ — выводимая пара, в которой два выделенных вхождения не терминала A соответствуют друг другу, и $A \rightarrow \alpha, \beta$ — правило из P , то $(\alpha \gamma \beta, \alpha' \gamma' \beta')$ — выводимая пара.

Вхождения не терминалов в цепочки γ и γ' соответствуют друг другу точно так же, как они соответствовали в правиле СУ-схемы. Вхождения не терминалов в цепочки α и β соответствуют вхождениям не терминалов в цепочки α' и β' в новой выводимой паре точно так же, как они соответствовали в старой выводимой паре. При необходимости это соответствие будет указываться верхними индексами.

Если между парами цепочек $(\alpha A \beta, \alpha' A \beta')$ и $(\alpha \gamma \beta, \alpha' \gamma' \beta')$ установлена описанная ранее связь, то будем писать $(\alpha A \beta, \alpha' A \beta') \Rightarrow_{Tb} (\alpha \gamma \beta, \alpha' \gamma' \beta')$.

Транзитивное замыкание, рефлексивно-транзитивное замыкание и k -ю степень отношения $\Rightarrow_{Tb}$ будем обозначать как $\Rightarrow^+_{Tb}$, $\Rightarrow^*_{Tb}$ и $\Rightarrow^k_{Tb}$, соответственно. В очевидных случаях можно опускать индекс Tb .

Определение. Переводом, определяемым схемой Tb , или $\tau(Tb)$ называют множество пар цепочек $\{(x, y) \mid (S, S) \Rightarrow^* (x, y), x \in \Sigma^* \text{ и } y \in \Delta^*\}$.

Пример 6.2 Пусть СУ-схема задана следующим образом:

$Tb = (\{S\}, \{i, +\}, \{i, +\}, P, S)$, где множество правил P содержит правила

$$1. S \rightarrow + S^1 S^2, S^1 + S^2$$

$$2. S \rightarrow i, i$$

Рассмотрим вывод в данной СУ-схеме:

$$\begin{aligned} (S, S) &\Rightarrow^1 (+ S^1 S^2, S^1 + S^2) \\ &\Rightarrow^1 (+ + S^3 S^4 S^2, S^3 + S^4 + S^2) \\ &\Rightarrow^1 +++ S^5 S^6 S^4 S^2, S^5 + S^6 + S^4 + S^2) \\ &\Rightarrow^2 +++ i S^6 S^4 S^2, i + S^6 + S^4 + S^2) \\ &\Rightarrow^2 +++ ii S^4 S^2, i + i + S^4 + S^2) \\ &\Rightarrow^2 +++ iii S^2, i + i + i + S^2) \\ &\Rightarrow^2 +++ iiii, i + i + i + i) \end{aligned}$$

Входной цепочке $+++ iiii$ соответствует выходная цепочка $i + i + i + i$.

Перевод, задается СУ-схемой: $\tau(Tb) = \{(+)^k (i)^{k+1}, i(+i)^k \mid k \geq 0\}$

Примечание: пусть $Tb = (V, \Sigma, \Delta, P, S)$ - СУ-схема, то $\tau(Tb)$ называется синтаксически управляемым переводом.

$G_i = (V, \Sigma, P^i, S)$, где $P^i = \{A \rightarrow \alpha \mid A \rightarrow \alpha, \beta, \beta \in P^i\}$, называется входной грамматикой СУ-схемы Tb .

$G_j = (V, \Delta, P^j, S)$, где $P^j = \{A \rightarrow \beta \mid A \rightarrow \alpha, \beta, \beta \in P^j\}$, называется выходной грамматикой СУ-схемы Tb .

Два способа синтаксически управляемого перевода: применение оператора вывода и метод преобразования деревьев разбора.

Метод преобразования деревьев вывода

Дерево вывода входной грамматики G_i преобразуется в дерево вывода выходной грамматики G_j . Перевод входной цепочки α можно получить, построив

ее дерево вывода, затем преобразовав это дерево в дерево вывода выходной грамматики, взяв крону выходного дерева как перевод цепочки α .

Алгоритм 6.1 Преобразование деревьев при помощи СУ-схемы

Вход: СУ-схема $T_b = (V, \Sigma, \Delta, P^i, S)$, входная грамматика $G_i = (V, \Sigma, P^i, S)$, выходная грамматика $G_j = (V, \Delta, P^j, S)$ и дерево вывода D в G_i , с кроной, принадлежавшей Σ^* .

Выход: Дерево вывода D' в G_0 , такое, что если x и y - кроны деревьев D и D' соответственно, то $(x, y) \in \tau(T_b)$.

Шаг 1. Применяется к внутренней вершине n дерева D , имеющей k прямых потомков $n_1, \dots, n_k$.

1.1 Устранить из множества вершин $n_1, \dots, n_k$ листья, помеченные терминальными символами или ϵ .

1.2 Пусть $A \rightarrow \alpha$ — правило входной грамматики G_i , соответствующее вершине n и ее прямым потомкам, т. е. A — метка вершины n , и α образуется конкатенацией меток вершин $n_1, \dots, n_k$. Выбрать из P^i некоторое правило вида $A \rightarrow \alpha, \beta$ (если таких правил несколько, то выбор произволен).

Переставить оставшиеся прямые потомки вершины n (если они есть) согласно соответствию между вхождением не терминалов в α и β . (Поддеревья, корнями которых служат эти потомки, переставляются вместе с ними).

1.3 Добавить в качестве прямых потомков вершины и листья с метками так, чтобы метки всех ее прямых потомков образовали цепочку β .

1.4 Применить шаг 1 к прямым потомкам вершины n , не являющимися листьями, в порядке слева направо.

2. Повторять шаг 1 рекурсивно, начиная с корня дерева D .

Результат дерево D' .

Алгоритм построения СУ-схемы правил продукций на основе 2 заданных грамматик с общим алфавитом нетерминальных символов

На входе: $G_1 = (T_1, V, P_1 = [p_1 p_2 \dots p_n], S_1)$, $G_2 = (T_2, V, P_2 = [p'_1 p'_2 \dots p'_n], S_2)$

На выходе: $T_b = (V, \Sigma, \Delta, P, S)$

$S = S_1$

$\Sigma = T_1$

$\Delta = T_2$

$P = \emptyset$

foreach ($i = \{1, 2, \dots, n\}, p_i \in P_1, q_i \in P_2$):

 if ($LHS(p_i) \neq LHS(q_i)$):

 Incorrect

Утверждение. Каждому вхождению определённого не терминала в цепочку α соответствует некоторое вхождение этого не терминала в цепочку β . Если некоторый не терминал входит в цепочку α более одного раза, то может возникнуть неоднозначность.

$X = RHS(p_i)$

L

L

```

foreach ( $x_j \in X, x_j \in V$ ):
     $L_1 = L_1 \cup x_j$ 
 $Y = \text{RHS}(q_i)$ 
foreach ( $x_j \in Y, x_j \in V$ ):
     $L_2 = L_2 \cup x_j$ 
if  $L_1 \setminus L_2 \neq \emptyset$  or  $L_2 \setminus L_1 \neq \emptyset$  :
    Incorrect
else:
     $w_i = (\text{LHS}(p_i), \text{RHS}(p_i), \text{RHS}(q_i))$ 

```

Пример. Представление правил в табличном виде с декомпозицией.

Транслирующая схема с разложением правой части.

Р Номер Правила	Правила	Декомпозиция правил		
		LHS(n)	RHS _L (n)	RHS _R (n)
1	$S \rightarrow 0AS, SAa$	S	0AS	SAa
2	$A \rightarrow 0SA, ASa$	A	0SA	ASa
3	$S \rightarrow 1, b$	S	1	b
4	$S \rightarrow 1, b$	A	1	b

Правила вывода цепочки NUMBER_INFERENCE_RULES:

1	2	3	3	2	3	4
---	---	---	---	---	---	---

Алгоритм построения дерева по слоям

Вход: CY-схема $T_b = (V, \Sigma, \Delta, P^i, S)$, NUMBER_INFERENCE_RULES = [1,2,...]
номера правил цепочки вывода

Выход: Дерево в виде упорядоченного множества

NODES = [] = create_node(LHS($P^i(\text{NUMBER_INFERENCE_RULES}[1])$)))

index = 1

i = 1

while (True)

node = NODES[index]

if (LHS($P^i(\text{NUMBER_INFERENCE_RULES}[i])$) == node.symbol)

foreach ($X \in \text{RHS}_L(P^i(\text{NUMBER_INFERENCE_RULES}[i]))$)

NODES = NODES $\cup$ create_node(X)

link(index, |NODES|)

end

i ++

end

index++

if (i > |NUMBER_INFERENCE_RULES|)

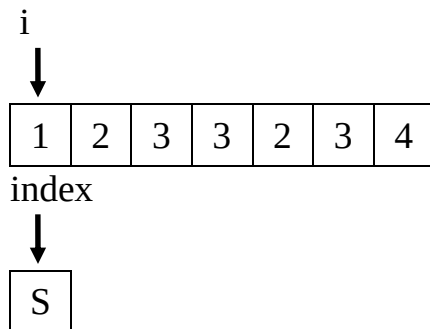
break

end

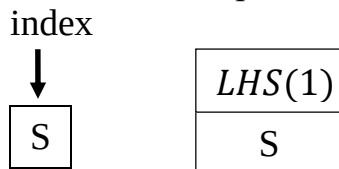
end

Выполним шаги алгоритма

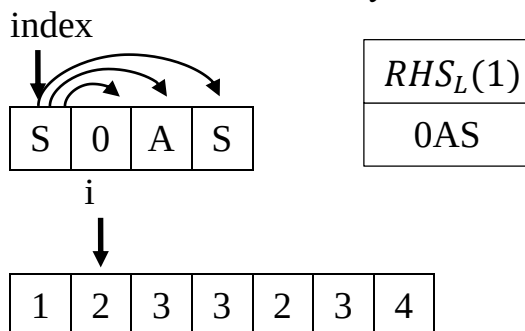
1. Создаем упорядоченное множество NODES и кладем туда левую часть первого правила из NUMBER_INFERENCE_RULES. Также создаем два счетчика для итерации по множествам.



2. Пока не встретим в NODES левую часть из текущего правила, сдвигаем index вправо.

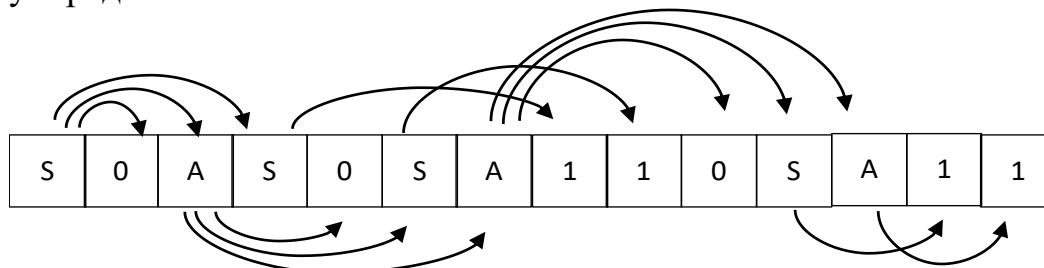


3. Если встретили, то для каждого символа из правой части (RHS_L): создаем node, помещаем только что созданную ноду в конец NODES и связываем ее ссылкой с текущей. Сдвигаем i один раз вправо, сдвигаем index вправо.



4. Повторить шаги 2-3, пока $i \leq |NUMBER_INFERENCE_RULES|$

В итоге мы получим послойное представление дерева в виде упорядоченного множества.



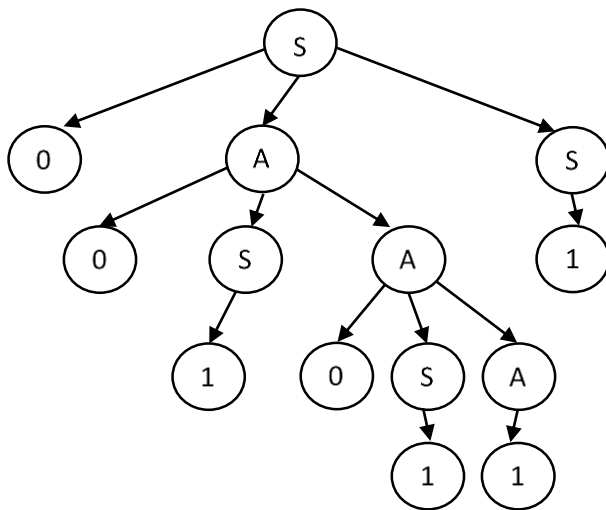


Рис. а

Алгоритм преобразования дерева вывода по СУ схеме

На входе: $Tb = (V, \Sigma, \Delta, P_i, S), D$

На выходе: D'

Копия создается

$D = \text{build_tree}()$

$i = 1$ // индекс в NUMBER_INFERENCE_RULES

$\text{index} = 1$ // индекс в D

while (true):

Nodes = getLinked(index, D)

Node= [] // множество вершин упорядоченное связанными с node

foreach symbol $\in$ $RHS_R(P[i])$:

if (symbol $\notin v$):

Node = Nodes $\cup$ create_node(symbol)

else if (symbol Node v):

foreach node $\in$ (Nodes):

if (node.link == symbol):

Nodes = Nodes $\cup$ node

Node = Node $\setminus$ node

break

end

end foreach

end

end foreach

link(i, NewLinkedNodes)

$i++$

if ($i > |\text{Number_inference_rules}|$)

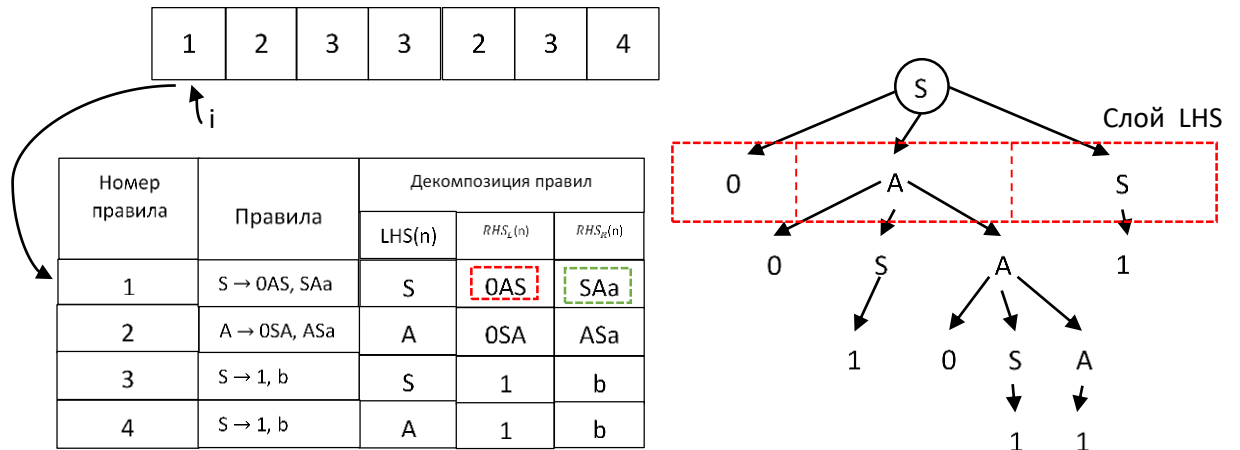
break

end

end

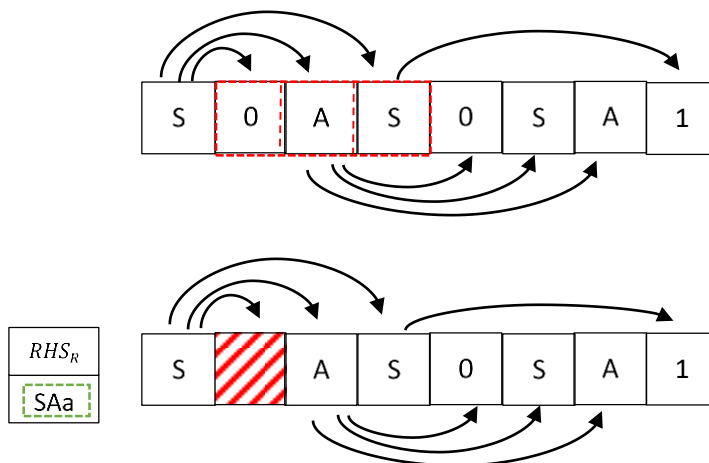
Выполним шаги алгоритма

1. Начинаем проход по Inference_rules и выбираем подходящее правило из таблицы

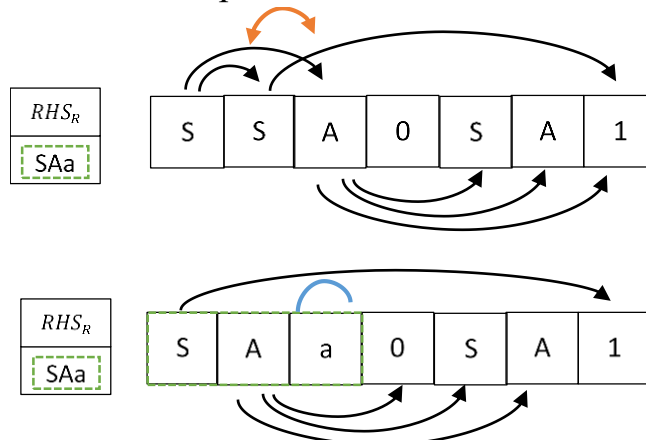


2. По LHS и RHS_L для соответствующего правила находим соответствующую крону и слой

3. Убираем все терминальные символы в RHS_L (Выделенная красная часть соответствует выжденному красным на рисунке)



4. Переставляем оставшиеся нетерминальные символы, а также добавляем терминальные для соответствия RHS_R



5 Продолжаем дальше рекурсивно проходить по Inference_rules и выполнять п. 1-4

Полученное дерево после первого выполнения п. 1-4

Определение. СУ-схема $Tb = (V, \Sigma, \Delta, P, S)$ называется простой, если для каждого правила $A \rightarrow \alpha, \beta \in P$ соответствующие друг другу

вхождения не терминалов встречаются в α и β в одном и том же порядке. Перевод, определяемый *простой* СУ-схемой, называется *простым* синтаксически управляемым переводом.

Соответствие нетерминалов в выводимой паре простой СУ-схемы определяется порядком, в котором эти нетерминалы появляются в цепочках, поэтому для нее легко построить транслятор, представляющий собой преобразователь с магазинной памятью.

Пример 6.4. Рассмотрим простую СУ-схему, имеющую следующие правила:

1. $E \rightarrow (E), E$
2. $E \rightarrow E + E, E + E$
3. $E \rightarrow T, T$
4. $T \rightarrow (T), T$
5. $T \rightarrow A * A, A * A$
6. $T \rightarrow i, i$
7. $A \rightarrow (E + E), (E + E)$
8. $A \rightarrow T, T$

и вывод входной цепочки $((i * (i * i) + i) * i)$, равный 3457358684586863686.

Соответствующий вывод пар цепочек имеет вид:

$(E, E) \Rightarrow_3 (T, T) \Rightarrow_4 ((T), T) \Rightarrow_5 ((A * A), A * A) \Rightarrow_7 (((E + E) * A), (E + E) * A) \Rightarrow_3 (((T + E) * A), ((T + E) * A)) \Rightarrow_5 (((A * A + E) * A), ((A * A + E) * A)) \Rightarrow_8 (((T * A + E) * A), ((T * A + E) * A)) \Rightarrow_6 (((i * A + E) * A), ((i * A + E) * A)) \Rightarrow_8 (((i * T + E) * A), ((i * T + E) * A)) \Rightarrow_4 (((i * (T) + E) * A), ((i * T + E) * A)) \Rightarrow_5 (((i * (A * A) + E) * A), ((i * A * A + E) * A)) \Rightarrow_8 (((i * (T * A) + E) * A), ((i * T * A + E) * A)) \Rightarrow_6 (((i * (i * A) + E) * A), ((i * i * A + E) * A)) \Rightarrow_8 (((i * (i * T) + E) * A), ((i * i * T + E) * A)) \Rightarrow_6 (((i * (i * i) + E) * A), ((i * i * i + E) * A)) \Rightarrow_3 (((i * (i * i) + T) * A), ((i * i * i + T) * A)) \Rightarrow_6 (((i * (i * i) + i) * A), ((i * i * i + i) * A)) \Rightarrow_8 (((i * (i * i) + i) * T), ((i * i * i + i) * T)) \Rightarrow_6 (((i * (i * i) + i) * i), ((i * i * i + i) * i)).$

Для входной цепочки $((i * (i * i) + i) * i)$ СУ-схема порождает выходную цепочку $(i * i * i + i) * i$. анализ позволяет сделать вывод, что СУ-схема отображает арифметические выражения, порождаемые грамматикой G_j , в арифметические выражения, не содержащие избыточных скобок.

6.3. Транслирующие грамматики с операционными символами

Построим процессор, выполняющий перевод инфиксных арифметических выражений в польскую инверсную запись.

Пусть на входной ленте процессора находится цепочка $a + b * c$, и процессор должен работать следующим образом:

- прочитав входной символ, a ;
- выдать символ, a на выходную ленту;
- почитать входной символ '+';
- почитать входной символ b ;

- выдать символ b на выходную ленту;
- почитать входной символ $'*'$;
- почитать входной символ c ;
- выдать символ c на выходную ленту;
- выдать символ $'*'$ на выходную ленту;
- выдать символ $'+'$ на выходную ленту.

Эта последовательность операций определяется порядком следования операндов в инфиксной записи совпадающим с ПОЛИЗ, но операции в ПОЛИЗ должны следовать сразу за своими операндами. Обозначим операции чтения символа из входной ленты - читаемыми символами, а операции записи - символами, помещаемыми на выходную ленту, заключёнными в фигурные скобки (операционными символами), то процесс перевода записывается следующим образом:

$a \{a\} + b \{b\} * c \{c\} \{*\} \{+\}$.

Определение. Транслирующей грамматикой называется пятерка объектов $G^T = (V, \Sigma_i, \Sigma_a, P, S)$, где Σ_i - словарь входных символов, Σ_a - словарь операционных символов, V - нетерминальный словарь, $S \in V$ - начальный символ транслирующей грамматики, P - конечное множество правил продукций вида $A \rightarrow \alpha$, в которых $A \in V$, а $\alpha \in (\Sigma_i \cup \Sigma_a \cup V)^*$. Транслирующая грамматика - это КС-грамматика, множество терминалов которой разбито на два множества: множество входных и множество операционных символов.

Алгоритм трансформации КС-грамматики G_0 в транслирующую грамматику.

Вход : КС-грамматика G_0

Выход : G_0^T - транслирующая грамматика

1. Входной язык L описывается входной КС-грамматикой G_0 .
2. В правила вывода G_0 грамматики вставляются операционные символы для описания семантических действий, связанных с соответствующими правилами.

В восходящих методах обработки языков широко применяются постфиксные транслирующие грамматики.

Определение. Постфиксной транслирующей грамматикой называется транслирующая грамматика у которой все операционные символы в правых частях правил вывода расположены правее всех входных и нетерминальных символов.

Транслирующая грамматика G_0^T , описывающая перевод инфиксных арифметических выражений в ПОЛИЗ, содержит следующие правила:

- | | |
|--------------------------------|----------------------------|
| 1. $E \rightarrow E + T \{+\}$ | 4. $T \rightarrow P$ |
| 2. $E \rightarrow T$ | 5. $P \rightarrow i \{i\}$ |
| 3. $T \rightarrow T * P \{*\}$ | 6. $P \rightarrow (E)$ |

Грамматика G_0^T является постфиксной транслирующей грамматикой.

Определение. G_0 называется входной грамматикой для транслирующей грамматики G_0^T , если она получена из транслирующей грамматики путем вычеркивания всех операционных символов. $L(G_0)$ называется входным языком.

Например, входной грамматикой G_0 для транслирующей грамматики G_0^T является для инфиксных арифметических выражений, содержащая правила:

- | | |
|--------------------------|------------------------|
| 1. $E \rightarrow E + T$ | 4. $T \rightarrow P$ |
| 2. $E \rightarrow T$ | 5. $P \rightarrow i$ |
| 3. $T \rightarrow T * P$ | 6. $P \rightarrow (E)$ |

Рассмотрим левый вывод цепочки $i + i * i$ во входной грамматике G_0^T :

$E \Rightarrow E + T \Rightarrow T + T \Rightarrow P + T \Rightarrow i + T \Rightarrow i + T * P \Rightarrow i + P * P \Rightarrow i + i * P \Rightarrow i + i * i$.

Эта же последовательность правил для транслирующей грамматики G_0^T дает следующий левый вывод:

$E \Rightarrow E + T \{+\} \Rightarrow T + T \{+\} \Rightarrow P + T \{+\} \Rightarrow i \{i\} + T \{+\} \Rightarrow i \{i\} + T * P \{*\} \{+\} \Rightarrow i \{i\} + P * P \{*\} \{+\} \Rightarrow i \{i\} + i \{i\} * P \{*\} \{+\} \Rightarrow i \{i\} + i \{i\} * i \{i\} \{*\} \{+\}$

Цепочки языка, порождаемого транслирующей грамматикой, называют активными цепочками. Входной частью активной цепочки называется последовательностью входных символов, полученная из активной цепочки после удаления всех операционных символов. Операционной частью активной цепочки называется последовательность операционных символов, полученная путем вычёркивания из активной цепочки всех входных символов.

Например, последовательность $i + i * i$ является входной частью активной цепочки $i \{i\} + i \{i\} * i \{i\} \{*\} \{+\}$, а последовательность $\{i\}\{i\}\{i\}\{*\}\{+\}$ - ее операционной частью.

Определение. Синтаксически управляемым переводом $\tau(G^T)$ урррррправляемым грамматикой G^T называется множество всех пар, первыми элементами которых являются входные части активной цепочки, а вторыми элементами — операционные части активной цепочки.

Применим алгоритм к входной грамматике G_0 для получения транслирующей грамматики G_0^T .

В ПОЛИЗ операнды располагаются в той же последовательности, что и в инфиксной записи, а знаки операций следуют непосредственно после своих операндов в том порядке, в котором они выполняются.

(в виде таблицы)

Для того чтобы идентификатор i выдавался сразу после его прочтения, правило (5) грамматики G_0 преобразуется к виду $P \rightarrow i \{i\}$.

Чтобы знак операции сложения выдавался непосредственно после своих операндов, правило (1) заменяется на $E \rightarrow E + T \{+\}$. Это правило интерпретируется следующим образом: обработка арифметического выражения E состоит из обработки первого операнда E , чтения символа '+', обработки второго операнда T и выдачи символа '+'. Аналогичные рассуждения позволяют следующим образом преобразовать правило (3) грамматики: $T \rightarrow T * P \{*\}$.

Утверждение. Один и тот же перевод можно определить разными транслирующими грамматиками

Пример 6.5. Пусть две транслирующие грамматики G_1^T и G_2^T определяющие перевод в ПОЛИЗ инфиксных бесскобочных арифметических выражений, выполняемых в порядке написания операций, выглядят следующим образом:

G_1^T

1. $E \rightarrow E + T \{+\}$
2. $E \rightarrow E * T \{*\}$
3. $E \rightarrow T$
4. $T \rightarrow i \{i\}$

G_2^T

1. $E \rightarrow i \{i\} R$
2. $R \rightarrow + i \{i\} \{+\} R$
3. $R \rightarrow * i \{i\} \{*\} R$
4. $R \rightarrow \varepsilon$

Выполним перевод цепочки $i + i * i$ с использованием транслирующих грамматик G_1^T и G_2^T .

Активная цепочка для G_1^T порождается следующим образом:

$E \Rightarrow^2 E * T \{*\} \Rightarrow^1 E + T \{+\} * T \{*\} \Rightarrow^3 T + T \{+\} * T \{*\} \Rightarrow^4 i \{i\} + T \{+\} * T \{*\} \Rightarrow^4 i \{i\} + i \{i\} \{+\} * T \{*\} \Rightarrow^4 i \{i\} + i \{i\} \{+\} * i \{i\} \{*\}$

Операционная часть цепочки $\{i\} \{i\} \{+\} \{i\} \{*\}$ является переводом входной цепочки $i + i * i$.

Активная цепочка для G_2^T порождается следующим образом:

$E \Rightarrow^1 i \{i\} R \Rightarrow^2 i \{i\} + i \{i\} \{+\} R \Rightarrow^3 i \{i\} + i \{i\} \{+\} * i \{i\} \{*\} R \Rightarrow^4 i \{i\} + i \{i\} \{+\} * i \{i\} \{*\}$

Операционная часть цепочки $\{i\} \{i\} \{+\} \{i\} \{*\}$ также является переводом входной цепочки $i + i * i$. Рис. 5.3 показывает деревья вывода цепочки $i \{i\} + i \{i\} \{+\} * i \{i\} \{*\}$ грамматик G_1^T и G_2^T соответственно.

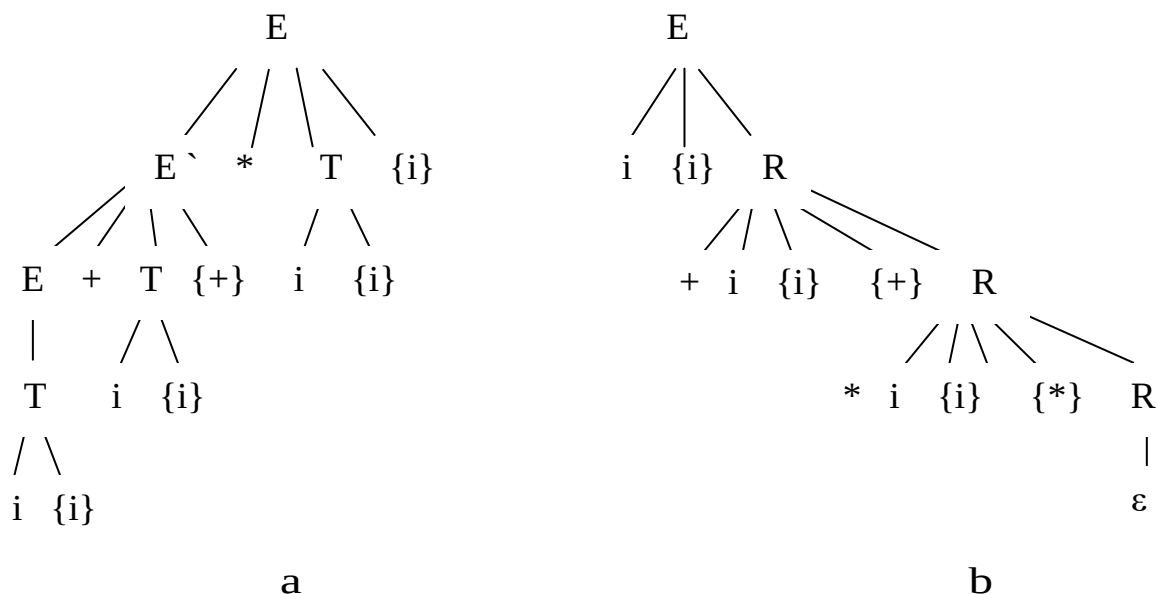


Рис. 6.3. Деревья вывода цепочки $i \{i\} + i \{i\} \{+\} * i \{i\} \{*\}$ по грамматикам G_1^T (a) и G_2^T (b)

Определение. Входную часть активной цепочки называют **входной цепочкой**, операционную часть – **выходной цепочкой**, а транслирующую грамматику G_0^T - грамматикой цепочечного перевода если при реализации появление входного символа в активной цепочке интерпретируется как операция чтения этого символа считывающим устройством, а вхождению операционного символа в правило вывода сопоставляется операция выдачи символа, заключённого в фигурные скобки.

Грамматика G_0^T является примером грамматики цепочечного перевода:

1. $E \rightarrow E + T \{+\}$
2. $E \rightarrow T$
4. $T \rightarrow P$
3. $T \rightarrow T * P \{*\}$
5. $P \rightarrow i \{i\}$
6. $P \rightarrow (E)$

Операционные символы интерпретируются именами семантических процедур, вызываемых при обработке входной цепочки. Активная цепочка задает последовательность наступления событий вызовов этих процедур по отношению к событиям чтения входных символов.

6.4 Атрибутные транслирующие грамматики

При определении транслирующей грамматики в понятие **входного символа** не включалось представление о том, что он является **лексемой**. Лексема состоит из двух компонентов: **типа и значения**.

Транслирующая грамматика описывает перевод только той части символа, который задает его тип. Расширением понятия грамматики цепочечного перевода, использующей при переводе оба компонента входного символа является **атрибутно транслирующая грамматики (АТ-грамматики)**. АТ-грамматики были предложены Дональдом Кнутом.

В АТ-грамматике символы снабжаются **атрибутами**, которые могут принимать значения из некоторого множества допустимых значений и **интерпретируются** как **семантическая** информация, связанная с конкретным вхождением символа в правило вывода грамматики.

При таком подходе значения атрибутов связываются с вершинами дерева вывода в АТ-грамматике, а правила вычисления значений атрибутов сопоставляются правилам вывода грамматики.

Атрибуты нетерминальных и операционных символов делятся на синтезированные и унаследованные атрибуты.

6.4 Синтезированные и унаследованные атрибуты

Пусть задана транслирующая грамматика G^T , допускающая в качестве входа арифметические выражения, построенные из символов множества $\{c, +, *, (,)\}$, где c - лексема, является целочисленной константой и порождающая на выходе значение этого выражения:

0. $S \rightarrow E \{РЕЗУЛЬТАТ\}$

1. $E \rightarrow E + T$ 2. $E \rightarrow T$ 3. $T \rightarrow T * P$ 4. $T \rightarrow P$ 5. $P \rightarrow c$ 6. $P \rightarrow (E)$

На рис. 5.4, а изображено дерево вывода для входной цепочки $c_5 + c_2 * c_3$ (индексы обозначают значения лексем). Входной цепочке должна соответствовать выходная цепочка $\{РЕЗУЛЬТАТ\}$.

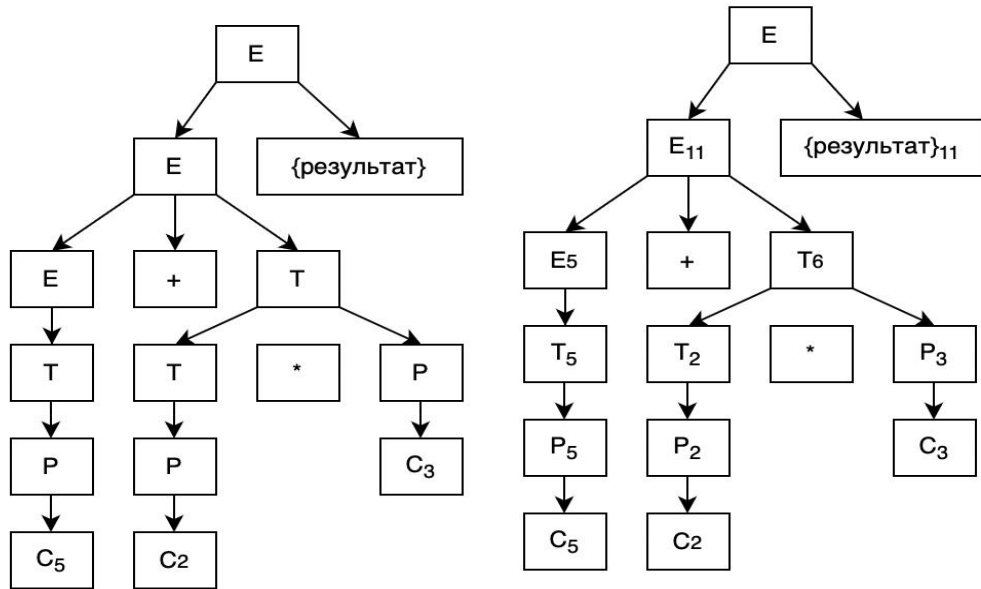


Рис. 5.4. Дерево вывода цепочки $c_5 + c_2 * c_3$ во входной (а) и атрибутивной транслирующей (б) грамматиках.

Вершины не терминалов E , T и P в дереве вывода представляет собой некоторое подвыражение входного выражения.

КР 1 Алгоритм преобразования исходной транслирующей грамматики в АТ-грамматику. (на примере есть КР)

Введём для каждого из символов E , T и P по одному атрибуту, значением которого будет значение подвыражения, порождаемого этим не терминалом.

Входной символ "с" и операционный символ $\{РЕЗУЛЬТАТ\}$ также имеют по одному атрибуту. Значение атрибута символа "с" равно значению константы, которую он представляет, значением операционного символа $\{РЕЗУЛЬТАТ\}$ является результат вычисления выражения.

Атрибуты символов могут принимать любые целочисленные значения из области допустимых для выражений, описываемых соответствующей входной грамматикой G_3 . На рис. 5.4, б изображено дерево вывода той же цепочки, что и на рис. 5.4, а, но на этом дереве не терминалы и операционный символ $\{РЕЗУЛЬТАТ\}$ помечены значениями своих атрибутов.

Выберем для каждого атрибута каждого вхождения символа в правило вывода грамматики уникальное имя и включим атрибуты в правила вывода в виде подстрочных индексов соответствующих символов.

Замечание: одни и те же имена атрибутов можно использовать в нескольких правилах, т. к. они локальны в правиле грамматики.

Сопоставим каждому правилу вывода грамматики правило вычисления значения атрибута не терминала из левой части правила, которому соответствует нетерминальная вершина дерева вывода.

Рассмотрим, например, вершину T дерева, изображённого на рис. 5.4, а. Правило вывода, применённое к этой вершине, имеет вид: $T \rightarrow T * P$. Оно определяет, что значение подвыражения, порождённого не терминалом из левой части правила, равно значению подвыражений, порождённых символами P и T из правой части правила, которые являются прямыми потомками вершины T .

Если p - атрибут символа T из левой части правила, а q и r - атрибуты символов T и P из правой части правила, то правило вычисления значения атрибута p будет иметь вид: $p \leftarrow q * r$, где символ ' $\leftarrow$ ' - знак операции присваивания.

Таким образом, начиная с атрибутов входных символов и поднимаясь по дереву от кроны к корню, можно определить все значения атрибутов нетерминалов из левых частей правил вывода.

Правила вычисления значений атрибутов для АТ-грамматики G_3^T , полученной по транслирующей грамматике G_3^T , будут выглядеть следующим образом:

$$0. S \rightarrow E_q \{РЕЗУЛЬТАТ\}_p \\ q \leftarrow p$$

$$1. E_p \rightarrow E_q + T_r \\ p \leftarrow q + r$$

$$2. E_p \rightarrow T_q \\ p \leftarrow q$$

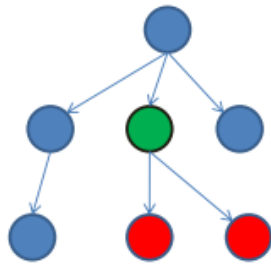
$$3. T_p \rightarrow T_q * P_r \\ p \leftarrow q * r$$

$$4. T_p \rightarrow P_q \\ p \leftarrow q$$

$$5. P_p \rightarrow c_q \\ p \leftarrow q$$

$$6. P_p \rightarrow (E_q) \\ p \leftarrow q$$

Определение. Синтезированными атрибутами называют атрибуты значения которых вычисляются при движении по дереву вывода снизу вверх. Термин "синтезированный" подчёркивает, что значение атрибута синтезируется из значений атрибутов потомков см. рис. Значение атрибута зеленой вершины вычисляется на основе красных вершин.



SDD involving only synthesized attributes is called ***S-attributed***

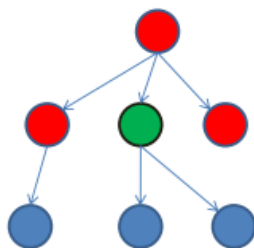
Synthesized Attributes

Attribute of the node is defined in terms of:

- Attribute values at children of the node
- Attribute value at node itself

Атрибуты операционных символов, например, {РЕЗУЛЬТАТ}, относятся к классу унаследованных атрибутов.

Определение. Унаследованными атрибутами называют атрибуты, значения которых вычисляются из значений атрибутов вершины родителя или атрибутов его соседей в дереве вывода см. рис. Значение атрибута зеленой вершины вычисляется на основе красных вершин.



Inherited Attributes

Attribute of the node is defined in terms of:

- Attribute values at parent of the node
- Attribute values at siblings
- Attribute value at node itself

Например, в грамматике G_3^T , значение атрибута p символа {РЕЗУЛЬТАТ}_p равно значению атрибута не терминала E (соседа слева), порождающего все выражение. Оно может быть вычислено только после того, как будут определены значения всех синтезированных атрибутов не терминалов.

6.5. Атрибутная грамматика и построение дерева зависимостей вычислений

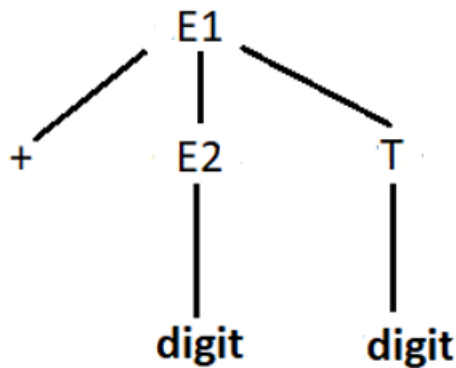
Правила атрибутной грамматики:

Правила продукции	Семантические правила
$E1 \rightarrow + E2 T$	$E1.val := E2.val + T.val$

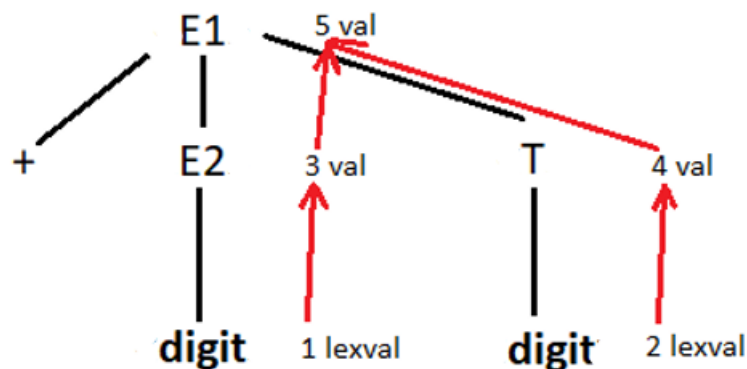
$E2 \rightarrow \mathbf{digit}$	$E2.val := \mathbf{digit.lexval}$
$T \rightarrow \mathbf{digit}$	$T.val := \mathbf{digit.lexval}$

Каждый из нетерминалов $E1$, $E2$ и T имеет синтезированный атрибут val ; терминал **digit** имеет синтезируемый атрибут $lexval$.

Синтаксическое дерево примет вид:



Дерево зависимостей вычислений:



Узлы 1 и 2 представляют атрибут $lexval$, связанный с двумя листьями, с метками **digit**.

Узел 3 представляет атрибут val , связанный с узлом с меткой $E2$. Ребро в узел 3 из 1 получается из семантического правила, определяющего $E2.val$ через **digit.lexval**. В действительности $E2.val$ равно **digit.lexval**, но ребро представляет зависимость, а не равенство.

Ребро в узел 4 из 2 получается из семантического правила, определяющего $T.val$ через **digit.lexval**. В действительности $T.val$ равно **digit.lexval**, но ребро представляет зависимость, а не равенство.

Имеются рёбра в узел 5 из узлов 3 (атрибут $E2.val$) и 4 (атрибут $T.val$), поскольку указанные значения складываются для получения атрибута val в узле 5.

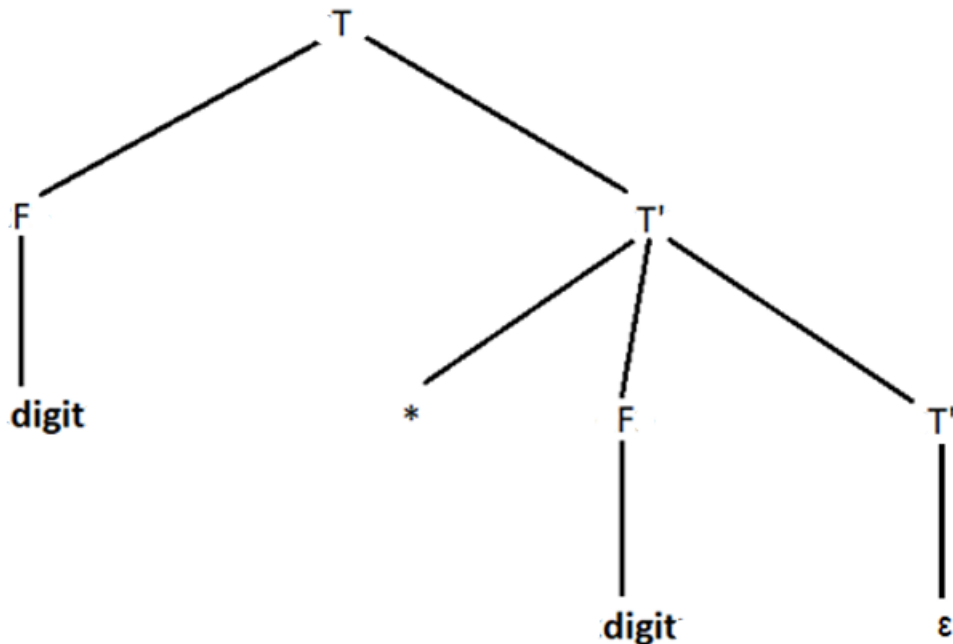
Правила атрибутивной грамматики:

Правила продукции	Семантические правила
$T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$

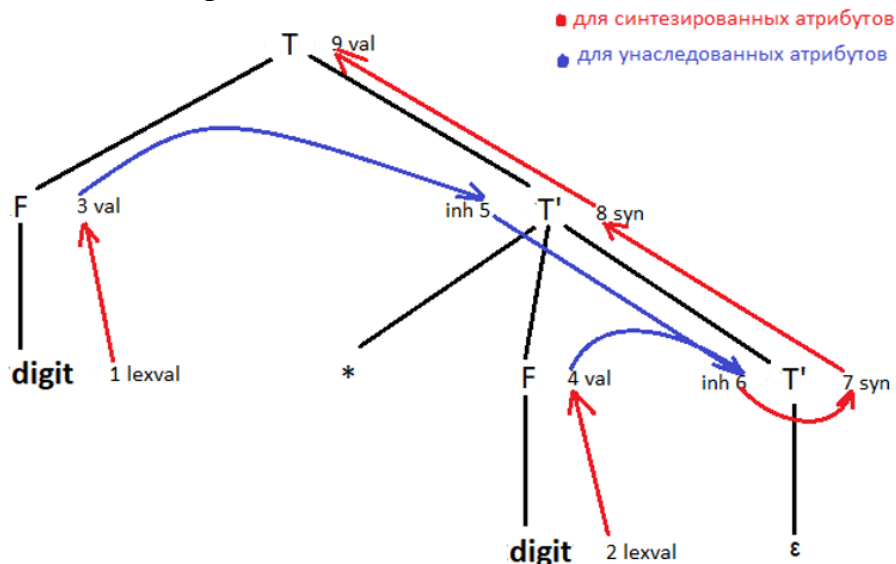
$T' \rightarrow * F T1'$	$T1'.inh = T'.inh * F.val$ $T'.syn = T1'.syn$
$T' \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

Каждый из нетерминалов T и F имеет синтезированный атрибут val ; терминал **digit** имеет синтезированный атрибут $lexval$. Нетерминал T' имеет два атрибута: наследуемый inh и синтезированный syn .

Синтаксическое дерево примет вид:



Дерево зависимостей вычислений:



Узлы 1 и 2 представляют атрибут $lexval$, связанный с двумя листьями с метками **digit**.

Узлы 3 и 4 представляют атрибут val , связанный с двумя узлами с метками F . Рёбра в узел 3 из 1 и в узел 4 из 2 получаются из семантического правила, определяющего $F.val$ через **digit.lexval**. В действительности $F.val$ равно **digit.lexval**, но ребро представляет зависимость, а не равенство.

Узлы 5 и 6 представляют наследуемый атрибут $T'.inh$, связанный с каждым появлением нетерминала T' . Причиной наличия ребра из 3 в 5 служит правило $T'.inh = F.val$, определяющее $T'.inh$ в правом дочернем узле корня с использованием значения $F.val$ в левом дочернем узле. Имеются также рёбра в узел 6 из узлов 5 (атрибут $T'.inh$) и 4 (атрибут $F.val$), поскольку указанные значения перемножаются для получения атрибута inh в узле 6.

Узлы 7 и 8 представляют синтезируемый атрибут syn , связанный с вхождениями T' . Ребро из узла 6 в узел 7 связано с семантическим правилом $T'.syn = T'.inh$, назначенным продукции 3. Ребро из узла 7 в узел 8 связано с семантическим правилом, назначенным продукции 2.

Наконец, узел 9 представляет атрибут $T.val$. Ребро из узла 8 в узел 9 вызвано семантическим правилом $T.val = T'.syn$, связанным с продукцией 1.

Теория к алгоритму:

Атрибутная грамматика $AG = (G, As, Ai, R)$

$G = (V, T, P, S)$ – КС-грамматика

As – конечное множество синтезируемых атрибутов

Ai – конечное множество наследуемых атрибутов

$(As \cap Ai = \emptyset)$

R – конечное множество семантических правил

Атрибутная грамматика AG сопоставляет каждому символу $X \in (V \cup T)$ множество $As(X)$ синтезируемых атрибутов и множество $Ai(X)$ наследуемых атрибутов. Множество всех синтезируемых атрибутов всех символов из $(V \cup T)$ обозначается AS , наследуемых – AI . Атрибуты разных символов являются различными атрибутами. Атрибутная грамматика AG сопоставляет каждому правилу $p \in P$ конечное множество $R(p)$ семантических правил вида

Алгоритм построения дерева по слоям

Вход: Атрибутная грамматика $AG = (G, As, Ai, R)$, -

$NUMBER_INFERENCE_RULES = [1, 2, \dots]$ номера правил цепочки вывода

Выход: Вывод поэтапного обхода дерева

$NODES = [] = create_node(LHS(P(1)))$

$index = 1$ // итератор в $NODES$

$i = 1$ // итератор в $NUMBER_INFERENCE_RULES$ (название ужас)

while (True)

$node = NODES[index]$

if ($LHS(P(NUMBER_INFERENCE_RULES [i])) == node.symbol$)

$leftIndex = index;$

foreach ($X \in RHS(P(NUMBER_INFERENCE_RULES [i]))$)

$NODES = NODES \cup create_node(X)$

$link(index, |NODES|)$

end

$i ++$

end

$rightIndex = |NODES|;$

$BOUNDARIES = BOUNDARIES \cup [leftIndex, rightIndex]$


```

        index++
        if (i > |NUMBER_INFERENCE_RULES|)
            break
        end
    end
end

```

Алгоритм преобразования атрибутивной грамматики в дерево зависимостей вычислений:

Вход: Атрибутивная грамматика $AG = (G, As, Ai, R)$,
 $NUMBER_INFERENCE_RULES = [1, 2, \dots]$ номера правил цепочки вывода,
 проход по слоям дерева разбора в виде упорядоченного множества $NODES$,
 упорядоченное множество интервалов в котором выполняется правило
 продукции $BOUNDARIES$.

Выход: Упорядоченное множество атрибутов $DEPENDENCES$;
 упорядоченное множество индексов вершин, которым соответствуют атрибуты
 $OWNERS$

```

DEPENDENCES = [] // множество атрибутов
SIZE = |DEPENDENCES|
OWNERS = [] // упорядоченное множество индексов вершин из NODES,
               которым соответствуют атрибуты
index = 1 // итератор в NODES
DEPENDENCES = DEPENDENCES ∪ create_node(As(LHS(P(1)))) ∪
create_node(Ai(LHS(P(1)))) // вносим атрибуты первого использованного
символа во множество атрибутов
while (|DEPENDENCES| - SIZE > 0)
    OWNERS = OWNERS ∪ index // ставим атрибутам первого
использованного символа соответствие с вершиной
end
foreach(n ∈ NUMBER_INFERENCE_RULES)
    foreach(X ∈ RHS(P(n)))
        SIZE = |DEPENDENCES|
        DEPENDENCES = DEPENDENCES ∪ create_node(As(X)) ∪
create_node(Ai(X)) // вносим атрибуты каждого использованного символа во
множество атрибутов
        while (|DEPENDENCES| - SIZE > 0) // пока мы не рассмотрим все
добавленные атрибуты
            OWNERS = OWNERS ∪ index // ставим атрибутам
использованного символа соответствие с вершиной
            SIZE++
        end
        index++;
    end
end
end

```

Вход: Атрибутная грамматика $AG = (G, As, Ai, R)$,
 $NUMBER_INFERENCE_RULES = [1, 2, \dots]$ номера правил цепочки вывода,
упорядоченное множество интервалов в котором выполняется правило
продукции $BOUNDARIES$, упорядоченное множество индексов вершин,
которым соответствуют атрибуты $OWNERS$.

Выход: Дерево зависимостей имеющее вид упорядоченного множества
 $DEPENDENCES$ со ссылками;

```

i = 1 // итератор в NUMBER_INFERENCE_RULES
while(i <= |NUMBER_INFERENCE_RULES|)
    left = BOUNDARIES[i][0] //индекс, который отвечает за индекс левой
    границы в OWNERS
    right = BOUNDARIES[i][1] //индекс, который отвечает за индекс правой
    границы в OWNERS
    leftIndex = 1 //итератор по OWNERS, который ищет левую границу
    rightIndex = 1 //итератор по OWNERS, который ищет правую границу
    while (OWNERS[rightIndex] <= right)
        if (OWNERS[leftIndex] < left)
            leftindex++
        end
        rightindex++
    end
    index = leftIndex++ //Первое вхождение left в OWNERS, index – итератор
    для DEPENDENCES
    foreach(X ∈ R(P(i)))
        if (|RHS(X)| == 1)
            while(index < rightIndex)
                if (DEPENDENCES[index] == LHS(X))
                    leftPart = index //индекс в который придёт ссылка
                end
                if (DEPENDENCES[index] == RHS(X))
                    rightPart = index // индекс из которого идёт ссылка
                end
                index++
            end
            link(rightPart, leftPart)
        end
        if (|RHS(X)| > 1)
            foreach(ELEM ∈ RHS(X))
                if(ELEM ∈ (As ∪ Ai))
                    while(index < rightIndex)
                        if (DEPENDENCES[index] == LHS(X))
                            leftPart = index // индекс в который придёт ссылка
                        end
                        if (DEPENDENCES[index] == RHS(X))
                            rightPart = index // индекс из которого идёт ссылка

```

```

                                end
                                index++
                            end
                            link(rightPart, leftPart)
                        end
                    end
                end
            end
        end
    end
end

```

6.5 Алгоритм вычисления атрибутов

Вход: NODES – массив зависимостей вершин вида [symbol, attr], LINKS – массив ссылок вида [start, end] и атрибутивная грамматика $AG = (G, A, R)$, где $G = (V, T, P, S)$ – КС-грамматика
 A – конечное множество атрибутов
 R – конечное множество семантических правил

Выход: вычисленные унаследованные атрибуты

Реализация алгоритма:

```

l = [start, end]
INHERIT_ATTRIBUTES := ∅
foreach (node | node ∈ NODES)
    i := 0
    foreach (l | l ∈ LINKS и end.attr == node.attr)
        i++
        INHERIT_ATTRIBUTES := INHERIT_ATTRIBUTES ∪ start.attr
    end
    if (i > 1) // количество ссылающихся на эту вершину >1
        j = 1 // индекс правила вычисления атрибута
        foreach (X | X → Y и X → Y ∈ P)
            if (X == node.symbol) // нашли правило вычисления атрибута
                Z := RHS(R(P(j)))
                Z = {Z1, Z2, Z3}
                node.attr := INHERIT_ATTRIBUTES[1] Z2
            INHERIT_ATTRIBUTES[2]
            else
                j++
            end
        end
    end
end

```

```

end
INHERIT_ATTRIBUTES := ∅
else if (i == 1)
    node.attr = start.attr //если атрибут синтезированный
end

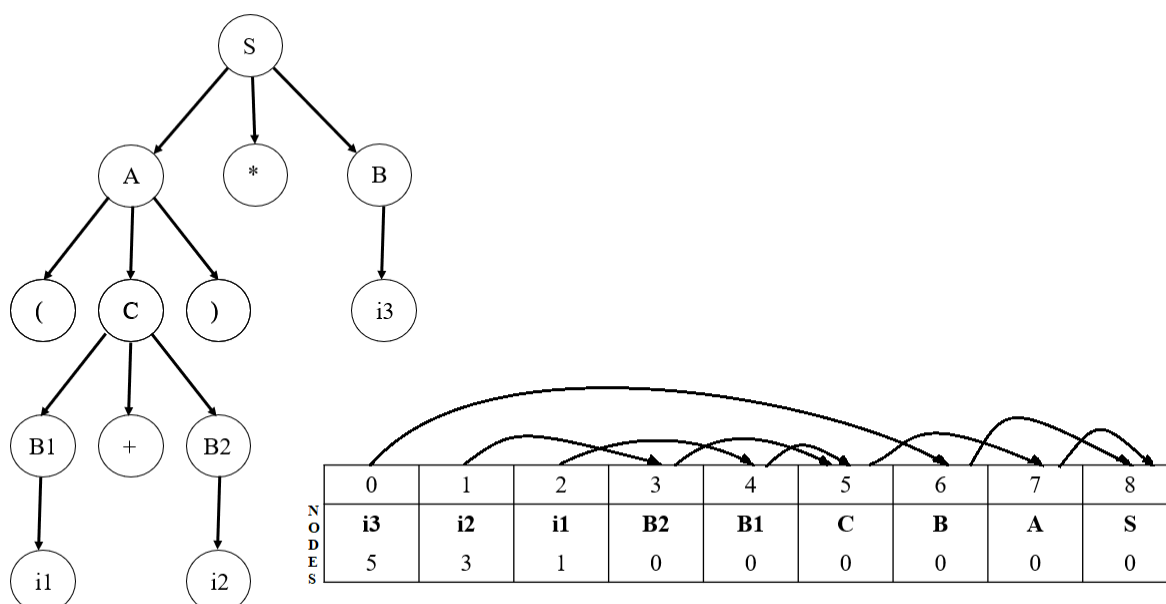
```

Пример работы программы:

Таблица NODES: S 0 A 0 B 0 C 0 B 0 i 1 i 3 i 5

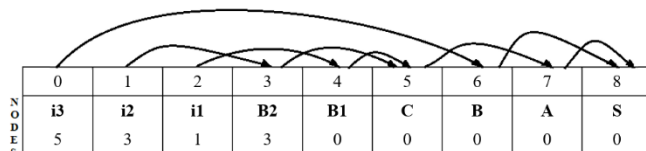
Таблица LINKS: 2 4 1 3 0 6 4 5 3 5 7 7 8 6 8

Представление дерева в одномерном массиве:

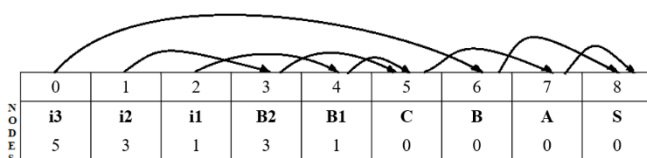


Выполним шаги алгоритма:

- 1) Для первой ноды ничего не произойдет, т. к. в нее не идет ни одна ссылка
- 2) Для второй и третьей ноды ничего не произойдет (по аналогии с п.1)
- 3) Для B2: в нее ведет одна ссылка, поэтому копируем атрибут из NODES[1] в NODES[3]



- 4) Для B1 аналогично п.3



- 5) На ноду С ссылаются две другие (B1 и B2), поэтому она вычисляется по правилу $B1 + B2$

	0	1	2	3	4	5	6	7	8
N O D E S	i3	i2	i1	B2	B1	C	B	A	S
	5	3	1	3	1	4	0	0	0

- 6) Для ноды В аналогично п.3

	0	1	2	3	4	5	6	7	8
N O D E S	i3	i2	i1	B2	B1	C	B	A	S
	5	3	1	3	1	4	5	0	0

- 7) Для ноды А аналогично п.3

	0	1	2	3	4	5	6	7	8
N O D E S	i3	i2	i1	B2	B1	C	B	A	S
	5	3	1	3	1	4	5	4	0

- 8) Для ноды S аналогично п.5, но правило вычисления атрибута другое: $A * B$

	0	1	2	3	4	5	6	7	8
N O D E S	i3	i2	i1	B2	B1	C	B	A	S
	5	3	1	3	1	4	5	4	20

6.6. Пример АТ-Грамматики.

Кр (другой пример Python ?создание списка переменных) Пример 6.7
 Пусть входная КС-грамматика G , порождающая описания переменных в некотором языке программирования, выглядит следующим образом:

1. $D \rightarrow t i L$ 2. $L \rightarrow , i L$ 3. $L \rightarrow \varepsilon$

Множество входных символов этой грамматики состоит из лексем: запятая, идентификатор i и имя типа t .

Семантика обработки описания заключается в занесении типа переменной в определённое поле элемента таблицы идентификаторов с помощью семантической операции установить ТИП с двумя параметрами: указатель на элемент таблицы идентификаторов, соответствующей описываемой переменной, и тип переменной.

Операцию установить ТИП лучше всего вызывать сразу после распознавания идентификатора. Указанная последовательность действий может быть описана транслирующей грамматикой G^T :

- $D \rightarrow t i \{ \text{ТИП} \} L$

$$L \rightarrow , i \{ \text{ТИП} \} L$$

$$L \rightarrow \varepsilon$$

Введём в транслирующую грамматику G^T атрибуты и правила их вычисления. Входные символы t и i имеют по одному атрибуту. Значением атрибута символа i является указатель на элемент таблицы идентификаторов, а атрибут символа t может принимать значения из множества {ЦЕЛЫЙ, ВЕЩЕСТВЕННЫЙ, ЛОГИЧЕСКИЙ}.

Операционный символ {ТИП} должен иметь два унаследованных атрибута, значения которых совпадают со значениями соответствующих фактических параметров процедуры установить ТИП. Значения унаследованных атрибутов символа {ТИП} для первого правила вывода приравниваются значениям соответствующих атрибутов входных символов i и t , входящих в правую часть того же правила левее символа {ТИП}.

Во второе правило тип описываемых переменных можно передать через унаследованный атрибут не терминала L . Значение этого унаследованного атрибута будет передаваться по дереву сверху вниз, начиная с вершины, где он получает начальное значение, равное значению атрибута входного символа t .

Атрибутная транслирующая грамматика G^{AT} выглядит следующим образом:

1. $D \rightarrow t_r i_a \{ \text{ТИП} \}_{a1, r1} L_{r2}$

$$a1 \leftarrow a$$

$$r1, r2 \leftarrow r$$

2. $L_r \rightarrow , i_a \{ \text{ТИП} \}_{a1, r1} L_{r2}$

$$a1 \leftarrow a$$

$$r1, r2 \leftarrow r$$

3. $L_r \rightarrow \varepsilon$

Замечание: запись атрибутного правила в виде $r1, r2 \leftarrow r$ означает, что значение r присваивается одновременно атрибутам $r1$ и $r2$.

Входной цепочке $t_{\text{целый}} i_5, i_9, i_2$ соответствует дерево вывода в грамматике G^{AT} , изображённое на рис. 5.5. (414)

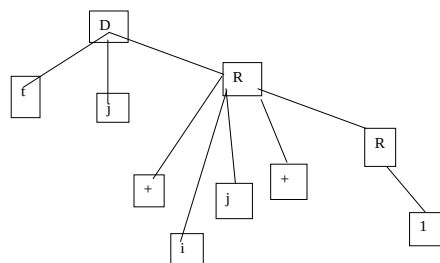


Рис. 6.5. Дерево вывода

В рассмотренных примерах все операционные символы имели унаследованные атрибуты. При определении атрибутной транслирующей грамматики можно обойтись без синтезированных атрибутов операционных

символов. Необходимость в синтезированных атрибутах операционных символов может возникнуть в некоторых практических реализациях переводов, поэтому в определение АТ-Грамматики включены оба типа атрибутов.

6.4.3. Определение атрибутной транслирующей грамматики

Атрибутная транслирующая грамматика – это транслирующая грамматика, обладающая следующими дополнительными свойствами:

1. Каждый символ грамматики (входной, нетерминальный и операционной) имеет конечное множество атрибутов, и каждый атрибут имеет (возможно, бесконечное) множество допустимых значений.

2. Все атрибуты нетерминальных и операционных символов делятся на синтезированные и унаследованные.

3. Унаследованные атрибуты подчиняются следующим правилам:

- каждому вхождению унаследованного атрибута в правую часть правила вывода сопоставляется правило, позволяющее вычислить значение этого атрибута как функцию некоторых других атрибутов символов, входящих в левую или правую часть данного правила;

- для каждого унаследованного атрибута начального символа грамматики задаётся начальное значение.

4. Правила вычисления значений синтезированных атрибутов определяются следующим образом:

- каждому вхождению синтезированного атрибута нетерминального символа в левую часть правила вывода сопоставляется правило, позволяющее вычислить значение этого атрибута как функцию некоторых других атрибутов символов, входящих в левую или правую часть данного правила;

- каждому синтезированному атрибуту операционного символа сопоставляется правило, позволяющее вычислить значение этого атрибута как функцию некоторых других атрибутов данного символа.

- каждому синтезированному атрибуту операционного символа сопоставляется правило, позволяющее вычислить значение этого атрибута как функцию некоторых других атрибутов данного символа.

$G_{AT} = (S_0, V, T, \{OP\}, Type, P)$

T

у

§

$\bar{V}$ – множество терминальных символов.

$\bar{T}$ – множество терминальных символов.

OP – множество операционных символов.

Type – множество атрибутов, разделенное на два множества: синтезируемых и наследуемых атрибутов. Атрибуты для символа записываются после точки подряд.

h Syn – множество синтезируемых атрибутов

синтезируемые атрибуты будут обозначаться символом «%» перед ним.

Inh – множество наследуемых атрибутов

наследуемые атрибуты будут обозначаться символом «/» перед ним.

P – множество правил порождения.

T

§

h

$Inh = \{ ..., e_{i1}, ..., e_{im}, ..., e_{j1}, ..., e_{jl}, ..., e_{k1}, ..., e_{kz} \}$

е

§:

$\bar{y}$

$\bar{v}$

$\bar{f}$

$\bar{f}$

$\bar{f}$

$\bar{f}$

$\bar{f}$

$S_1, ..., S_{im}, e_{i1}, ..., e_{im} \rightarrow V_j.S_{j1}, ..., S_{jr}.e_{j1}, ..., e_{jl} \{OP\}_k.S_{k1}, ..., S_{kv}.e_{k1}, ..., e_{kz}$ (любая

цепочка) $\leftarrow f(S_{in}, e_{i1}, ..., e_{im}, S_{j1}, ..., S_{jr}, S_{k1}, ..., S_{kv})$

$S_{i1} \leftarrow f(S_{i1}, e_{i1}, ..., e_{im}, S_{j1}, ..., S_{jr}, S_{k1}, ..., S_{kv})$

$, ..., S_{in}, ..., S_{j1}, ..., S_{jr}, ..., S_{k1}, ..., S_{kv}$

$e_{j1} \leftarrow f(e_{j1}, e_{i1}, ..., e_{im}, S_{j1}, ..., S_{jr}, S_{k1}, ..., S_{kv})$

...

$e_{jl} \leftarrow f(e_{jl}, e_{i1}, ..., e_{im}, S_{j1}, ..., S_{jr}, S_{k1}, ..., S_{kv})$

$e_{k1} \leftarrow f(e_{j1}, e_{i1}, ..., e_{im}, S_{j1}, ..., S_{jr}, S_{k1}, ..., S_{kv})$

...

$e_{kz} \leftarrow f(e_{j1}, e_{i1}, ..., e_{im}, S_{j1}, ..., S_{jr}, S_{k1}, ..., S_{kv})$

Атрибуты записываются в виде индексов соответствующих символов АТ-грамматики, при этом для каждого атрибута указывается его тип. Например:

$X_{a,b}$ синтезированный a, унаследованный b

Правила вычисления атрибутов записываются в виде операторов присваивания, левая часть которых - атрибут или список атрибутов, а правая часть - функция.

Рассмотрим АТ-грамматику G_A , описывающую перевод оператора присваивания некоторого языка программирования в цепочку тетрад с кодами операций: СЛОЖИТЬ, УМНОЖИТЬ, ПРИСВОИТЬ.левой частью оператора присваивания является идентификатор, а правой частью - бесскобочное арифметическое выражение, выполняемое слева направо в порядке написания операций.

Входными символами грамматики являются символы множества $\{i, +, *, =\}$, где i - лексема, представляющая идентификатор. Каждый идентификатор имеет один атрибут, значением которого является указатель на соответствующий элемент таблицы идентификаторов.

Операционные символы $\{+\}$, $\{*\}$, $\{=\}$ соответствуют кодам операций тетрад, получаемых на выходе. Символы $\{+\}$ и $\{*\}$ имеют по три унаследованных атрибута, значениями которых являются указатели на элементы таблицы, в которой хранятся значения левого операнда, правого операнда и результата соответственно. Операционный символ $\{=\}$ имеет два унаследованных атрибута. Значение первого атрибута - указатель на элемент таблицы идентификаторов, соответствующий идентификатору из левой части оператора присваивания, а значением второго атрибута является указатель на элемент таблицы, в котором хранится значение выражения из правой части оператора присваивания.

Словарь нетерминальных символов состоит из символов S , E и R . Начальный символ S не имеет атрибутов, символ E имеет один синтезированный атрибут, значением которого является указатель на элемент таблицы, содержащий значение выражения, порождаемого E , а символ R имеет два атрибута: унаследованный (первый) и синтезированный. Значением унаследованного атрибута является указатель на элемент таблицы, соответствующий промежуточному результату, предшествующему R . Этот промежуточный результат является также левым операндом первой операции, порождаемой нетерминальным символом R , если такая операция имеет место. Значение синтезированного атрибута символа R - это указатель на элемент таблицы, соответствующий значению подвыражения, которое получается после присоединения цепочки, порождённой символом R , к цепочке, представляющей левый операнд.

Правила вывода АТ-Грамматики G_A с соответствующими правилами вычисления следующих атрибутов:

E_t синтезированный t

$R_{p,t}$ унаследованный p синтезированный t

Атрибуты операционных символов унаследованные.

1) $S \rightarrow i_{p_1} = E_{q_1} \{=\} p_2, q_2$

$p_2 \leftarrow p_1$

$q_2 \leftarrow q_1$

2) $E_{t_2} \rightarrow i_{p_1} R_{p_2, t_1}$

$p_2 \leftarrow p_1$

$$\begin{aligned}
& t_2 \leftarrow t_1 \\
& 3) R_{p_1, t_2} \rightarrow i_{q_1} \{+\} p_2, q_2, r_1 R_{p_2, t_1} \\
& r_1, r_2 \leftarrow \text{GETNEW} \\
& p_2 \leftarrow p_1 \\
& q_2 \leftarrow q_1 \\
& t_2 \leftarrow t_1 \\
& 4) R_{p_1, t_2} \rightarrow^* i_{q_1} \{*\} p_2, q_2, r_1 R_{p_2, t_1} \\
& \quad r_1, r_2 \leftarrow \text{GETNEW} \\
& \quad p_2 \leftarrow p_1 \\
& q_2 \leftarrow q_1 \\
& t_2 \leftarrow t_1 \\
& 5) R_{p_1, t_2} \rightarrow \varepsilon \\
& p_2 \leftarrow p_1
\end{aligned}$$

В атрибутных правилах, связанных с правилами вывода 3) и 4), используется процедура-функция без параметров GETNEW, которая выдаёт значение указателя на свободную позицию таблицы, где будут запоминаться промежуточные результаты. Эта процедура не является функцией, т. к. при разных обращениях к ней получаются разные результаты. Пользуясь процедурой GETNEW, мы несколько отступаем от определения АТ-грамматики.

Вместо процедуры GETNEW, можно ввести дополнительные атрибуты для запоминания адресов занятых позиций таблицы. Однако при использовании процедуры GETNEW атрибутные правила получаются более простыми, и такой подход можно порекомендовать при практическом конструировании компиляторов.

АТ-Грамматики используются для получения атрибутных деревьев вывода, атрибутных активных цепочек и атрибутных переводов.

Кр: 2 Алгоритм. Построение атрибутного дерева вывода.

foreach (i [1,4,2]) S => * w задана

1. Для транслирующей грамматике построить дерево вывода активной цепочки, состоящей из входных и операционных символов без атрибутов.

2. Присвоить начальные значения унаследованным атрибутам начального символа грамматики.

3. Присвоить значения атрибутам входных символов, входящих в дерево вывода.

4. Найти атрибут, отсутствующий в дереве вывода, аргументы правила вычисления которого уже известны. Вычислить значение этого атрибута и добавить его в дерево.

5. Повторять шаг 4 до тех пор, пока значения всех атрибутов символов, входящих в дерево вывода, не будут вычислены или пока шаг 4 может выполняться.

На рис. 5.6 приведено атрибутное дерево вывода в АТ-грамматике, соответствующее входной цепочке $i_5 = i_7 + i_2 * i_3$ при условии, что процедура GETNEW выдаёт последовательные адреса ячеек, начиная с адреса 100. (i2 bug)

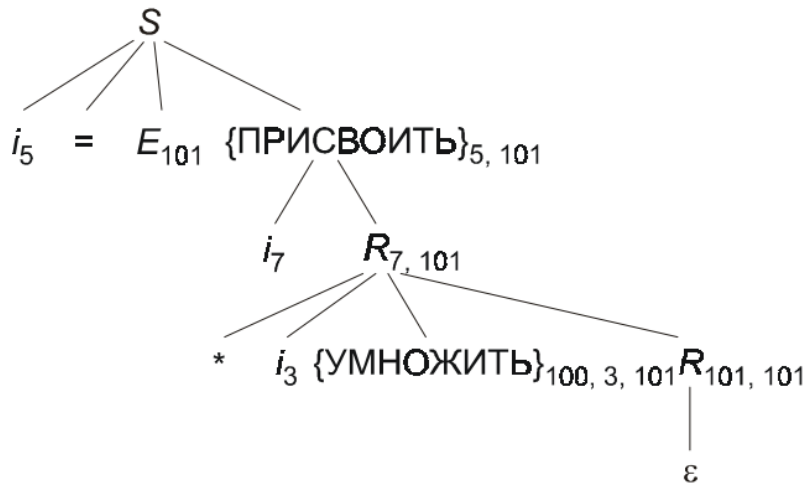


Рис. 5.6

При применении правила (5) синтезированному атрибуту не терминала R присваивается значение его унаследованного атрибута. Это значение передаётся вверх по дереву вывода как синтезированный атрибут не терминала из левой части правил (4), (3) и (2) и используется в качестве второго атрибута операционного символа {ПРИСВОИТЬ}.

Последовательность атрибутных входных и операционных символов, полученная по атрибутному дереву вывода в АТ-грамматике, называется атрибутной активной цепочкой.

Множество пар, первым элементом которых является атрибутная входная цепочка, а вторым элементом - атрибутная последовательность операционных символов атрибутной активной цепочки, называется атрибутным

Пример 6.8. Грамматика G_A для входной цепочки $i_5 = i_7 + i_2 * i_3$ определяет следующую атрибутную последовательность операционных символов:

$$\{+\}_{7,2,100} \{*\}_{100,3,101} \{:=\}_{5,100}$$

Определение. Дерево вывода в АТ-грамматике называется завершённым, если в результате применения алгоритма построения дерева значения всех атрибутов всех символов окажутся вычисленными.

Для каждого атрибута, входящего в дерево вывода, существует правило его вычисления, однако может оказаться, что между атрибутами возникла циклическая зависимость, из-за чего нельзя получить завершённое дерево.

Определение. АТ-грамматика называется корректной тогда и только тогда, когда в результате применения алгоритма построения атрибутного дерева вывода получается завершённое дерево.

Рассмотрим два подкласса корректных АТ-грамматик, используемых при проектировании компиляторов: L-атрибутные транслирующие грамматики и S-атрибутные транслирующие грамматики.

6.4.4. Вычисление значений атрибутов

После построения дерева вывода входной цепочки возникает вопрос о порядке вычисления значений атрибутов.

По определению, атрибут можно вычислить, если известны значения всех атрибутов, от которых зависит его значение. Число деревьев вывода так же, как и входных цепочек, бесконечно, поэтому важно по самой грамматике уметь определять, является ли множество её правил вычисления атрибутов корректным.

Алгоритмы проверки корректности АТ-Грамматики приведены в [2, 25]. Они основаны на построении графа зависимостей и его анализе.

Узлами графа зависимостей служат атрибуты, которые нужно вычислить, а дугам ставятся в соответствие зависимости, определяющие, какие атрибуты вычисляются раньше, а какие позже.

Граф зависимостей $R(D)$ представляет собой ориентированный граф, который строится для некоторого дерева вывода D следующим образом:

- узлами $R(D)$ являются пары (X, a) , где X - узел дерева D , а a - атрибут символа, служащего меткой узла X ;
- дуга из узла (X_1, a_1) в узел (X_2, a_2) проводится, если семантическое правило, вычисляющее значение атрибута a_1 , непосредственно использует значение атрибута a_1 .

Вычисление значения двоичного числа по его символьному представлению можно задать при помощи следующей атрибутной грамматики [24]:

$$N_{v_4} \rightarrow L_{v_2, l_2, S_2} \cdot L_{v_3, l_3, S_{23}}$$

$$L_{v_2, l_2, S_2} \rightarrow B_{v_1, S_1}$$

$$V_4 \leftarrow V_2 + V_3$$

$$V_2 \leftarrow V_1$$

$$S_2 \leftarrow 0$$

$$S_1 \leftarrow S_2$$

$$S_3 \leftarrow -l_3 l_2 \leftarrow 1$$

$$N_{v_4} \rightarrow L_{v_2, l_2, S_2}$$

$$B_{v_1, S_1} \rightarrow 0$$

$$V_4 \leftarrow V_2 V_1 \leftarrow 0$$

$$S_2 \leftarrow 0$$

$$L_{v_2, l_2, S_2} \rightarrow L_{v_3, l_3, S_3} B_{v_1, S_1}$$

$$B_{v_1, S_1} \rightarrow 1$$

$$V_2 \leftarrow V_3 + V_1 V_1 \leftarrow 1$$

$$S_1 \leftarrow S_2$$

$$S_3 \leftarrow S_2 + 1$$

$$l_2 \leftarrow l_3 + 1$$

В этой грамматике атрибуты имеют следующую семантику: v — значение (рациональное число), s — масштаб (целое число) и — длина числа (целое число).

Используя грамматику, построим дерево D вывода цепочки 101.01

Для построения узлов графа зависимостей необходимо последовательно рассмотреть вершины дерева D и для каждой из них построить столько узлов, сколько атрибутов имеет символ, которым помечена эта вершина.

Например, для корня дерева, обозначенного символом N с одним атрибутом, нужно в граф включить один узел, поместив его парой (N, V_4) .

Для определения дуг графа необходимо использовать семантические правила. Например, рассмотрим узел Дерева зависимостей (N, V_4) . При построении дерева D непосредственные потомки корня N определялись правилом грамматики $N_{v_4} \rightarrow L_{v_2, l_2, s_2} \cdot L_{v_3, l_3, s_3}$. Атрибут V_4 не терминала N зависит, в соответствии с семантическим правилом $V_4 \leftarrow V_2 + V_3$, от атрибута V_2 левого сына, помеченного символом L , и атрибута V_3 правого сына. Поэтому в граф зависимостей необходимо включить дуги $((L, V_2), (N, V_4))$ и $((L, V_3), (N, V_4))$. Поступая аналогичным образом, мы можем построить граф зависимостей.

В [24] доказано, что атрибутная грамматика корректна, если граф зависимостей не имеет циклов.

Общие алгоритмы вычисления атрибутов весьма неэффективны, поэтому на практике используют методы, в которых для определения порядка вычисления не рассматриваются семантические правила. Например, если порядок вычисления атрибутов задаётся методом синтаксического анализа, то он не требует анализа семантических правил и построения графа зависимости.

6.5. Методика проектирования перевода

При проектировании перевода следует руководствоваться следующим:

1. Описание перевода строится последовательно для конструкций входного языка в порядке их усложнения, начиная с простейших конструкций, например: выражение, оператор присваивания, оператор цикла.

2. Выходная цепочка (результат перевода) включает в себя объекты (сущности) трех видов:

- объекты, передаваемые в выходную цепочку из входной цепочки;
- объекты, не изменяющиеся в процессе перевода (терминалы выходного языка);
- объекты, генерируемые во время перевода.

3. Транслирующая грамматика определяет порядок применения операционных символов (элементов перевода), строящих выходную цепочку,

атрибуты же используются только для передачи значений символов из одних правил в другие.

4. Операционный символ, включаемый в правило вывода грамматики, может генерировать выходную цепочку до тех пор, пока в неё не должен быть включён объект, перемещаемый из входной цепочки и недоступный в данном правиле грамматики.

Проектирование описания перевода некоторой (одной!) конструкции входного языка выполняется в несколько этапов.

Неформальное описание перевода. Неформальное описание перевода представляет собой пару цепочек, первая из которых является конструкцией входного языка, а вторая - ее представлением в выходном языке. При разработке описания перевода рекомендуется:

- строить описание перевода на основе описания синтаксиса и семантики входного языка, учитывая все возможные форматы представления входной конструкции (например, if-then и if-then-else);
- конструкции, перевод которых уже описан и которые являются частью данной конструкции, считать терминальными;
- после построения неформального описания перевода выделить в нем символы, передаваемые из входной цепочки, и символы, генерируемые при его построении.

Описание синтаксиса. Описание синтаксиса выполняется в БНФ или расширенной БНФ. После стандартных преобразований БНФ преобразуется в КС-Граматику, описывающую рассматриваемую конструкцию входного языка.

Определение транслирующей грамматики. проанализируем последовательность используемых при построении перевода правил грамматики, а также смоделируем действия, выполняемые операционными символами грамматики. Должны быть уточнены:

1. метод синтаксического анализа, используемый при анализе и переводе данной конструкции, и способ его реализации;
 2. интерпретация операционных символов (Цепочка символов, помещаемых на выходную ленту, или имя подпрограммы, которую необходимо выполнить).
- Исходными данными при построении транслирующей грамматики являются: КС-Грамматика, описывающая синтаксис конструкции;
 - последовательность правил грамматики (разбор), используемых процессором при построении перевода;
 - объекты, доступные процессору, выполняющему перевод (для процессора с магазинной памятью это текущий символ входной цепочки и, в зависимости от метода синтаксического анализа, весь магазин или его часть).

Для построения транслирующей грамматики необходимо выполнить следующие действия:

1. Выбрать простейшую входную цепочку, соответствующую данной конструкции, и определить ее разбор.

2. В неформальном описании перевода выделить очередной (вначале первый) символ выходной цепочки, который должен быть передан в неё из входной цепочки.

3. На основании доступности (видимости) данных процессору и с учётом разбора, определить правило грамматики, в которое нужно включить элемент перевода (операционный символ), переносящий нужный символ в выходную цепочку. Если действия успешны, то перейти к шагу 5, иначе выполнить следующий шаг 4.

4. Если нельзя выполнить шаг 3, то соответствующий операционный символ включить в наиболее подходящее правило грамматики (в дальнейшем этот символ будет передан в него при помощи атрибутов), определив, если нужно, дополнительную выходную ленту процессора.

5. Возложить на новый (включённый) операционный символ действия по записи в выходную цепочку всех символов, вплоть до следующего символа, который должен быть передан из входной цепочки.

6. Повторить шаги 2-5 для всех символов, передаваемых из входной цепочки.

7. Тестировать транслирующую грамматику на более сложных, например, вложенных входных цепочках.

В простых случаях (простая конструкция входного языка, простое ее представление в выходном языке, удачные метод синтаксического анализа и его реализация, удачное расположение операционных символов) может оказаться, что для описания перевода достаточно только транслирующей грамматики.

Определение атрибутивной транслирующей грамматики. При необходимости передачи данных между узлами синтаксического дерева транслирующую грамматику следует расширить до атрибутивной транслирующей грамматики.

По определению, передача значений атрибутов в атрибутивных транслирующих грамматиках возможна только в следующих направлениях:

- от символа левой части символам правой части правила вывода;
- от символов правой части правила символу левой части этого же правила вывода;
- между символами правой части правила вывода.

Для реализации передачи значения символа необходимо выполнить следующие действия:

1. Определить источник передаваемого значения (символ грамматики) и приписать ему атрибут.

2. Определить приёмник значения (символ грамматики) и приписать ему атрибут.

3. Учитывая ограничения в передаче атрибутов, определить маршрут передачи значения, для чего удобно пользоваться синтаксическим деревом грамматики.

4. Описать передачу значения атрибута правилами АТ-грамматики, стараясь, чтобы функции вычисления атрибутов были простейшими (копирующие правила).

Тестирование АТ-грамматики. Тестирование АТ-грамматики заключается в моделировании работы построенного по ней процессора, выполняющего перевод. При большом числе тестов и/или большой их длине эта работа может быть выполнена только при использовании соответствующих средств

автоматизации. Для простых языковых конструкций, когда длина теста и их число невелики, а тестирование выполняется отдельно для каждой конструкции, начиная с простейших, работа эта не только необходима, но и реально выполнима.

Тестирование АТ-Грамматики позволяет определять два вида ошибок:

- ошибки в структуре выходной цепочки (неверный порядок следования символов в выходной цепочке);
- ошибки в значениях символов выходной цепочки, что связано с ошибками в передаче значений атрибутов.

При тестировании реального описания перевода желательно использовать комплексные тесты, определяющие оба вида ошибок.

6.6. Пример проектирования АТ-грамматики

Выполним перевод оператора цикла, имеющего следующий формат:

```
for <параметр> = <начальное значение> to <конечное значение> step <шаг>  
<тело цикла> next <параметр>
```

Введём следующие ограничения:

<параметр> - идентификатор целой переменной;

<начальное значение>, <конечное значение>, <шаг> - целые положительные константы;

<тело цикла> - оператор цикла или терминал (не подлежащий переводу оператор);

кодирование символов входной и выходной программы не рассматривается.

Оператор цикла, который требуется перевести, имеет вид:

```
for i = c1 to c2 step c3 <тело Цикла> next i
```

Допустим, что выходной язык не содержит оператора цикла. Тогда результат перевода (рис. 5.9) можно представить следующей последовательностью операторов:

```
i := c1; m1: if i > c2 then goto m2; <тело цикла>; i := i + c3; goto m1; m2:
```

Анализ выполняемого перевода позволяет сделать вывод о том, что выходная цепочка включает в себя объекты (сущности) трех видов:

- Объекты, передаваемые в выходную цепочку из входной цепочки (переменная цикла *i*, начальное значение *c1*, конечное значение *c2*, шаг *c3*).
- Объекты, не изменяющиеся в процессе перевода (множество терминальных символов выходного языка { ;, :, :=, +, >, if, then, goto }).
- Объекты, генерируемые во время перевода (метки *m1* и *m2*).

Составим БНФ, которая описывает синтаксис заданного оператора цикла, выбирая такую структуру описания, которая позволяет проиллюстрировать практически все проблемы синтеза АТ-грамматик:

(1) <оператор цикла> ::= <заголовок цикла> <тело цикла> <конец цикла>

(2) <заголовок цикла> ::= for <начальное значение> <конечное значение> <шаг>

(3) <начальное значение> ::= <идентификатор> = <константа>

(4) <конечное значение> ::= to <константа>

- (5) <шаг> ::= step <константа>
- (6) <тело цикла> ::= <оператор цикла>
- (7) <тело цикла> ::= <терминал>
- (8) <конец Цикла> ::= next <идентификатор >

Введём обозначения: s - <оператор цикла>, A - <заголовок цикла>, B - <тело цикла>, C - <конец цикла >, D - <начальное значение>, E - <конечное значение>, F - <шаг>. Тогда правила вывода КС-грамматики, описывающей синтаксис входного языка, имеют вид:

- 1) $S \rightarrow A B C$ 5) $F \rightarrow \text{step cns}$
- 2) $A \rightarrow \text{for } D E F$ 6) $B \rightarrow s$
- 3) $D \rightarrow \text{id} = \text{cns}$ 7) $B \rightarrow \text{term}$
- 4) $E \rightarrow \text{to cns}$ 8) $C \rightarrow \text{next id}$

При построении транслирующей грамматики требуется сделать некоторые предположения о ходе выполнения перевода и порядке анализа входной цепочки и построения выходной цепочки.

Для определённости будем считать, что:

- используется восходящий метод синтаксического анализа, выполняемый процессором с магазинной памятью;
- выполняется цепочечный перевод, в котором операционный символ вида {OUT := "str"} означает запись в выходную ленту OUT цепочки символов "str".

Определим действия, выполняемые первым по порядку элементом перевода (операционным символом), и правило, в которое он должен быть помещён. Для этого ещё раз рассмотрим неформальное описание перевода (см. рис. 5.9). Очевидно, что начать запись в выходную ленту процессор сможет только тогда, когда ему будет доступен параметр цикла id. Когда процессор выполняет свёртку, используя правило грамматики с номером (3), то id входит в основу и Доступен процессору. Следовательно, в это правило мы можем включить операционный символ, который записывает на выходную ленту начальный участок перевода, вплоть до символа cns. Правило транслирующей грамматики с номером (3) будет выглядеть следующим образом:

$$(3) D \rightarrow \text{id} = \text{cns} \{ \text{OUT} := \text{"id := cns; m1 : if id >"} \}.$$

При выполнении свёртки по этому правилу операционный символ записывает на выходную ленту параметр цикла id. Запись этой переменной в OUT может быть выполнена функцией "читать третий сверху символ магазина процессора и записать его на выходную ленту". Аналогичные действия ("Читать верхний символ магазина и записать его на выходную ленту") выполняются для начального значения параметра цикла cns. Остальные символы не зависят от входной цепочки (терминалы выходного языка) и просто помещаются на выходную ленту.

Рассуждая подобным образом, мы получим следующие правила транслирующей грамматики, содержащие элементы перевода (в порядке их использования при анализе входной цепочки и построении выходной Цепочки):

- (4) $E \rightarrow \text{to cns} \{ \text{OUT} := \text{"cns then goto m2;"} \}$
- (7) $B \rightarrow \text{term} \{ \text{OUT} := \text{"term;"} \}$

(5) $F \rightarrow \text{step cns}\{ \text{OUT} := \text{"id := id + cns; goto m1; m2 : " } \}$

Добавив остальные правила грамматики, получим следующую транслирующую грамматику:

- 1) $S \rightarrow A B C$
- 2) $A \rightarrow \text{for D E F}$
- 3) $D \rightarrow \text{id = cns}\{ \text{OUT} := \text{"id := cns; m1 : if id >"} \}$
- 4) $E \rightarrow \text{to cns}\{ \text{OUT} := \text{"cns then goto m2;"} \}$
- 5) $F \rightarrow \text{step cns}\{ \text{OUT} := \text{"id := id + cns; goto m1; m2: " } \}$
- 6) $B \rightarrow S$
- 7) $B \rightarrow \text{term}\{ \text{OUT} := \text{"term;"} \}$
- 8) $C \rightarrow \text{next id}$

При восходящих методах синтаксического анализа (см. гл. 8, 9) строится обращённый правый вывод (разбор) цепочки - последовательность номеров правил, используемых при свёртках, - который для рассматриваемого примера равен: (3), (4), (5), (2), (7), (8), (1)). При этом на выходной ленте формируется цепочка:

`id := cns1; m1: if id > cns2 then goto m2; id := id + cns3; goto m1; m2: term;`

Анализ полученной цепочки позволяет сделать следующие выводы:

1. Тело цикла (терминальный оператор `term`) включается неправильно (не перед увеличением параметра цикла на значение шага, а в конец выходной цепочки). Это связано с тем, что правило (5) применяется раньше, чем правило (7).

2. При использовании правила (5) значение параметра цикла `id` недоступно (`id` вытолкнут из магазина при свёртке по третьему правилу грамматики). Значение `id` должно быть передано в элемент перевода правила (5) для правильной генерации выходной цепочки.

3. При переводе вложенных циклов возникнет конфликт меток `m1` и `m2`, т. к. `m1` и `m2` - фиксированные значения. Значения меток должны генерироваться при выполнении перевода и передаваться от места возникновения метки к месту её использования.

Ошибки, обнаруженные в выходной цепочке, можно легко устранить, расширив транслирующую грамматику до атрибутивной транслирующей грамматики. Первую ошибку можно исправить несколькими способами:

- включить в правило (4) атрибут, в который записать координаты выходной цепочки, куда должно быть вставлено тело цикла, передать значение этого атрибута в правило (5) и использовать его для выполнения включения тела цикла в выходную Цепочку;
- связать с не терминалом `F` атрибут, значением которого является строка символов. При использовании правила (5) записать в эту строку операторы, реализующие изменение параметра цикла, затем передать значение этого атрибута в правило (1), где и переписать его значение в выходную цепочку;

- пополнить атрибутный процессор, выполняющий перевод, Дополнительной лентой, на которую записывать операторы, реализующие изменение переменной Цикла в правиле (5). В конце перевода при свёртке по правилу (1) сцепить вспомогательную ленту с выходной лентой.

Применение первых двух способов не вызывает принципиальных затруднений, но неэффективно при реализации процессора, поэтому используем последний способ устранения ошибки: на выходную ленту OUT1 будем записывать операторы установки начального значения параметра цикла, проверки завершения цикла и тело цикла, а на дополнительную ленту OUT2 - операторы изменения переменной цикла с заданным шагом.

Вторая и третья ошибки устраняются стандартным способом: включением в правила грамматики атрибутов и функций их вычисления.

Преобразуем транслирующую грамматику в транслирующую атрибутную грамматику.

Рассмотрим правило транслирующей грамматики:

$$D \rightarrow id = cns \{ OUT1 := "id := cns; m1 : if id >" \}$$

При выполнении свёртки по этому правилу значения терминальных символов *id* и *cns* требуется передать из входной цепочки в выходную. Это реализуется передачей соответствующих атрибутов в операционный символ и использованием их при построении выходной цепочки:

$$D \rightarrow id_{a1} = cns_{a2} \{ OUT1 := "id_{n1} := cns_{n2} ; m1 : if id >" \}_{n1, n2} \\ s1, n1 \leftarrow a2; n2 \leftarrow a2$$

Запись "*id l*" в операционном символе означает, что в данное место выходной цепочки нужно поместить идентификатор, значение которого определяется унаследованным атрибутом *n1* (значения *id* и *cns* - это не числовые значения этих объектов, а ссылки на таблицы, которые содержат необходимую информацию).

Так как переменная цикла *id* используется и в других правилах грамматики (правила 5) и 8)), то ее значение нужно передать в эти правила, связав с символом *D* атрибут и определив функцию для его вычисления. Окончательно правило грамматики примет вид:

$$3) D_{s1} \rightarrow id_{a1} = cns_{a2} \{ OUT1 := "id_{a1} := cns_{n2} ; m1 : if id >" \}_{n1, n2} \\ s1, n1 \leftarrow a1; \\ n2 \leftarrow a2$$

Для определения способа передачи значения переменной цикла *id* (атрибут *a1*) в правила 5) и 8) грамматики, рассмотрим фрагмент дерева грамматики, изображённый на рис. 5.5.

Процесс передачи значения лексемы *id* (атрибут *a1*) из правила (3) грамматики отмечен на рис. 5.5 сплошными линиями:

1. Из правой части правила 3 значение атрибута входного символа *a1* присваивается синтезированному атрибуту *s1* символа *D* из левой части этого же правила.
2. Из правой части правила 2 значение синтезированного атрибута *s1* нетерминального символа *D* присваивается унаследованному атрибуту *n1* не терминала *F* из правой части этого же правила.

3. Из левой части правила 5 значение унаследованного атрибута $n1$ не терминала F передаётся в правую часть унаследованному атрибуту $n2$ операционного символа.

Соответствующие описанному процессу правила АТ-Грамматики имеют вид:

- 3) $D_{s1} = id_{a1} \{ OUT1 := "id_{n1} := cns_{n2} ; m1: if id >" \}_{n1, n2}$
 $s1, n1 \leftarrow a1; n2 \leftarrow a2$
 2) $A \rightarrow \text{for } D_{s1} \text{ E } F_{n1}$
 $n1 \leftarrow s1$
 5) $F_{n1} \rightarrow \text{step cns} \{ OUT2 := "id := id + cns; goto m1; m2 : " \}_{n2}$
 $n2 \leftarrow n1$

Передача значения параметра цикла в правило (8) грамматики изображено на рис. 5.5 штриховыми линиями и описывается следующими правилами АТ-грамматики:

2. $A_{s2} \rightarrow \text{for } D_{s1} \text{ E } F_{n1}$
 $s2, n1 \leftarrow s1$
 (1)
 $S \rightarrow A_{s2} \text{ B } C_{n1}$
 $n1 \leftarrow s2$
 8) $C_{n1} \rightarrow \text{next id} \{ \text{if } n2 \square n3 \text{ then Error} \}$
 $n2 \leftarrow n1$

Особенностью правила (8) построенной АТ-Грамматики является то, что его операционный символ должен интерпретироваться не как элемент цепочечного перевода, а как выполнение действий, напрямую не связанных с генерацией выходной цепочки. В этот операционный символ передаются значения параметра цикла из заголовка цикла и оператора его завершения. Если эти значения не равны, то должно возникать состояние ошибки процессора (Error).

Процесс генерации и передачи меток иллюстрируется на рис. 5.12. Метки $m1$ и $m2$ генерируются в операционных символах правил (3) и (4) грамматики соответственно с помощью процедуры-функции NewLabel, возвращающей при обращении к ней уникальное значение метки. Правила АТ-грамматики, описывающие процесс генерации меток, имеют вид:

- 3) $D_{s1}, s2 \rightarrow id_{a1} = cns_{a2}$
 $\{ OUT1 := "id_{n1} := cns_{n2}; Label_{s3}: if id >" \}_{n1, n2, n3}$
 $s1, n1 \leftarrow 01;$
 $n2 \leftarrow a2; s3 \leftarrow \text{NewLabel}; n3 \leftarrow s3; 2 \leftarrow n3;$
 4) $E_{s3} \rightarrow \text{to cns} \{ OUT1 := "cns_{then} goto Label_{s1};" \}_{n2}$
 $s1 \leftarrow \text{NewLabel} ; n2 \leftarrow s1; s3 \leftarrow n2;$
 2) $A \rightarrow \text{for } D_{s1, s2} E_{s3} F_{n1, n2, n3}$
 $n1 \leftarrow s1; n3 \leftarrow s3$
 5) $F_{n1, n2, n3} \rightarrow \text{step cns} \{ OUT2 := "id := id + cns; goto Label_{n4};$
 $Label_{n5}:" \}_{n4, n5}$
 $n4 \leftarrow n2; n5 \leftarrow n3$

Сцепление выходных лент OUT1 и OUT2 должно быть выполнено один раз сразу после анализа и перевода всей входной цепочки. для реализации сцепления

выходных лент грамматику необходимо пополнить новым (нулевым) правилом, включив в него операционный символ, выполнявший действия по конкатенации выходных лент. Результирующая АТ-грамматика будет иметь вид:

- 0) $S_0 \rightarrow S \{ OUT1 := OUT1 \parallel OUT2 \}$
- 1) $S \rightarrow A_{s1} B C_{n1}$
 $n1 \leftarrow S1$
- 2) $A_{s4} \rightarrow \text{for } D_{s1,s2} E_{s3} F_{n1, n2, n3}$
 $s4, n1 \leftarrow s1; n2 \leftarrow s2; n3 \leftarrow s3;$
- 3) $D_{s1,s2} \rightarrow id_{a1} = cns_{a2}$
 $\{ OUT1 := " id_{n1} := cns_{n2}; Label_{s3}: \text{if id} > " \}_{n1, n2, n3}$
 $s1, n1 \leftarrow 01; n2 \leftarrow 02; s3 \leftarrow \text{NewLabel}; n3 \leftarrow s3; s2 \leftarrow n3;$
- 4) $E_{s3} \rightarrow \text{to } cns_{a2}; \{ OUT1 := "cns_{n1} \text{ then goto } Label_{s1}; " \}_{n1, n2}$
 $n1 \leftarrow 01; s1 \leftarrow \text{NewLabel};$
 $n2 \leftarrow s1; s3 \leftarrow n2;$
- 5) $F_{n1, n2, n3} \rightarrow \text{step } cns_{a1}$
 $\{ OUT2 := "id_{n6} := id_{n6} + cns_{n7}; \text{goto } Label_{n5}; Label_{n4}; " \}_{n4, n5, n6}$
 $n4 \leftarrow n2; n5 \leftarrow n3; n6 \leftarrow n1; n7 \leftarrow a1;$
- 6) $B \rightarrow S$
- 7) $B \rightarrow \text{term}; \{ OUT1 := "term"; \}$
- 8) $C_{n1} \rightarrow \text{next } id_{a1} \{ \text{if } n2 \neq n3 \text{ then Error} \}_{n4, n3}$
 $n2 \leftarrow n1; n3 \leftarrow a1$

При переводе вложенных операторов цикла использования двух выходных лент в процессоре и их сцепление при выполнении перевода может привести к ошибкам в структуре выходной цепочки. рассмотрим перевод следующей входной цепочки:

```

for i1 = c5 to c12 step c13
  for i2 = c21 to c22 step c23
    top
  next i2
next i1

```

разбор которой в виде последовательности номеров правил и синтаксического дерева приведён на рис. 5.13.

К моменту использования правила (O) на выходных лентах OUT1 и OUT2 будут находиться следующие цепочки (на данном этапе проверки считаем, что вычисление атрибутов выполняется правильно):

```

OUT1:
i1 := c5;
Label5: if i1 > c12 then goto Label12;
i2 := c21;
Label21: if i1 > c22 then goto Label22; top;
OUT2:
i1 := i1 + c13; goto Label5; Label12:
i2 := i2 + c23; goto Label21; Label22:

```

После выполнения сцепления выходная (основная) лента будет содержать перевод:

```

OUT1:
i1 := c5;
Label5: if i1 > c12 then goto Label12;
i2 := c21;
Label21: if i1 > c22 then goto Label22; top;
OUT2:
i1 := i1 + c13; goto Label5; Label12:
i2 := i2 + c23; goto Label21; Label22:
i2 := c21;
Label21: if i1 > c22 then goto Label22;
top;
i1 := i1 + c13; goto Label5;
Label12:
i2 := i2 + c23;
goto Label21;
Label22:

```

Анализ выходной цепочки показывает, что перевод построен неверно: операторы завершения, внешнего и вложенного циклов следуют в обратном порядке. Связано это с тем, что вначале на выходную ленту OUT2 записываются операторы завершения внешнего цикла, и только после этого вложенного. При формировании же перевода на выходную ленту вначале должны быть записаны операторы завершения вложенного цикла, а затем - внешнего. Для того чтобы исправить эту ошибку, необходимо записать на ленту OUT2 и чтение из неё выполнять с одного конца, т. е. реализовать её как стек с операциями PushOUT2 ("Втолкнуть на ленту OUT2") и PopOUT2 ("Вытолкнуть с ленты OUT2").

После включения операций PushOUT2 и PopOUT2 в правила (O) и (5) окончательно получим следующий вид атрибутивной транслирующей грамматики:

- 0) $S_0 \rightarrow S \{ \text{while (OUT2 not empty) OUT1 := OUT1 } \& \text{ PopOUT2 } \}$
- 1) $S \rightarrow A_{s1} \ B \ C_{n1}$
 $n1 \leftarrow s1$
- (2)
- $A_{s4} \rightarrow \text{for } D_{s1,s2} \ E_{s3} \ F_{n1, n2, n3}$
 $s4, n1 \leftarrow s1; n2 \leftarrow s2; n3 \leftarrow s3;$
- 3) $D_{s1,s2} \rightarrow id_{n1} = cns_{a1}$
 $\{ \text{OUT1 := "id}_{n1} := cns_{n1}; Label_{s3}: \text{if id >"} \}_{n1, n2, n3}$
 $s1, n1 \leftarrow a1; n2 \leftarrow a2; s3 \leftarrow \text{NewLabel}; n3 \leftarrow s3; s2 \leftarrow n3;$
- 4) $E_{s3} \rightarrow \text{to } cns_{a1} \{ \text{OUT1 := "cns}_{n1} \text{ then goto Label}_{s3};" \}_{n1, n2}$
 $n1 \leftarrow a1; s1 \leftarrow \text{NewLabel}; n2 \leftarrow s1; s3 \leftarrow n2;$
- 5) $F_{n1, n2, n3} \rightarrow \text{step } cns_{a1}$
 $\{ \text{PushOUT2("id := "id}_{n6} := id_{n6} + cns_{n7}; goto Label_{n5}; Label_{n4}:" :")}_{n4, n5, n6}$
- 6) $B \rightarrow 8$
- 7) $B \rightarrow \text{term}; \{ \text{OUT1 := "term; } \}$

8) $C_{n1} \rightarrow \text{next } id_{a1} \{ \text{if } n2 \neq n3 \text{ then Error} \}_{n2, n3}$

$n2 \leftarrow n1; n3 \leftarrow a1$

Рекомендуется самостоятельно проверить правильность окончательного варианта построения АТ-Грамматики для различных тестовых входных цепочек.

6.7. Трансляция класса C++ в класс Python на основе АТ-грамматики

Выполним перевод класса, имеющего следующий формат:

```
class <name> {
public:
    <variables>
    <constructor>
    <destructor>
```

private:

protected:

};

Введём следующие ограничения:

<name> - имя класса

<variables> - объявление переменной

<constructor> - конструктор

<destructor> - деструктор

Таблица Трансляции

Пример входного класса C++	Результат трансляции в Python
<pre>class MyClass { public: int x = 0; MyClass() { x = 1; }; ~MyClass() { x = 0; }; private: protected: };</pre>	<pre>class MyClass: x = 0 def __init__(self): self.x = 1 def __del__(self): x = 0</pre>

Построим СУ-схему:

№	Правила грамматики C++	Элемент перевода Python
1	<header> → class <name> {	<header> → class <name>:
2	<name> → N	<name> → N
3	<body> → public: <variables> <constructor> <destructor> private: protected:	<body> → <variables> <constructor> <destructor>

4	<code><variables> → int x = <val>;</code>	<code><variables> → x = val</code>
5	<code><val> → X</code>	<code><val> → X</code>
6	<code><constructor> → <name>() { x = <val> }</code>	<code><constructor> → def __init__(self): self.x = <val></code>
7	<code><destructor> → ~<name>() { x = <val> }</code>	<code><destructor> → def __del__(self): self.x = <val></code>
8	<code><end> → };</code>	<code><end> → “\n”</code>

Анализ выполняемого перевода позволяет сделать вывод о том, что выходная цепочка включает в себя объекты (сущности) трёх видов:

- Объекты, передаваемые в выходную цепочку из входной цепочки (имя класса, название переменной).

- Объекты, не изменяющиеся в процессе перевода (множество терминальных символов выходного языка {;, ., (,), =, class}.

- Объекты, генерируемые во время перевода (def, __init__, __del__, self, :,).

Составим БНФ, которая описывает синтаксис заданного класса, выбирая такую структуру описания, которая позволяет проиллюстрировать практически все проблемы синтеза АТ-грамматик:

1. `<class> ::= <header> <body> <end>`
2. `<header> ::= class <name> {`
3. `<body> ::= public: <variables> <constructor> <destructor> private: protected:`
4. `<variables> ::= int x = <val>`
5. `<val> ::= <const>`
6. `<constructor> ::= <name>() { x = <val> }`
7. `<destructor> ::= ~<name>() { x = <val> }`
8. `<end> ::= };`
9. `<name> ::= <const>`

Введём обозначения: S - `<class>`, H - `<header>`, B - `<body>`, E - `<end>`, N - `<name>`, V - `<variables>`, C - `<constructor>`, D - `<destructor>`, X - `<val>`.

Тогда правила вывода КС-грамматики, описывающей синтаксис входного языка, имеют вид:

1. $S \rightarrow H B E$
2. $H \rightarrow \text{class } N:$
3. $B \rightarrow V C D$
4. $V \rightarrow x = X;$
5. $X \rightarrow \text{cns}$
6. $C \rightarrow \text{def } __\text{init}__(\text{self}): \text{self.X} = 1$
7. $D \rightarrow \text{def } __\text{del}__(\text{self}): \text{self.X} = 0$
8. $E \rightarrow \text{“}\backslash\text{n”}$
9. $N \rightarrow \text{cns}$

При построении транслирующей грамматики требуется сделать некоторые предположения о ходе выполнения перевода и порядке анализа входной цепочки и построения выходной цепочки.

Для определённости будем считать, что:

- используется восходящий метод синтаксического анализа, выполняемый процессором с магазинной памятью;
- выполняется цепочечный перевод, в котором операционный символ вида $\{OUT := "str"\}$ означает запись в выходную ленту OUT цепочки символов "str".

Определим действия, выполняемые первым по порядку элементом перевода (операционным символом), и правило, в которое он должен быть помещён. Для этого ещё раз рассмотрим неформальное описание перевода. Очевидно, что начать запись в выходную ленту процессор сможет только тогда, когда ему будет доступен параметр класса name. Когда процессор выполняет свёртку, используя правило грамматики с номером (2), то name входит в основу и доступен процессору. Следовательно, в это правило мы можем включить 176 операционный символ, который записывает на выходную ленту начальный участок перевода, вплоть до символа cns. Правило транслирующей грамматики с номером (2) будет выглядеть следующим образом:

2. $H \rightarrow \text{class } N: = \text{cns} \{ OUT := \text{"class cns:"} \}$

При выполнении свёртки по этому правилу операционный символ записывает на выходную ленту параметр класса name. Запись этой переменной в OUT может быть выполнена функцией "читать второй сверху символ магазина процессора и записать его на выходную ленту". Также на вход подаются значения переменной:

4. $V \rightarrow x = X; = \text{cns} \{ \text{"x = cns"} \}$

6. $C \rightarrow \text{def } __\text{init}__(self): self.X = \text{cns} = 1 \{ \text{"def } __\text{init}__(self): self.cns = 1"} \}$

7. $D \rightarrow \text{def } __\text{del}__(self): self.X = \text{cns} = 1 \{ \text{"def } __\text{del}__(self): self.cns = 0"} \}$

Остальные символы не зависят от входной цепочки (терминалы выходного языка) и просто помещаются на выходную ленту.

Добавив остальные правила грамматики, получим следующую транслирующую грамматику:

1. $S \rightarrow H B E$

2. $H \rightarrow \text{class } \text{cns} \{ \text{"class cns:"} \}$

3. $B \rightarrow V C D$

4. $V \rightarrow x = \text{cns}; \{ \text{"x = cns"} \}$

5. $X \rightarrow \text{cns}$

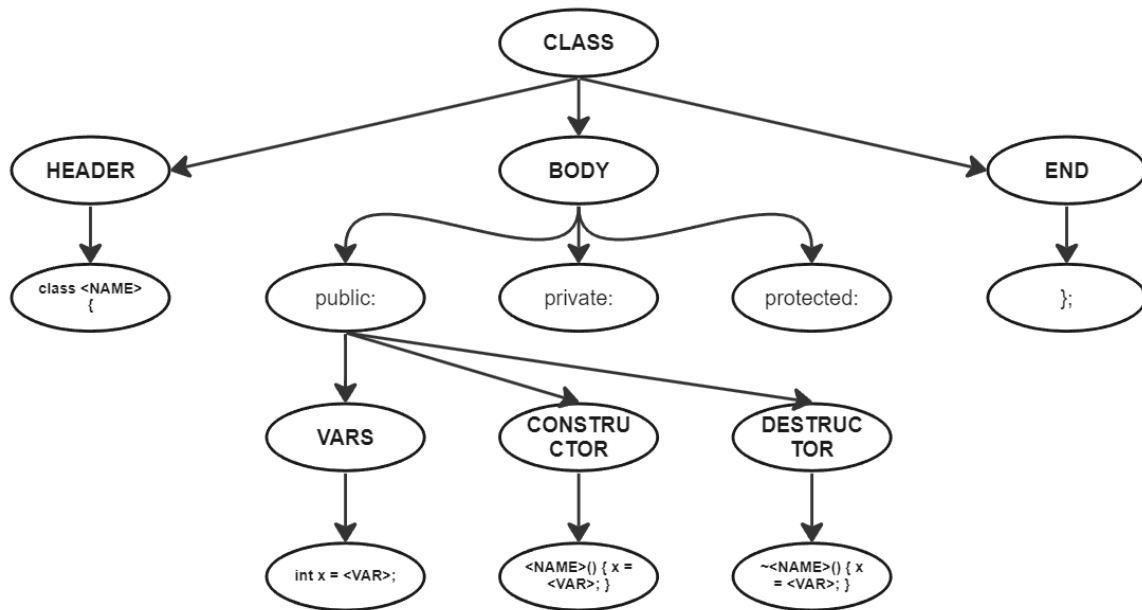
6. $C \rightarrow N() \{ x = \text{cns} \} \{ \text{"def } __\text{init}__(self): self.x = \text{cns"} \}$

7. $D \rightarrow \sim N() \{ x = \text{cns} \} \{ \text{"def } __\text{del}__(self): self.x = \text{cns"} \}$

8. $E \rightarrow \text{"\n"}$

9. $N \rightarrow \text{cns}$

Построим дерево синтаксического разбора входных цепочек:



Как видно, в конструкторе и деструкторе необходимо знать значение NAME. Мы можем передать его с помощью атрибутов.

Рассмотрим правило:

2. $H \rightarrow \text{class } N: = \text{cns} \{ \text{"class cns:"} \}$

С помощью атрибутов преобразуем его в:

2. $H \rightarrow \text{class } \text{cns}_{a1} \{ \text{"class cns}_{n1}:" \}_{n1}$

$n1 \leftarrow a1$

То есть в выходной строке вместо имени будем записывать значение, переданное во входной строке. Также это значение нужно «пробросить» в конструктор и деструктор:

6. $C \rightarrow N_{a1}() \{ x = \text{cns} \} \{ \text{"def __init__(self): self.x = cns}_{n2} \}_{n2}$

$n2 \leftarrow a1$

7. $D \rightarrow \sim N_{a1}() \{ x = \text{cns} \} \{ \text{"def __del__(self): self.x = cns}_{n3} \}_{n3}$

$n3 \leftarrow a1$

После преобразований правила примут вид:

1. $S \rightarrow H B E$

2. $H \rightarrow \text{class } \text{cns}_{a1} \{ \text{"class cns}_{n1}:" \}_{n1}$

$n1 \leftarrow a1$

3. $B \rightarrow V C D$

4. $V \rightarrow x = \text{cns}; \{ \text{"x = cns"} \}$

5. $X \rightarrow \text{cns}$

6. $C \rightarrow N_{a1}() \{ x = \text{cns} \} \{ \text{"def __init__(self): self.x = cns}_{n2} \}_{n2}$

$n2 \leftarrow a1$

7. $D \rightarrow \sim N_{a1}() \{ x = \text{cns} \} \{ \text{"def __del__(self): self.x = cns}_{n3} \}_{n3}$

$n3 \leftarrow a1$

8. $E \rightarrow \text{"}\backslash\text{n"}$

9. $N \rightarrow \text{cns}$

Значение переменных брать с более верхнего уровня дерева нет необходимости, так что добавление атрибутов не требуется.

6.8. Трансляция switch-case C++ в класс Python на основе АТ-грамматики

Для упрощения задачи поставим несколько ограничений:

1) <параметр> является терминалом или нетерминалом и имеет тип int

2) <значение i> является целым положительным числом

кодирование символов входной и выходной программы не рассматривается.

Оператор switch-case, который требуется перевести	результат перевода
<pre>switch (param) { case const1: <блок 1> break; case const2: < блок 2> break; ... case constn: < блок n> break; default: <блок default> }</pre>	<pre>if param == const1: <блок 1> elif param == const2: < блок 2> ... elif param == constn: < блок n> else: <блок default></pre>

Switch-case (C++) разбивается вертикальными чертами (черта ставится в том случае, если проходя по строке, мы находим место, где может быть изменяемая величина/переменная) и все, что слева от черты транслируется в аналогичную(по смыслу) часть для Python, ставится слева от вертикальной черты в схеме для Python, далее берется еще одна часть ограниченная чертами и по аналогии транслируется. Выполняется пока не закончится входная строка.

Анализ выполняемого перевода позволяет сделать вывод о том, что выходная цепочка включает в себя объекты (сущности) двух видов:

- Объекты, передаваемые в выходную цепочку из входной цепочки (параметр ветвления param, значения условий c1, c2, ... , cn и блоки исполняемые при соответствии условию).

- Объекты, не изменяющиеся в процессе перевода (множество терминальных символов выходного языка { :, =, i, f, e, l, s, {, } }).

Составим БНФ, которая описывает синтаксис заданного оператора switch-case:

(1) <оператор ветвления> ::= <заголовок ветвления> <тело случаев> <конец ветвления>

(2) <заголовок ветвления> ::= switch (<параметр>) {

(3) <тело случаев> ::= case <значение>: <тело>; break; <тело случаев>

(4) <тело случаев> ::= default: <тело>;

(5) <конец ветвления> ::= }

Введём обозначения: S - <оператор ветвления>, A - <заголовок ветвления>, B - <тело случаев>, C - <конец ветвления>. Тогда правила вывода КС-грамматики, описывающей синтаксис входного языка, имеют вид:

1. $S \rightarrow ABC$

2. $A \rightarrow \text{switch (param) \{}$

3. $B \rightarrow \text{case const: body; break; B}$

4. $B \rightarrow \text{default: body;}$

5. $C \rightarrow \}$

Пример вывода цепочки

$S \Rightarrow^1 ABC \Rightarrow^5 AB\} \Rightarrow^2 \text{switch (param) \{ B} \Rightarrow^3 \text{switch (param) \{ case const: body; break; B} \Rightarrow^4 \text{switch (param) \{ case const: body; break; default: body; break;}}$

При построении транслирующей грамматики требуется сделать некоторые предположения о ходе выполнения перевода и порядке анализа входной цепочки и построения выходной цепочки.

Для определённости будем считать, что:

- используется восходящий метод синтаксического анализа, выполняемый процессором с магазинной памятью;
- выполняется цепочечный перевод, в котором операционный символ вида {"str"} означает запись в выходную ленту цепочки символов "str".

Определим действия, выполняемые первым по порядку элементом перевода (операционным символом), и правило, в которое он должен быть помещён. Для этого ещё раз рассмотрим неформальное описание перевода. Очевидно, что начать запись в выходную ленту процессор сможет только тогда, когда ему будет доступен параметр и первая константа. Когда процессор выполняет свёртку, используя правило грамматики с номером (3), то param и const входит в основу и доступен процессору. Следовательно, в это правило, мы можем включить операционный символ, который записывает на выходную ленту начальный участок перевода, вплоть до символа sps. Правило транслирующей грамматики с номером (3) будет выглядеть следующим образом:

3. $B \rightarrow \text{case const: body; break; } B \{ \text{„if param == const:\n\tbody\nel“ } \}$

При выполнении свёртки по этому правилу операционный символ записывает на выходную ленту первый случай вместе с param и const.

Запись этой переменной на выходную ленту может быть выполнена функцией "читать третий сверху символ магазина процессора и записать его на выходную ленту". Аналогичные действия ("Читать верхний символ магазина и записать его на выходную ленту") выполняются для начального значения параметра цикла const. Остальные символы не зависят от входной цепочки (терминалы выходного языка) и просто помещаются на выходную ленту.

Рассуждая подобным образом, мы получим следующие правила транслирующей грамматики, содержащие элементы перевода (в порядке их использования при анализе входной цепочки и построении выходной цепочки):

3. $B \rightarrow \text{case const: body; break; } B \{ \text{„if param == const:\n\tbody\nel“ } \}$

4. $B \rightarrow \text{default: body; break; } \{ \text{„se:\n\tbody“ } \}$

Добавив остальные правила грамматики, получим следующую транслирующую грамматику:

1. $S \rightarrow ABC$

2. $A \rightarrow \text{switch (param) } \{$

3. $B \rightarrow \text{case const: body; break; } B \{ \text{„if param == const:\n\tbody\nel“ } \}$

4. $B \rightarrow \text{default: body; break; } \{ \text{„se:\n\tbody“ } \}$

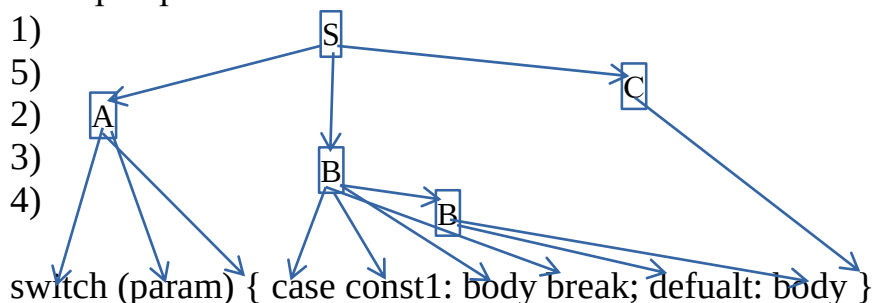
5. $C \rightarrow \}$

Пример вывода цепочки

$(S, "") \Rightarrow^1 (ABC, "") \Rightarrow^5 AB\} \Rightarrow^2 (\text{switch (param) } \{ B\}, "") \Rightarrow^3 (\text{switch (param) } \{ \text{case const: body; break; } B\}, \text{„if param == const:\n\t body \n el“}) \Rightarrow^4 (\text{switch (param) } \{ \text{case const: body; break; default: body; break;}\}, \text{„if param == const:\n\t body \n else:\n\t body “})$

Дерево вывода транслирующей грамматики:

Номера правил:



При восходящих методах синтаксического анализа строится обращённый правый вывод (разбор) цепочки - последовательность номеров правил, используемых при свёртках, - который для рассматриваемого примера равен: (4), 3), 2), 5), 1)). При этом на выходной ленте формируется цепочка:

```
if param == const1:
    body
else:
    body
```

Теперь мы можем расширить транслирующую грамматику до атрибутивной транслирующей грамматики.

Для этого включить в правило (2) атрибут, в который записать координаты выходной цепочки, куда должно быть вставлен параметр ветвления, передать значение этого атрибута в правило (3) и использовать его для выполнения включения сравнения с параметром константы в выходную Цепочку;

Так же нужно включить в правила грамматики атрибуты и функций их вычисления.

Преобразуем транслирующую грамматику в транслирующую атрибутивную грамматику.

Рассмотрим правило транслирующей грамматики:

2. $A \rightarrow \text{switch}(\text{param}) \{$

При выполнении свёртки по этому правилу значения терминального символа param требуется передать из входной цепочки в выходную. Это реализуется передачей соответствующих атрибутов в операционный символ и использованием их при построении выходной цепочки. Пометим эти атрибуты одинаковыми именами и опустим правила передачи значений:

$A_s \rightarrow \text{switch}(\text{param}_{n1}) \{$

Запись « param_s » в операционном символе означает, что в данное место выходной цепочки нужно поместить параметр, значение которого определяется унаследованным атрибутом $n1$ (значения param - это не числовые значения этих объектов, а ссылки на таблицы, которые содержат необходимую информацию). Так как переменная цикла param используется и в других правилах грамматики (правила 3)), то ее значение нужно передать в эти правила, связав с символом A атрибут и определив функцию для его вычисления. Окончательно правило грамматики примет вид:

2. $A_s \rightarrow \text{switch}(\text{param}_{n1}) \{$
 $s \leftarrow n1$

Для определения способа передачи значения параметра ветвления (атрибут n1) в правило 3) грамматики, рассмотрим фрагмент алгоритма трансляции грамматики.

Процесс передачи значения лексемы param (атрибут n1) из правила (2) нашей грамматики:

1. Из правой части правила 2 значение атрибута входного символа n1 присваивается синтезированному атрибуту s символа A из левой части этого же правила.
2. Из правой части правила 1 значение синтезированного атрибута s нетерминального символа A присваивается унаследованному атрибуту a не терминала B.
3. Из левой части правила 3 значение унаследованного атрибута a не терминала B передаётся в правую часть унаследованному атрибуту s операционного символа.

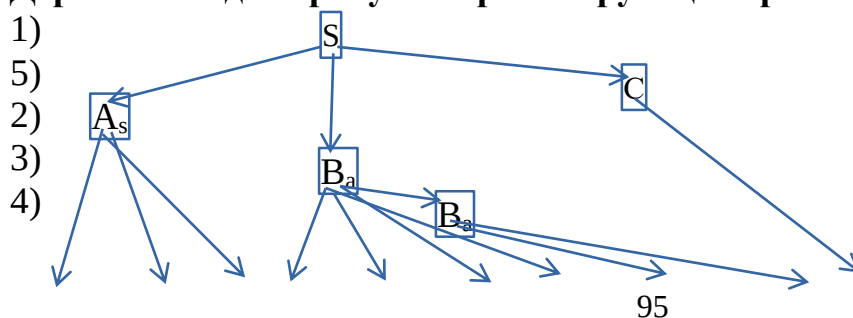
Соответствующие описанному процессу правила АТ-Грамматики имеют вид:

1. $S \rightarrow A_s B_a C$
 $a \leftarrow s$
2. $A_s \rightarrow \text{switch (param}_{n1}) \{$
 $s \leftarrow n1$
3. $B_a \rightarrow \text{case const}_{n1}: \text{body}_{n2}; \text{break}; B \{ \text{„if param}_s == \text{const}_{n1}: \backslash n \backslash t \text{body}_{n2} \backslash n e l \text{“} \}_{s, n1, n2}$
 $s \leftarrow a$
4. $B_a \rightarrow \text{default: body}_{n1}; \text{break}; \{ \text{„se:} \backslash n \backslash t \text{body}_{n1} \text{“} \}_{n1}$
 $C \rightarrow \}$

Пример перевода

Входная цепочка символов C++	После трансляции в Python
<pre>switch(variable) { case 1: body1 break; case 2: body2 break; default: body3 }</pre>	<pre>if variable == 1: body1 elif variable == 2: body2 else: body3</pre>

Дерево вывода атрибутно-транслирующей грамматики:



```
switch (paramn1) { case constn1: bodyn2 break; default: bodyn1 }
```

Алгоритм 1. построения АТ-грамматики из исходной Транслирующей грамматики

Вход: транслирующая грамматика $G(T, V, S_0, P)$

P:

1. $S \rightarrow ABC$
2. $A \rightarrow \text{switch (param) \{$
3. $B \rightarrow \text{case const: body; break; B \{ „if param == const:\n\tbody\nel“ \}$
4. $B \rightarrow \text{default: body; break; \{ „se:\n\tbody“ \}$
- $C \rightarrow \}$

Выход: АТ-грамматика АТG

VF - список нетерминалов

F - пустое множество

A - множество атрибутов

AF - цепочка выражения атрибутов

Sign $\square$ **T** – множество математических операций

L – множество лексем

```
f
o
r      y = a
e      foreach (Xy1,Xy2 ... Xyn) {
a
c
h
By  $\rightarrow$  Xy1,Xy2 ... Xyn  $\square$  P) { // Идём по правилам берем отдельный терминал или
// проверяем входит он или нет в множ-во правил
f
Xyj  $\square$  Sign) { // проверяем входит он или нет в множ-во математических
операций
G
G
y // присваиваем атрибуты предшествующему
A // присваиваем атрибуты следующему
N      }
AF  $\square$  y1,y2 // добавляем в множество атрибутов
F      F = F  $\square$  f0(y , AF)
i      AF = пустое множество
fi) // и после знака
Xyj  $\square$  V) { // если находим нетерминал
s      6.9. Трансляция do while C++ в класс Python на основе АТ-грамматики
yi) // если находим знак, то получаем символ идущий перед ним
```


Выполним разработку описания перевода оператора цикла, имеющего

Оператор цикла, который требуется перевести	Тогда результат перевод можно представить в виде
<pre> int var=const1; do{ <тело цикла> var+=const3; }while(var<const2); </pre>	<pre> var=const1 while(1): <тело цикла> var+=const3 if (var<const2): break </pre>

следующий формат:

```

<параметр>=<начальное значение>
do{
    <тело цикла>
    <параметр>+=<шаг>
}while(<параметр><<конечное значение>);

```

Для упрощения введём следующие ограничения:

- <параметр> - идентификатор целой переменной;
- <начальное значение>, <конечное значение>, <шаг> - целые положительные константы;
- <тело цикла> - оператор цикла или терминал (не подлежащий переводу оператор);

кодирование символов входной и выходной программы не рассматривается.

На схеме показано, как транслируется цикл do while (C++) в цикл while (Python).

Задание последовательности **Выдачи (операционный символ out)** на Цикл do while (C++) разбивается вертикальными чертами (черта ставится в том случае, если проходя по строке, мы находим место, где может быть изменяемая величина/переменная) и все, что слева от черты транслируется в аналогичную(по смыслу) часть для Python, ставится слева от вертикальной черты в схеме для Python, далее берется еще одна часть ограниченная чертами и по аналогии транслируется. Выполняется пока не закончится входная строка. Построим СУ-схему:

№	Правило грамматики c++	Элемент перевода Python
1	<header> → <variables>	<header> → <variables>

2	<variables>→int var=<val>;	<variables>→var=<val>
3	<val>→X	<val>→X
4	<body>→do {var+=<val>	<body>→while(1):
5	<end>→} while(<exception>;	<end>→if not <exception> break
6	<exception>→var < <val>	<exception>→var < <val>

Анализ выполняемого перевода позволяет сделать вывод о том, что выходная цепочка включает в себя объекты (сущности) двух видов:

- Объекты, передаваемые в выходную цепочку из входной цепочки (переменная цикла var, начальное значение const1, конечное значение const2, шаг const3).
- Объекты, не изменяющиеся в процессе перевода (множество терминальных символов выходного языка { ;, +=, :, =, +, <, do, while, {, } }).

Составим БНФ, которая описывает синтаксис заданного оператора цикла:

- (1) <оператор цикла> ::= <заголовок цикла> <тело цикла> <конец цикла>
- (2) <заголовок цикла> ::= <начальное значение> do
- (3) <начальное значение> ::= <идентификатор> =<константа>
- (3) <тело цикла> ::= <терминал>
- (4) <конец цикла> ::= <шаг> while <конечное значение>
- (6) <конечное значение> ::= <константа> <<константа>
- (7) <шаг> ::= ++<константа>

Сначала составим КС-грамматику.

Введём обозначения: S - <оператор цикла>, A - <заголовок цикла>, B - <тело цикла>, C - <конец цикла>, D - <начальное значение>, E - <конечное значение>, F - <шаг>. Тогда правила вывода КС-грамматики, описывающей синтаксис входного языка, имеют вид:

- 1) S → ABC
- 2) A → Ddo{
- 3) D → int var=const1;\n
- 4) B → <body>
- 5) C → F\n}while(E);
- 6) F → var+=const3;
- 7) E → var<const2

Пример:

S =>¹ ABC =>² Ddo{BC =>³ int var=const1;\ndo{BC =>⁴ int var=const1;\ndo{bodyC =>⁵ int var=const1;\ndo{bodyF\n}while(E); =>⁶ int var=const1;\ndo{<body>var+=const3;\n}while(E); =>⁷ int var=const1;\ndo{<body>var+=const3;\n}while(var<const2);

При построении транслирующей грамматики требуется сделать некоторые предположения о ходе выполнения перевода и порядке анализа входной цепочки и построения выходной цепочки.

Для определённости будем считать, что:

- используется восходящий метод синтаксического анализа, выполняемый процессором с магазинной памятью;
- выполняется цепочечный перевод, в котором операционный символ вида $\{ "str" \}$ означает запись в выходную ленту цепочки символов "str".

Определим действия, выполняемые первым по порядку элементом перевода (операционным символом), и правило, в которое он должен быть помещён. Для этого ещё раз рассмотрим неформальное описание перевода. Очевидно, что начать запись в выходную ленту процессор сможет только тогда, когда ему будет доступен параметр цикла var. Когда процессор выполняет свёртку, используя правило грамматики с номером (3), то var входит в основу и Доступен процессору. Следовательно, в это правило, мы можем включить операционный символ, который записывает на выходную ленту начальный участок перевода, вплоть до символа sns. Правило транслирующей грамматики с номером (3) будет выглядеть следующим образом:

(3) $D \rightarrow \text{int var}=\text{const1}; \{ "var=\text{const1}\backslash\text{nwhile}(1):\backslash\text{n}" \}.$

При выполнении свёртки по этому правилу операционный символ записывает на выходную ленту параметр цикла var.

Запись этой переменной в OUT может быть выполнена функцией "читать третий сверху символ магазина процессора и записать его на выходную ленту". Аналогичные действия ("Читать верхний символ магазина и записать его на выходную ленту") выполняются для начального значения параметра цикла const. Остальные символы не зависят от входной цепочки (терминалы выходного языка) и просто помещаются на выходную ленту.

Рассуждая подобным образом, мы получим следующие правила транслирующей грамматики, содержащие элементы перевода (в порядке их использования при анализе входной цепочки и построении выходной Цепочки):

(4) $B \rightarrow \langle \text{body} \rangle \{ "body" \}$

(6) $F \rightarrow \text{var}+=\text{const3}; \{ "\backslash\text{tvar}+=\text{const3}" \}$

(7) $E \rightarrow \text{var}<\text{const2} \{ "\backslash\text{tif not var } <\text{const2 break}" \}$

Добавив остальные правила грамматики, получим следующую транслирующую грамматику:

1) $S \rightarrow ABC$

2) $A \rightarrow D\text{do}\{$

3) $D \rightarrow \text{int var}=\text{const1}; \{ "var=\text{const1}\backslash\text{nwhile}(1):\backslash\text{n}" \}$

4) $B \rightarrow \langle \text{body} \rangle \{ "body" \}$

5) $C \rightarrow F\}\text{while}(E);$

6) $F \rightarrow \text{var}+=\text{const3}; \{ "\backslash\text{tvar}+=\text{const3}\backslash\text{n}" \}$

7) $E \rightarrow \text{var}<\text{const2} \{ "\backslash\text{tif not var } <\text{const2 break}" \}$

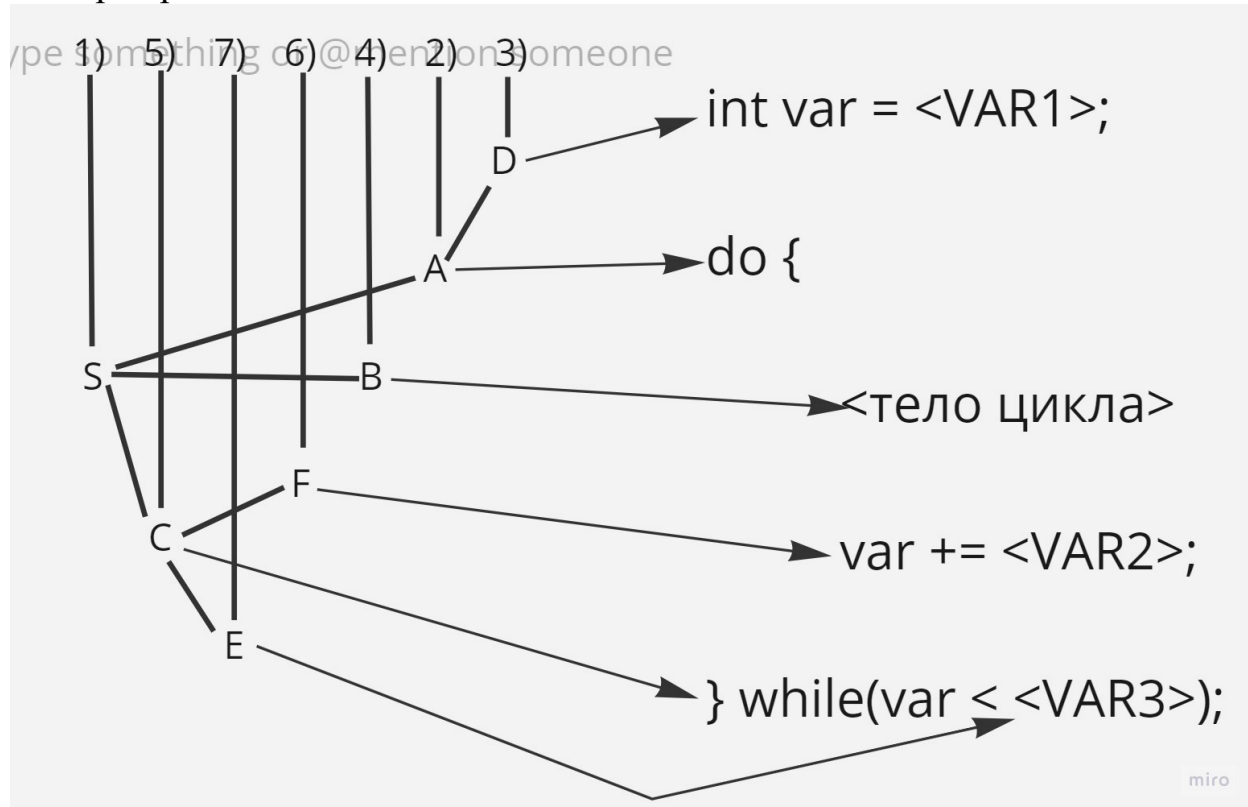
Пример:

$(S, "") \Rightarrow^1 (ABC, "") \Rightarrow^2 (D\text{do}\{BC, "" \Rightarrow^3 (\text{int var}=\text{const1}; \backslash\text{ndo}\{BC, "var=\text{const1}\backslash\text{nwhile}(1):") \Rightarrow^4 (\text{int var}=\text{const1}; \backslash\text{ndo}\{bodyC, "var=\text{const1}\backslash\text{nwhile}(1):\backslash\text{nbodydy}") \Rightarrow^5 (\text{int var}=\text{const1}; \backslash\text{ndo}\{bodyF\backslash\text{n}\}\text{while}(E);, "var=\text{const1}\backslash\text{nwhile}(1):\backslash\text{nbody}") \Rightarrow^6 (\text{int$

```
var=const1;\ndo{<body>var+=const3;\n}while(E);,"var=const1\nwhile(1):\nbody
\tvar+=const3\n") => (int
var=const1;\ndo{<body>var+=const3;\n}while(var<const2);,"
var=const1\nwhile(1):\nbody \tvar+=const3\n\tif not var <const2 break")
```

Дерево вывода транслирующей грамматики:

Номера правил:



При восходящих методах синтаксического анализа строится обращённый правый вывод (разбор) цепочки - последовательность номеров правил, используемых при свёртках, - который для рассматриваемого примера равен: (3), 2), 4), 6), 7), 5), 1)). При этом на выходной ленте формируется цепочка:

```
var=const1
while(1):
    <тело цикла>
    var += const3
    if not var<const2 break
```

Теперь мы можем расширить транслирующую грамматику до атрибутивной транслирующей грамматики. Для этого можно использовать один из вариантов:

- включить в правило (7) атрибут, в который записать координаты выходной цепочки, куда должно быть вставлено тело цикла, передать значение этого атрибута в правило (6) и использовать его для выполнения включения тела цикла в выходную Цепочку;
- связать с не терминалом А атрибут, значением которого является строка символов. При использовании правила (6) записать в эту строку операторы, реализующие изменение параметра цикла, затем передать

значение этого атрибута в правило (1), где и переписать его значение в выходную цепочку;

- пополнить атрибутный процессор, выполняющий перевод, Дополнительной лентой, на которую записывать операторы, реализующие изменение переменной Цикла в правиле (6). В конце перевода при свёртке по правилу (1) сцепить вспомогательную ленту с выходной лентой.

Применение первых двух способов не вызывает принципиальных затруднений, но неэффективно при реализации процессора, поэтому используем последний способ: на выходную ленту OUT1 будем записывать операторы установки начального значения параметра цикла, проверки завершения цикла и тело цикла, а на дополнительную ленту OUT2 - операторы изменения переменной цикла с заданным шагом.

Так же нужно включить в правила грамматики атрибуты и функций их вычисления.

Преобразуем транслирующую грамматику в транслирующую атрибутную грамматику.

Рассмотрим правило транслирующей грамматики:

3) $D \rightarrow \text{int var} = \text{const1}; \{ \text{"var} = \text{const1} \backslash \text{nwhile}(1) : " \}$

При выполнении свёртки по этому правилу значения терминальных символов var и const1 требуется передать из входной цепочки в выходную. Это реализуется передачей соответствующих атрибутов в операционный символ и использованием их при построении выходной цепочки. Пометим эти атрибуты одинаковыми именами и опустим правила передачи значений:

3) $D \rightarrow \text{int var} = \text{const1}_{a1}; \{ \text{"var} = \text{const1}_{n1} \backslash \text{nwhile}(1) " \}_{n1}$
 $n1 \leftarrow a1$

Запись "var=const1_{n1}" в операционном символе означает, что в данное место выходной цепочки нужно поместить идентификатор, значение которого определяется унаследованным атрибутом n1 (значения var и const1- это не числовые значения этих объектов, а ссылки на таблицы, которые содержат необходимую информацию).

Так как переменная цикла var используется и в других правилах грамматики (правила 6) и 7)), то ее значение нужно передать в эти правила, связав с символом D атрибут и определив функцию для его вычисления. Окончательно правило грамматики примет вид:

3) $D \rightarrow \text{int var} = \text{const1}_{a1}; \{ \text{"var} = \text{const1}_{n1} \backslash \text{nwhile}(1) " \}_{n1}$
 $n1 \leftarrow a1$

Для определения способа передачи значения переменной цикла id (атрибут a1) в правила 6) и 7) грамматики, рассмотрим фрагмент алгоритма трансляции грамматики.

Процесс передачи значения лексемы var (атрибут a1) из правила (3) нашей грамматики:

1. Из правой части правила 3 значение атрибута входного символа a_1 присваивается синтезированному атрибуту s_1 символа D из левой части этого же правила.

2. Из правой части правила 2 значение синтезированного атрибута s_1 нетерминального символа D присваивается унаследованному атрибуту n_1 не терминала F и унаследованному атрибуту a_2 не терминала E из правой части этого же правила.

3. Из левой части правила 6 значение унаследованного атрибута n_1 не терминала F передаётся в правую часть унаследованному атрибуту a операционного символа. По аналогии поступаем с правилом 7)

Соответствующие описанному процессу правила АТ-Грамматики имеют вид:

3) $D \rightarrow \text{int var} = \text{const1}_{a_1}; \{ \text{"var} = \text{const1}_{n_1} \backslash \text{nwhile}(1) " \}_{n_1}$
 $n_1 \leftarrow a_1$

6) $F \rightarrow \text{var} += \text{const3}_{a_2}; \{ \text{"\tvar} += \text{const3}_{n_2} " \}_{n_2}$
 $n_2 \leftarrow a_2$

7) $E \rightarrow \text{var} < \text{const2}_{a_3}; \{ \text{" \tif not var} < \text{const2}_{n_3} \text{ break} " \}_{n_3}$
 $n_3 \leftarrow a_3$

Сцепление выходных лент (вывод правил 3,6,7) правил должно быть выполнено один раз сразу после анализа и перевода всей входной цепочки. для реализации сцепления выходных лент грамматику необходимо пополнить новым (нулевым) правилом, включив в него операционный символ, выполняя действия по конкатенации выходных лент.

Результирующая АТ-грамматика будет иметь вид:

1) $S \rightarrow ABC$

2) $A \rightarrow D \text{ do } \{$

3) $D \rightarrow \text{int var} = \text{const1}_{a_1}; \{ \text{"var} = \text{const1}_{n_1} \backslash \text{nwhile}(1) " \}_{n_1}$
 $n_1 \leftarrow a_1$

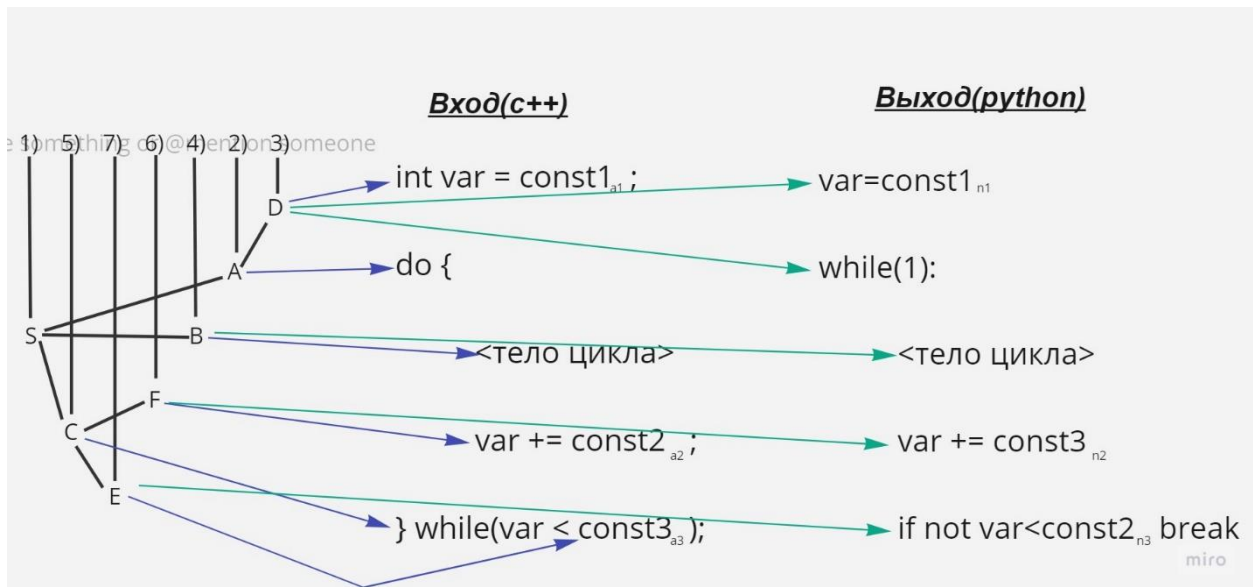
4) $B \rightarrow \langle \text{body} \rangle \{ \text{"body"} \}$

5) $C \rightarrow F \} \text{while}(E);$

6) $F \rightarrow \text{var} += \text{const3}_{a_2}; \{ \text{"\tvar} += \text{const3}_{n_2} " \}_{n_2}$
 $n_2 \leftarrow a_2$

7) $E \rightarrow \text{var} < \text{const2}_{a_3} \{ \text{" \tif not var} < \text{const2}_{n_3} \text{ break} " \}_{n_3}$
 $n_3 \leftarrow a_3$

Дерево вывода атрибутно-транслирующей грамматики:



Алгоритм 1. построения АТ-грамматики из исходной Транслирующей грамматики

Вход: транслирующая грамматика $G(T, V, S_0, P)$

P:

- 1) $S \rightarrow ABC$
- 2) $A \rightarrow D \text{ do } \{$
- 3) $D \rightarrow \text{int var} = \text{const1}; \{ \text{"var=const1\nwhile(1):"} \}$
- 4) $B \rightarrow \langle \text{body} \rangle \{ \text{"body"} \}$
- 5) $C \rightarrow F \} \text{while}(E$
- 6) $F \rightarrow \text{var} += \text{const3}; \{ \text{"\tvar+=const3"} \}$
- 7) $E \rightarrow \text{var} < \text{const2}); \{ \text{"\tif (var<const2) break"} \}$

Выход: АТ-грамматика ATG

VF = Список нетерминалов

F = пустое множество

A – множество атрибутов

AF = цепочка выражения атрибутов

Sign $\subset T$ – множество математических операций

L – множество лексем

```
foreach (By  $\rightarrow$  Xy1, Xy2 ... Xyn  $\in$  P) { // Идём по правилам берем отдельный терминал или нетерминал
    y = a
    foreach (Xy1, Xy2 ... Xyn) {
        if (Xyj  $\in$  T) { // проверяем входит он или нет в множ-во правил
            if (Xyj  $\in$  Sign) { // проверяем входит он или нет в множ-во математических операций
                y1 = GetPrevious(yi) // если находим знак, то получаем символ идущий перед ним
                y2 = GetNext(yi) // и после знака
                y1 += a // присваиваем атрибуты предшествующему
                y2 += a // присваиваем атрибуты следующему
                AF = AF  $\cup$  y1, y2 // добавляем в множество атрибутов
            }
        }
    }
}
```

```

    }
    } else if (Xyj ∈ V) { // если находим нетерминал
        yj = a
        AF = AF ∪ yj
    }
}
F = F ∪ f0(y, AF)
AF = пуст.мн-во
}

```

6.10. Трансляция while C++ в класс Python на основе АТ-грамматики

Выполним разработку описания перевода оператора цикла, имеющего следующий формат:

```

int <параметр>=<начальное значение>;

while (<параметр> < <конечное значение>) {
    <тело цикла>;
    <параметр>+=<шаг> ;
}

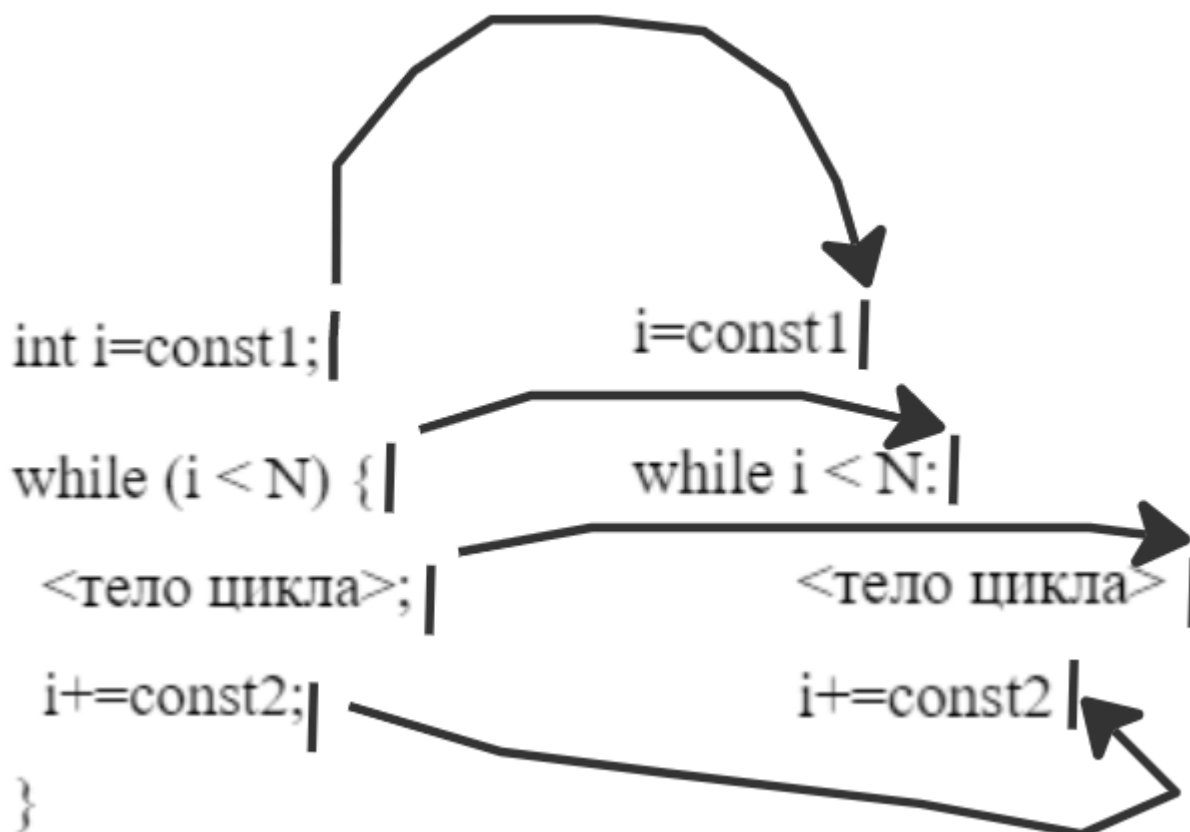
```

Для упрощения задачи поставим несколько ограничений:

- 1) <параметр> идентификатор целой переменной;
- 2) <начальное значение>, <конечное значение>, <шаг> - целые положительные константы;
- 3) <тело цикла> - оператор цикла или терминал (не подлежащий переводу оператор);

Оператор цикла, который требуется перевести	Результат перевода в Python
<pre> int i=const1; while (i < N) { <тело цикла>; i+=const2; } </pre>	<pre> i=const1 while i < N: <тело цикла> i+=const2 </pre>

На схеме показано, как транслируется цикл for (C++) в цикл while (Python).



Задание последовательности **Выдачи** (операционный символ out)

Цикл while (C++) разбивается вертикальными чертами (черта ставится в том случае, если проходя по строке, мы находим место, где может быть изменяемая величина/переменная) и все, что слева от черты транслируется в аналогичную(по смыслу) часть для Python, ставится слева от вертикальной черты в схеме для Python, далее берется еще одна часть ограниченная чертами и по аналогии транслируется. Выполняется пока не закончится входная строка.

Анализ выполняемого перевода позволяет сделать вывод о том, что выходная цепочка включает в себя объекты (сущности) двух видов:

- 1) Объекты, передаваемые в выходную цепочку из входной цепочки (переменная цикла i , конечное значение N).
- 2) Объекты, не изменяющиеся в процессе перевода (множество терминальных символов выходного языка { ;, :, +=, <, while, {, } }.

Составим БНФ, которая описывает синтаксис заданного оператора цикла:

(1)<оператор цикла> ::= <начальное значение><заголовок цикла><тело цикла><шаг><конец цикла>

(2) <начальное значение> ::= <идентификатор> =<константа>

(3) <заголовок цикла> ::= while<конечное значение>

(4) <конечное значение> ::= <константа><<константа>

(5) <тело цикла> ::= <терминал>

(6) <шаг> ::= <константа>+<константа>

(7) <конец цикла> ::= }

Сначала составим КС-грамматику.

Введем обозначения: S - <оператор цикла>, A -<начальное значение>, B - <заголовок цикла>, C - <тело цикла>, D - <шаг>, E -<конец цикла>, F - <конечное значение>. Тогда правила вывода КС-грамматики, описывающей синтаксис входного языка, имеют вид:

1) $S \rightarrow ABCDE$

2) $A \rightarrow \text{int } i = \text{const1};$

3) $B \rightarrow \text{while } F$

4) $F \rightarrow (i < N) \{$

5) $C \rightarrow \text{body};$

6) $D \rightarrow i += \text{const2};$

7) $E \rightarrow \}$

Пример вывода цепочки:

$S \Rightarrow^1 ABCDE \Rightarrow^2 \text{int } i = \text{const1}; BCDE \Rightarrow^3 \text{int } i = \text{const1}; \text{while } FCDE \Rightarrow^4 \text{int } i = \text{const1}; \text{while } (i < N) \{ CDE \Rightarrow^5 \text{int } i = \text{const1}; \text{while } (i < N) \{ \text{body}; DE \Rightarrow^6 \text{int } i = \text{const1}; \text{while } (i < N) \{ \text{body}; i += \text{const2}; E \Rightarrow^7 \text{int } i = \text{const1}; \text{while } (i < N) \{ \text{body}; i += \text{const2};$

При построении транслирующей грамматики требуется сделать некоторые предположения о ходе выполнения перевода и порядке анализа входной цепочки и построения выходной цепочки.

Для определённости будем считать, что:

- 1) Используется восходящий метод синтаксического анализа, выполняемый процессором с магазинной памятью;
- 2) Выполняется цепочечный перевод, в котором операционный символ вида {"str"} означает запись в выходную ленту OUT цепочки символов "str".

Определим действия, выполняемые первым по порядку элементом перевода (операционным символом), и правило, в которое он должен быть помещён. Для этого ещё раз рассмотрим неформальное описание перевода. Очевидно, что начать запись в выходную ленту процессор сможет только тогда, когда ему будет доступен параметр цикла i . Когда процессор выполняет свертку, используя правило грамматики с номером (2), то i входит в основу и доступен процессору. Следовательно, в это правило, мы можем включить операционный символ, который записывает на выходную ленту начальный участок перевода, вплоть до символа N . Правило транслирующей грамматики с номером (2) будет выглядеть следующим образом: (2) $A \rightarrow \text{int } i=\text{const1}; \{“i=\text{const1}”\}$.

При выполнении свертки по этому правилу операционный символ записывает на выходную ленту параметр цикла i . Рассуждая подобным образом, мы получим следующие правила транслирующей грамматики, содержащие элементы перевода (в порядке их использования при анализе входной цепочки и построении выходной Цепочки):

- (2) $A \rightarrow \text{int } i=\text{const1}; \{“i=\text{const1}\n”\}$
- (4) $F \rightarrow (i < N) \{ \{“i < N:\n”\}$
- (5) $C \rightarrow \text{body}; \{“\tbody\n”\}$
- (6) $D \rightarrow i+=\text{const2}; \{“\ti+=\text{const2}\n”\}$

Добавив остальные правила грамматики, получим следующую транслирующую грамматику:

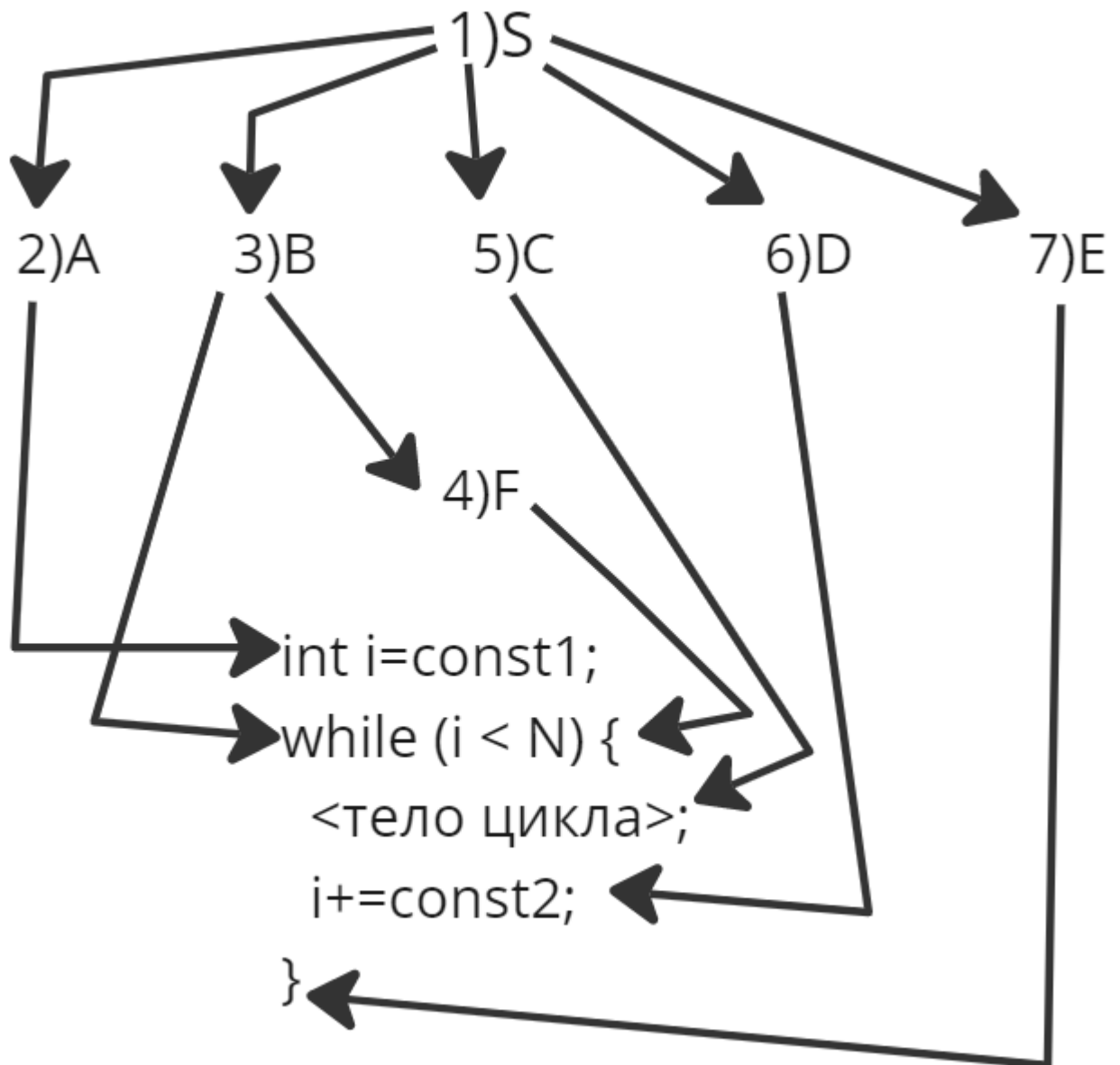
- 1) $S \rightarrow ABCDE$
- 2) $A \rightarrow \text{int } i=\text{const1}; \{“i=\text{const1}\n”\}$
- 3) $B \rightarrow \text{while } F \{“\text{while } ”\}$
- 4) $F \rightarrow (i < N) \{ \{“i < N:\n”\}$
- 5) $C \rightarrow \text{body}; \{“\tbody\n”\}$
- 6) $D \rightarrow i+=\text{const2}; \{“\ti+=\text{const2}\n”\}$
- 7) $E \rightarrow \}$

Пример вывода цепочки:

$(S, “”) \Rightarrow^1 (ABCDE, “”) \Rightarrow^2 (\text{int } i=\text{const1};BCDE, “i=\text{const1}\n”) \Rightarrow^3 (\text{int } i=\text{const1};\text{while } FCDE, “i=\text{const1}\n\text{while } ”) \Rightarrow^4 (\text{int } i=\text{const1};\text{while } (i < N) \{CDE, “i=\text{const1}\n\text{while } i < N:\n”) \Rightarrow^5 (\text{int } i=\text{const1};\text{while } (i < N) \{\text{body};DE, “i=\text{const1}\n\text{while } i < N:\n\tbody\n”) \Rightarrow^6 (\text{int } i=\text{const1};\text{while } (i < N) \{\text{body};i+=\text{const2};E, “i=\text{const1}\n\text{while } i < N:\n\tbody\n\ti+=\text{const2}\n”) \Rightarrow^7 (\text{int }$

```
i=const1;while (i < N) {body;i+=const2;}, "i=const1\nwhile i <
N:\n\tbody\n\ti+=const2\n")
```

Дерево вывода атрибутно-транслирующей грамматики:



При восходящих методах синтаксического анализа строится обращенный правый вывод (разбор) цепочки - последовательность номеров правил, используемых при свертках, - который для рассматриваемого примера равен: (2), 4), 3), 5), 6), 7), 1)). При этом на выходной ленте формируется цепочка:

```
i=const1
while i < N:
    <тело цикла>
    i+=const2
```

Теперь мы можем расширить транслирующую грамматику до атрибутивной транслирующей грамматики.

$$1) S \rightarrow A_a B_b C D_d E$$

$$b, d \leftarrow a$$

$$2) A_a \rightarrow \text{int } i_{n1} = \text{const1}_{n2}; \{ \text{"i}_{n1} = \text{const1}_{n2} \backslash n"} \}_{n1, n2}$$

$$a \leftarrow n1$$

$$3) B_b \rightarrow \text{while } F_f \{ \text{"while "} \}$$

$$f \leftarrow b$$

$$4) F_f \rightarrow (i_s < N_{n1}) \{ \{ \text{"i}_s < N_{n1} : \backslash n"} \}_{s, n1}$$

$$s \leftarrow f$$

$$5) C \rightarrow \text{body}; \{ \text{"\tbody \n"} \}$$

$$6) D_d \rightarrow i_s += \text{const2}_{n1}; \{ \text{"i}_s += \text{const2}_{n1} \backslash n"} \}_{s, n1}$$

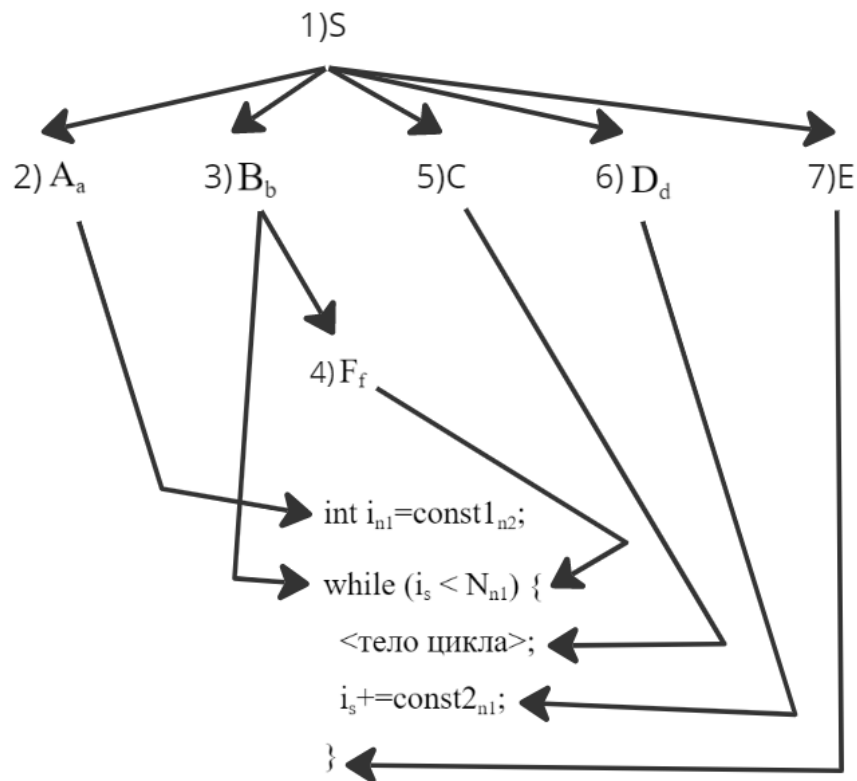
$$s \leftarrow d$$

$$7) E \rightarrow \}$$

Пример перевода из C++

Вход c++	Результат перевода в Python
<pre>int i=const1; while (i < N) { <тело цикла>; i+=const2; }</pre>	<pre>i=const1 while i < N: <тело цикла> i+=const2</pre>

Дерево вывода атрибутивно-транслирующей грамматики:



Алгоритм 1. построения АТ-грамматики из исходной Транслирующей грамматики

Вход: транслирующая грамматика $G(T, V, S, P)$

P:

1) $S \rightarrow ABCDE$

2) $A \rightarrow \text{int } i = \text{const1}; \{ \text{"i=const1\n"} \}$

3) $B \rightarrow \text{while } F \{ \text{"while "} \}$

4) $F \rightarrow (i < N) \{ \{ \text{"i < N:\n"} \} \}$

5) $C \rightarrow \text{body}; \{ \text{"\tbody\n"} \}$

6) $D \rightarrow i += \text{const2}; \{ \text{"\ti+=const2\n"} \}$

7) $E \rightarrow \}$

Выход: АТ-грамматика ATG

VF = Список нетерминалов

F = пустое множество

A = множество атрибутов

AF = цепочка выражения атрибутов

$Sign \subset T$ – множество математических операций

L – множество лексем

```
foreach (By  $\rightarrow$   $Xy_1, Xy_2 \dots Xy_n \in P$ ) { // Идём по правилам берем отдельный терминал или нетерминал
  y = a
  foreach ( $Xy_1, Xy_2 \dots Xy_n$ ) {
    if ( $Xy_j \in T$ ) { // проверяем входит он или нет в множ-во правил
      if ( $Xy_j \in Sign$ ) { // проверяем входит он или нет в множ-во математических
операций
        y1 = GetPrevious(yi) // если находим знак, то получаем символ идущий
перед ним
        y2 = GetNext(yi) // и после знака
        y1 += a // присваиваем атрибуты предшествующему
        y2 += a // присваиваем атрибуты следующему
        AF = AF  $\cup$  y1,y2 // добавляем в множество атрибутов
      }
    } else if ( $Xy_j \in V$ ) { // если находим нетерминал
      yj = a
      AF = AF  $\cup$  yj
    }
  }
  F = F  $\cup$  f0(y , AF)
  AF = пуст.мн-во
}
```

Контрольные вопросы

1. Как определяется понятие семантики языков программирования?
2. Как можно описать простейшие переводы?
3. Какие переводы называются синтаксически управляемыми?
4. Что такое элемент перевода?
5. Дайте определение СУ-схемы. Какие цепочки называются входными и выходными?
6. Как определить входную и выходную грамматики СУ-схемы?
7. Как происходит порождение пар цепочек под управлением СУ-схемы?
8. Как происходит преобразование деревьев под управлением СУ-схем?
9. Какие СУ-схемы называются простыми?
10. Какие переводы позволяют описать простые СУ-схемы?
11. Какая грамматика называется транслирующей грамматикой?
12. Что такое входная и выходная грамматики транслирующей грамматики?

13. Как определяется активная цепочка, ее входная и операционная части?
14. Как связаны вывод цепочки и дерево вывода активной цепочки?
15. Какая Грамматика называется атрибутивной транслирующей грамматикой?
16. Какие атрибуты называются унаследованными и синтезированными?
17. В чем основные различия между унаследованными и синтезированными атрибутами?
18. Что представляет собой правило вычисления атрибутов?
19. Как строится граф зависимостей?
20. Как граф зависимостей используется при вычислении атрибутов?
21. Какие виды объектов включаются в выходную цепочку при переводе?
22. Из каких этапов состоит процесс построения транслирующей грамматики?
23. Из каких этапов состоит процесс построения АТ-Грамматики?
24. Какие типы ошибок позволяет найти тестирование АТ-Грамматики?

Упражнения

1. Разработайте простую СУ-схему, описывающую перевод арифметических скобочных выражений, содержащих операции '+' и '*' в:
 - 1.1. постфиксную запись.
 - 1.2. префиксную запись.
2. В языке ALGOL-60 выражения строятся с помощью операций, имеющих следующие приоритеты (в порядке убывания):

1) ↑	4) ≤ < = ≠ > ≥	7) ∨
2) */ ÷	5) ¬	8) →
3) + -	6) ∧	9) ≡

 Разработайте простую СУ-схему, описывающую перевод инфиксных выражений, содержащих перечисленные операции, в постфиксную запись.
3. Постройте МП-преобразователи, реализующие СУ-схемы из упр. 1.
4. Для простой СУ-схемы:

$$E \rightarrow + EE, EE +$$

$$E \rightarrow * EE, EE *$$

$$E \rightarrow a, a$$
 постройте эквивалентный МП-преобразователь.
5. Для заданной СУ-схемы:

$$S \rightarrow a S^{(1)} b A c S^{(2)} d S^{(3)}, e A c S^{(2)} d S^{(3)} f S^{(1)} g$$

$$S \rightarrow i, i$$

$$A \rightarrow i, i$$
 постройте переводы для входных цепочек:
 - 5.1. aibicd .
 - 5.2. aaibicidibicid .
6. Постройте вывод в транслирующей грамматике G0T цепочек:
 - 6.1. $i^* (i+i)$.
 - 6.2. $(i+i) * (i+i)$.
7. Определите транслирующую грамматику, допускающую в качестве входа произвольную цепочку из нулей и единиц и порождающую на выходе:
 - 7.1. обращение входной цепочки.

7.2.цепочку 0^n1^m , где n - число нулей, а m - число единиц во входной цепочке.

8.Постройте транслирующую грамматику, определяющую перевод логических выражений, составленных из логических переменных, скобок и знаков операций дизъюнкции, конъюнкции и отрицания:

8.1.из инфиксной записи в ПОЛИЗ.

8.2.из ПОЛИЗ в инфиксную запись.

8.3.из инфиксной записи в функциональную запись такую, например, что выражение $a \cap (b \cup c)$ будет представлено в виде $f\cap (a, f\cup (b, c))$, где $f\cap$ и $f\cup$ - отдельные символы.

9.Постройте АТ-Граматику, описывающую перевод оператора присваивания некоторого Гипотетического языка программирования в цепочку тетради с кодами операций: ПРИСВОИТЬ, СЛОЖИТЬ, ВЫЧЕСТЬ, УМНОЖИТЬ, ДЕЛИТЬ.левой частью оператора присваивания является идентификатор, а правой частью бесскобочное арифметическое выражение, выполняемое справа налево в порядке написания операций. В арифметическом выражении можно использовать идентификаторы и знаки арифметических операций: +, -, *, /.

10. Постройте АТ-грамматику, описывающую перевод оператора присваивания некоторого гипотетического языка программирования в цепочку тетрад с кодами операций: ПРИСВОИТЬ, И, ИЛИ. НЕ.левой частью оператора присваивания является идентификатор, а правая часть представляет собой логическое выражение, составленное из идентификаторов, скобок и знаков логических операций: $\cap$, $\cup$, $\neg$. порядок выполнения логического выражения определяется обычным приоритетом логических операций.

11. Постройте АТ-грамматику, описывающую перевод в тетрадку с кодом операции ПЕРЕХОД_ПО_РАВНО условного оператора, которому соответствует следующее правило входной грамматики:

условный оператор $\rightarrow$ if = goto .

Лексема представляет собой номер оператора, к которому осуществляется переход в случае равенства, а - идентификатор.

12. Для входной грамматики, описывающей арифметические выражения, с правилами вывода:

$S \rightarrow EE \rightarrow (E)$

$E \rightarrow E := EE \rightarrow i$

$E \rightarrow E + E$

где - лексема, значением которой является указатель на элемент таблицы идентификаторов, а семантика выражений такая же, как в языке С, постройте АТ-грамматику, которая проверяет, представляет ли левая часть выражения значение, и, в случае успеха, строите ПОЛИЗ выражения.

13. Для входной грамматики, описывающей объявление переменных, с правилами вывода:

1. $D \rightarrow L$ 4. $\rightarrow$ int

2. $L \rightarrow , L$ 5. $\rightarrow$ real

3. $L \rightarrow :$

постройте АТ-грамматику, которая заносит значение типа каждого идентификатора в таблицу идентификаторов.

14. Для входной грамматики, описывающей арифметические выражения, с правилами вывода:

$$E \rightarrow (E + E)$$

$$E \rightarrow (E * E)$$

$$E \rightarrow i$$

где i - лексема, значением которой является указатель на элемент таблицы идентификаторов, определить постфиксную 8-атрибутную грамматику, описывающую перевод этих выражений в цепочку тетради с концами операций: СЛОЖИТЬ и УМНОЖИТЬ, и построить 8-атрибутный ДМП-процессор, выполняющий заданный перевод.

Глава 7. Языковые процессоры: L-атрибутные и S-атрибутные транслирующие грамматики

L-атрибутные и S-атрибутные транслирующие грамматики - два подкласса корректных АТ-грамматик, часто используемых при проектировании языковых процессоров.

7.1. L-атрибутные и S-атрибутные транслирующие грамматики КР алгоритм проверки АТ-грамматики.

Определение. АТ-грамматика называется L-атрибутной тогда и только тогда, когда выполняются следующие условия:

1. Аргументами правила вычисления значения унаследованного атрибута символа из правой части правила вывода могут быть только унаследованные атрибуты символа из левой части и произвольные атрибуты символов из правой части, расположенные левее рассматриваемого символа.
2. Аргументами правила вычисления значения синтезированного атрибута символа из левой части правила вывода являются унаследованные атрибуты этого символа или произвольные атрибуты символов из правой части.
3. Аргументами правила вычисления значения синтезированного атрибута операционного символа могут быть только унаследованные атрибуты этого символа.

Является ли произвольная АТ-грамматика L-атрибутной, можно проверить, независимо исследуя каждое правило вывода и каждое правило вычисления значения атрибута. Примером L-атрибутной транслирующей грамматики является грамматика G^A .

При выполнении условия 1 определения унаследованные атрибуты каждой вершины дерева вывода зависят (непосредственно или косвенно) только от атрибутов входных символов, расположенных в дереве левее данной вершины, что позволяет использовать L-атрибутные грамматики в нисходящих синтаксических анализаторах. Условия 2 и 3 введены с целью сделать АТ-грамматику корректной.

В L-атрибутной транслирующей грамматике атрибуты символов A, B и C из правила вывода

$A \rightarrow BC$ можно вычислять в следующем порядке:
унаследованные атрибуты символа A;
унаследованные атрибуты символа B;
синтезированные атрибуты символа B;
унаследованные атрибуты символа C;
синтезированные атрибуты символа C;
синтезированные атрибуты символа A.

В рамках описания алгоритма псевдокодом будем считать, что p - синтезированный p , r - унаследованный r .

$\square(\square_{\square\square}, \square)$. $\square\square\square : \square_{\square 1}, \square, \square_{\square 2}, \square, \dots, \square_{\square\square}, \square$ - аргументы правила вычисления синтезированного значения атрибута $\square_{\square\square}, \square$

$\alpha(\alpha_{\alpha\alpha}. \alpha)$. $\alpha\alpha\alpha : \alpha_{\alpha 1}. \alpha, \alpha_{\alpha 2}. \alpha, \dots, \alpha_{\alpha\alpha}. \alpha$ - аргументы правила вычисления унаследованного значения атрибута $\alpha_{\alpha\alpha}. \alpha$
 $\alpha_{\alpha\alpha}. \alpha\alpha\alpha\alpha\alpha\alpha\alpha\alpha\alpha$ - атрибут символа $\alpha_{\alpha\alpha}$
 $\{\alpha_{\alpha 1} \dots \alpha_{\alpha-1}\}$ - множество символов $\alpha_{\alpha\alpha} : \forall \alpha = 1, 2, \dots, \alpha - 1$

$\{\alpha(\alpha_{\alpha 1} \dots)\}$ - множество правил вычислений атрибута символа $\alpha_{\alpha 1}$

Алгоритм:

```

 $\alpha\alpha\alpha\alpha\alpha\alpha\alpha\alpha (\alpha_{\alpha} \rightarrow \alpha_{\alpha 1}, \dots, \alpha_{\alpha\alpha})$  // для всех правил грамматики
 $\alpha\alpha\alpha\alpha\alpha\alpha\alpha\alpha (\alpha_{\alpha\alpha} \in \{\alpha\})$  // для каждого символа, справа
 $\alpha\alpha (\alpha_{\alpha\alpha}. \alpha\alpha\alpha\alpha\alpha\alpha\alpha\alpha\alpha = \alpha)$ 
 $\alpha\alpha !(\alpha(\alpha_{\alpha\alpha}. \alpha). \alpha\alpha\alpha \subseteq \{\alpha_{\alpha 1} \dots \alpha_{\alpha-1} \cup \alpha_{\alpha}. \alpha\})$ 
 $\alpha\alpha\alpha\alpha\alpha\alpha \alpha\alpha\alpha\alpha\alpha$ 
 $\alpha + +$ 
 $\alpha\alpha (\alpha_{\alpha}. \alpha)$ 
 $\alpha\alpha !(\alpha(\alpha_{\alpha}. \alpha). \alpha\alpha\alpha \in \{\alpha_{\alpha 1} \dots \alpha_{\alpha} \cup \alpha_{\alpha}. \alpha\})$ 
 $\alpha\alpha\alpha\alpha\alpha\alpha \alpha\alpha\alpha\alpha\alpha$ 
 $\alpha\alpha (\alpha_{\alpha\alpha} = \alpha\alpha \alpha\alpha\alpha \alpha_{\alpha\alpha}. \alpha\alpha\alpha\alpha\alpha\alpha\alpha\alpha\alpha = \alpha)$ 
 $\alpha\alpha !(\alpha(\alpha_{\alpha\alpha}. \alpha). \alpha\alpha\alpha \subseteq \{\alpha_{\alpha\alpha}. \alpha\})$ 
 $\alpha\alpha\alpha\alpha\alpha\alpha \alpha\alpha\alpha\alpha\alpha$ 

 $\alpha\alpha\alpha \alpha\alpha\alpha\alpha\alpha\alpha\alpha\alpha$ 
 $\alpha + +$ 
 $\alpha\alpha\alpha \alpha\alpha\alpha\alpha\alpha\alpha\alpha\alpha$ 

 $\alpha\alpha\alpha\alpha\alpha\alpha \alpha\alpha\alpha\alpha$ 

```

Определение. АТ-грамматика называется S-атрибутной тогда и только тогда, когда она является L-атрибутной и все атрибуты нетерминалов синтезированные.

Ограничения, накладываемые на L-атрибутную транслирующую грамматику, позволяют вычислять значения атрибутов в процессе нисходящего анализа входной цепочки. Нисходящий детерминированный анализатор для LL(1)-грамматик требует, чтобы L-атрибутная транслирующая грамматика, описывающая перевод, имела форму простого присваивания.

7.2 Форма простого присваивания

При определении АТ-грамматики в правила вычисления атрибутов записывались в виде операторов присваивания, левые части которых представляют собой атрибут или список атрибутов, а правые части — функцию, использующую в качестве аргументов значения некоторых атрибутов.

Простейший случай функции из правой части оператора присваивания - тождественная или константная функция, например $a \leftarrow b$ или $x, y \leftarrow 1$.

Определение. Правило вычисления атрибутов называется **копирующим правилом** тогда и только тогда, когда левая часть правила - это атрибут или список атрибутов, а правая часть - константа или атрибут. Правая часть называется источником копирующего правила, а каждый атрибут из левой части - приемником копирующего правила.

Если источники нескольких копирующих правил совпадают, то их приемники можно объединить в одну левую часть. Например, правила $z, w \leftarrow a$ и $x, y \leftarrow z$ можно записать в виде: $x, y, z, w \leftarrow a$, т. к. источнику второго правила z , согласно первому правилу, присваивается значение a . Аналогично $x \leftarrow u$ и $a, b \leftarrow u$ можно записать как $a, b, x \leftarrow u$.

Определение. Множество копирующих правил называется независимым, если источник каждого правила из этого множества не входит ни в одно из других правил множества.

Два независимых копирующих правила, нельзя объединять.

Определение. Атрибутная транслирующая грамматика имеет форму простого присваивания тогда и только тогда, когда:

1. Некопирующими являются только правила вычисления синтезируемых атрибутов операционных символов.
2. Для каждого правила вывода грамматики соответствующее множество копирующих правил независимо.

Пример. Пусть дана L-атрибутная транслирующая грамматика, порождающая префиксные арифметические выражения над константами, в форме простого присваивания. Атрибутные правила вывода этой грамматики имеют следующий вид: E_p - синтезированный p , $\{\text{ответ}\}_r$ - унаследованный r :
 $G = (\{S, E_p\}, \{c_r, *, +\}, \{\{\text{ответ}\}_r, P, S\})$, где

$P = \{$

1. $S \rightarrow E_p \{\text{ответ}\}_r$

$r \leftarrow p$

2. $E_p \rightarrow +E_q E_r \{\text{сложить}\}_{A,B,R}$

$p \leftarrow q + r$

3. $E_p \rightarrow *E_q E_r$

$p \leftarrow q * r$

4. $E_p \rightarrow c_r$

$p \leftarrow r$

$\}$

Правила преобразования:

1. Каждой функции $f(x_1, x_2, \dots, x_n)$, входящей в правило вычисления атрибутов, связанное с некоторым правилом вывода грамматики, создать соответствующий ей операционный символ $\{f\}$, который определяется следующим образом:
 $\{f\} \ x_1 \ x_2, \dots, x_n, p$

унаследованные $x_1, x_2, \dots, x_n$
синтезированный p

$$p \leftarrow f(x_1, x_2, \dots, x_n)$$

2. Для каждого не копирующего правила $z_i, z_2, \dots, z_b \leftarrow f(y_1, y_2, \dots, y_n)$ связанного с некоторым правилом вывода грамматики, найти символы $a_1 \dots, a_n$, которые не содержатся в этом правиле вывода, и вставить в его правую часть символ $\{f\} a_n \leftarrow a_2, \dots, a_n$. Заменить не копирующее правило на следующие $(n + 1)$ копирующих правил:

$a_i \leftarrow y_n$ для каждого аргумента y_i ,

$z_1, z_2, \dots, z_m \leftarrow \Gamma$

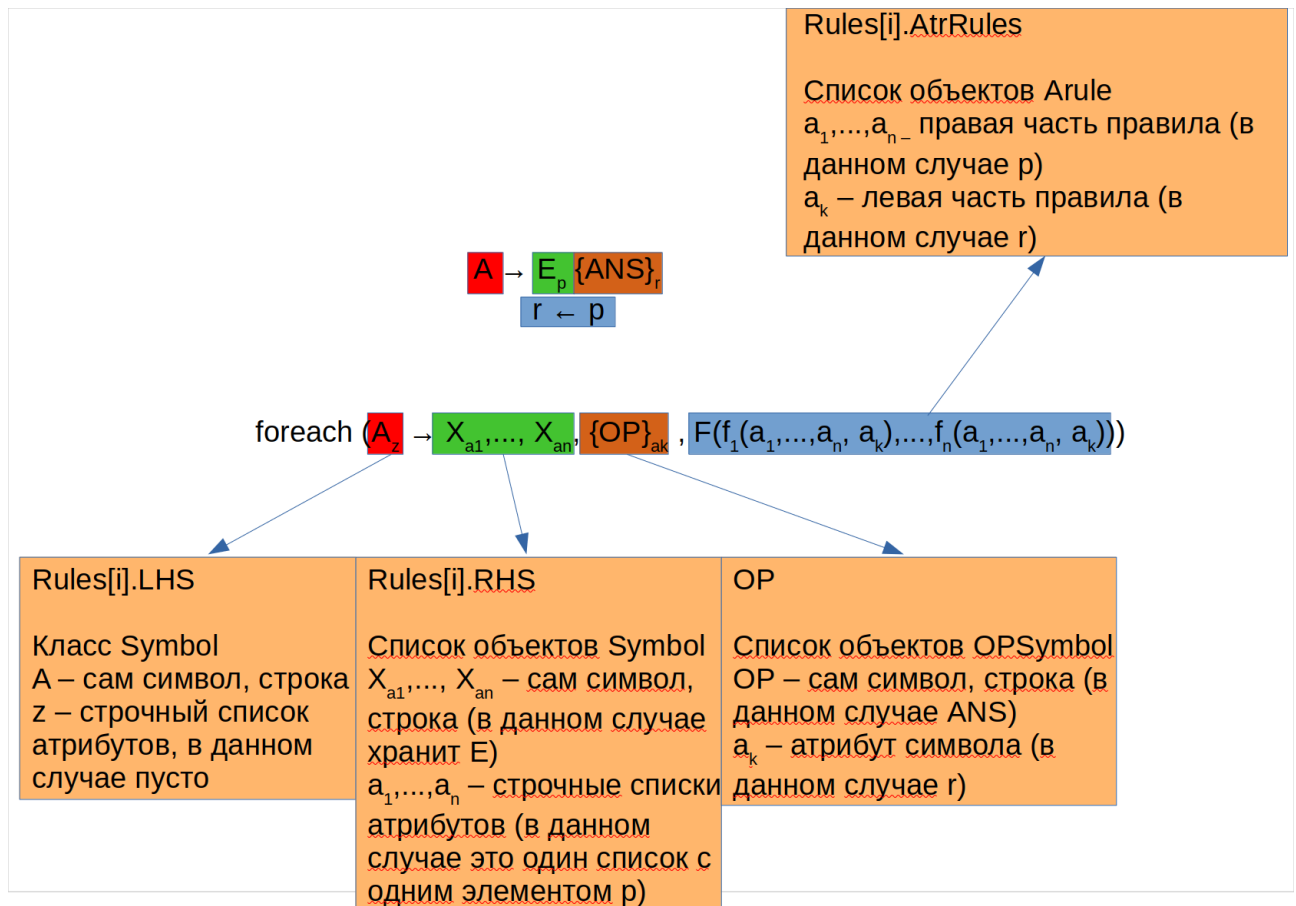
При включении в правило вывода операционного символа необходимо соблюдать следующие условия:

- операционный символ должен располагаться правее всех символов правой части правила вывода, атрибутами которых являются аргументы $y_1, \dots, y_n$;
- операционный символ должен располагаться левее всех символов правой части правила вывода, атрибутами которых служат $z_1, \dots, z_n$;
- с учётом предыдущих ограничений операционный символ может быть вставлен в любое место правой части правила вывода, но предпочтение следует отдать самой левой из возможных позиций, т.к. это позволяет упростить реализацию синтаксического анализатора.

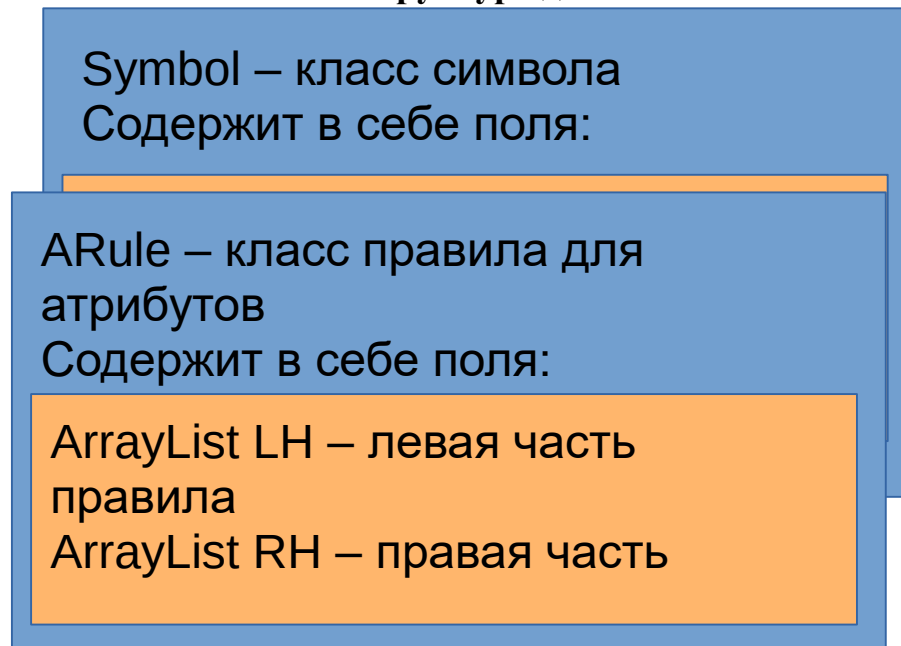
2. Два копирующих правила, соответствующие одному и тому же правилу вывода, необходимо объединить, если источник одного из них входит в другое. Это достигается удалением правила с лишним источником и объединением его приемников с приемниками оставшегося правила.

Если в качестве источника копирующего правила используется константная функция, являющаяся процедурой-функцией без параметров (например, функция GETNEW из АТ-грамматики), то такие атрибутивные правила объединять нельзя, т. к. два разных вызова функции без параметров могут давать разные значения.

В фреймворке реализация под пунктом **I4**.



Структура данных



OPSymbol – класс

Rule – класс правила

Содержит в себе поля:

Symbol LHS – левая часть правила

List<Symbol> RHS – правая часть правила, состоящая из списка СИМВОЛОВ

List<ARule> AtrRules – список правил для атрибутов

Примеры:

Символ X имеет один синтезируемый атрибут a и два наследуемых b и c.
Символы Y и Z имеют по одному наследуемому атрибуту d и e соответственно.
Тогда запись будет выглядеть так:

Type = (Syn, Inh)

Syn = {a}

Inh = {b, c, d, e}

X

a

Пример грамматики:

$c \rightarrow Y. d Z.e$

Type = (Syn, Inh)

Syn = {a, c, d, e, f, z}

Inh = {b, c, f, x, y}

x

~~y~~

~~y~~ $z \leftarrow x + y$

1) $S_0 \rightarrow E.x \{ответ\}.b$

$b \leftarrow a$

2) $E.c \rightarrow E.d + T.e \{+\}.x,y,z$

$x \leftarrow d$

$y \leftarrow e$

$c \leftarrow z$

$$3) E.f \rightarrow T.g$$

$$g \leftarrow f$$

$$4) T.c \rightarrow T.d * P.e \{*\}.x,y,z$$

$$x \leftarrow d$$

$$y \leftarrow e$$

$$c \leftarrow z$$

$$5) T.f \rightarrow P.g$$

$$f \leftarrow g$$

$$6) P.f \rightarrow (E.g)$$

$$f \leftarrow g$$

$$7) P.a \rightarrow C.b$$

$$a \leftarrow b$$

$F(f(b, a), f(x, d), f(y, e), f(c, z), f(f, g), f(a, b)).$

Type = (Syn, Inh)

Syn = {a, c, d, e, f, z}

Inh = {b, x, y}

Левая часть правила		Правая часть правила	Множество операционных символов правила	Множество атрибутивных функций
A_z	$\rightarrow$	$X_{a1}, \dots, X_{an}$	$\{OP\}_{ak}$	$F(f_{a1}(a_k, a_1, \dots, a_n), \dots, f_{an}(a_k, a_1, \dots, a_n))$
S_0	$\rightarrow$	$E.a$	$\{\text{ответ}\}.b$	$f(b, a)$
$E.c$	$\rightarrow$	$E.d + T.e$	$\{+\}.x, y, z$	$f(x, d), f(y, e), f(c, z), +(z, x, y)$
$E.f$	$\rightarrow$	$T.g$	$\emptyset$	$f(f, g)$
$T.c$	$\rightarrow$	$T.d * P.e$	$\{*\}.x, y, z$	$f(x, d), f(y, e), f(c, z), *(z, x, y)$
$T.f$	$\rightarrow$	$P.g$	$\emptyset$	$f(f, g)$
$P.f$	$\rightarrow$	$(E.g)$	$\emptyset$	$f(f, g)$
$P.a$	$\rightarrow$	$C.b$	$\emptyset$	$f(a, b)$

Представление в виде списка:

A $[X_{a1}, \dots, X_{an}]$,
z $\{OP\}_{ak}$,
 $F = [f([a_k, a_1, \dots, a_n]), \dots, f([a_k, a_1, \dots, a_n])]]$
]

$$[A_Z, [X_{a_1}, \dots, X_{a_n}], \{OP\}_{ak}, F = [f([a_k, a_1, \dots, a_n]), \dots, f([a_k, a_1, \dots, a_n])]$$

Е
Е
А
Г
У
Е
Е
Х
а
Е — функция простого присваивания

ВЫХОД: эквивалентная LAT-грамматика $G' = (S_0, V, T, \{OP\}', Type', P')$, $Type' =$

```

foreach (Az → Xa1, ..., Xan, {OP}a, F {fa1(ak, a1, ..., an), ..., fan(ak, a1, ..., an)})
  f
  a
  i //пустое множество атрибутов
  f
  foreach (aj ∈ {ak, a1, ..., an})
    f
    if (aj ∉ T)
      f0 //считается, что множества могут быть непустыми
      f0 //копирующая функция
      F = F ∪ f0(aj', aj)
      a = a ∪ aj'
    end if
  end foreach
  Syn = Syn ∪ a1" // a1" – новый синтезируемый атрибут
  F
  a
  f0(a1" a1") //копирующая функция
  a1" O //удалить из правил правило f
end if
end foreach
end foreach
OPa //добавить новый операционный символ

```

Пусть дана L-атрибутная транслирующая грамматика G_0 , допускающая в качестве входа арифметические выражения в префиксной форме, построенные из символов множества $\{c, +, *\}$, где c — лексема, являющаяся целочисленной константой, и порождающая на выходе значение этого выражения.

$G_0(T, V, P, S_0, \text{Type}, OS)$, где

$T = \{+, *, c\}$,

$V = \{E, T, P, S_0\}$,

$\text{Type} = (\text{Inh}, \text{Syn})$,

$\text{Inh} = \{r\}$,

$\text{Syn} = \{p, q, t\}$,

$OS = \{\{\text{ответ}\}_p\}$

P :

0. $S_0 \rightarrow E_q\{\text{ответ}\}_h; F\{f(q, h)\}$

1. $E_p \rightarrow +E_qT_t; F\{f(p, +(q, t))\}$

2. $E_p \rightarrow T_q; F\{f(p, q)\}$

3. $T_p \rightarrow *T_qP_t; F\{f(p, *(q, t))\}$

4. $T_p \rightarrow P_q; F\{f(p, q)\}$

5. $P_p \rightarrow E_q; F\{f(p, q)\}$

6. $P_p \rightarrow c_h; F\{f(p, h)\}$

Преобразуем L-атрибутную транслирующую грамматику G_0 в эквивалентную L-атрибутную транслирующую грамматику в форме простого присваивания.

Преобразуются правила (1) и (3):

1. $E_p \rightarrow +E_qT_t; F\{f(p, +(q, t))\}$

3. $T_p \rightarrow *T_qP_t; F\{f(p, *(q, t))\}$

↓ (добавляем копирующие функции для новых атрибутов)

1. $E_p \rightarrow +E_qT_t; F\{f(p, +(q, t)), f(a, q), f(b, t), f(p, r)\}$

3. $T_p \rightarrow *T_qP_t; F\{f(p, *(q, t)), f(a, q), f(b, t), f(p, r)\}$

↓ (добавляем копирующие функции для операций сложения и умножения)

1. $E_p \rightarrow +E_qT_t; F\{f(p, +(q, t)), f(a, q), f(b, t), f(p, r), f(r, +(a, b))\}$

3. $T_p \rightarrow *T_qP_t; F\{f(p, *(q, t)), f(a, q), f(b, t), f(p, r), f(r, *(a, b))\}$

↓ (удаляем старые правила)

1. $E_p \rightarrow +E_qT_t; F\{f(a, q), f(b, t), f(p, r), f(r, +(a, b))\}$

3. $T_p \rightarrow *T_qP_t; F\{f(a, q), f(b, t), f(p, r), f(r, *(a, b))\}$

↓ (добавляем операционные символы с новыми атрибутами)

1. $E_p \rightarrow +E_q T_t \{+\}_{a,b,r}; F\{f(a,q), f(b,t), f(p, r), f(r, +(a, b))\}$
3. $T_p \rightarrow *T_q P_t \{*\}_{a,b,r}; F\{f(a, q), f(b, t), f(p, r), f(r, *(a, b))\}$

Получим грамматику $G_1(T, V, P, S0, Type, OS)$, где

$T = \{+, *, c\}$,

$V = \{E, T, P, S0\}$,

$Type = (Inh, Syn)$,

$Inh = \{h, a, b\}$,

$Syn = \{p, q, t, r\}$,

$OS = \{\{\text{ответ}\}_h, \{+\}_{a,b,r}, \{*\}_{a,b,r}\}$,

P :

0. $S0 \rightarrow E_q \{\text{ответ}\}_h; F\{f(q,h)\}$
1. $E_p \rightarrow +E_q T_t \{+\}_{a,b,r}; F\{f(a,q), f(b,t), f(p, r), f(r, +(a, b))\}$
2. $E_p \rightarrow T_q; F\{f(p, q)\}$
3. $T_p \rightarrow *T_q P_t \{*\}_{a,b,r}; F\{f(a, q), f(b, t), f(p, r), f(r, *(a, b))\}$
4. $T_p \rightarrow P_q; F\{f(p, q)\}$
5. $P_p \rightarrow E_q; F\{f(p, q)\}$
6. $P_p \rightarrow c_h; F\{f(p, h)\}$

A_p	$\rightarrow$	$X_{a1}, \dots, X_{an}$	$\{OP_{a1}, \dots, OP_{ak}\}$	$F\{f_1(a_1, \dots, a_n, a_{n+1}, \dots, a_k), \dots, f_n(a_1, \dots, a_n, a_{n+1}, \dots, a_k)\}$
$S0_\emptyset$	$\rightarrow$	$E_{\{q\}}$	$\{\{\text{ответ}\}_{\{h\}}\}$	$\{f(q,h)\}$
$E_{\{p\}}$	$\rightarrow$	$+_\emptyset E_{\{q\}} T_{\{t\}}$	$\{\{+\}_{\{a,b,r\}}\}$	$\{f(a,q), f(b,t), f(p, r), f(r, +(a, b))\}$
$E_{\{p\}}$	$\rightarrow$	T_q	$\emptyset$	$\{f(p, q)\}$
$T_{\{p\}}$	$\rightarrow$	$*_\emptyset E_{\{q\}} T_{\{t\}}$	$\{\{*\}_{\{a,b,r\}}\}$	$\{f(a, q), f(b, t), f(p, r), f(r, *(a, b))\}$
$T_{\{p\}}$	$\rightarrow$	$P_{\{q\}}$	$\emptyset$	$\{f(p, q)\}$
$P_{\{p\}}$	$\rightarrow$	$E_{\{q\}}$	$\emptyset$	$\{f(p, q)\}$
$P_{\{p\}}$	$\rightarrow$	$c_{\{h\}}$	$\emptyset$	$\{f(p, h)\}$

Необходимо проверить правильность преобразования грамматики G_1 из грамматики G_0 . Для этого подсчитаем количество атрибутов и операционных символов.

Алгоритм подсчёта количества атрибутов.

Вход: АТ-грамматика $G=(T, V, P, S0, A, OS)$

Выход: counter - количество атрибутов грамматики G

counter := 0

foreach ($A_p \rightarrow X_{a1}, \dots, X_{an}, \{OP_{a1}, \dots, OP_{ak}\}, F\{f_1(a_1, \dots, a_n, a_{n+1}, \dots, a_k), \dots, f_n(a_1, \dots, a_n, a_{n+1}, \dots, a_k)\}$)

 foreach ($a_j \in p$)

 counter := counter + 1

```

end foreach
foreach ( $X_{aj} \in \{X_{a1}, \dots, X_{an}\}$ )
    foreach ( $a_i \in a_j$ )
        counter := counter + 1
    end foreach
end foreach
foreach ( $OP_{aj} \in \{OP_{a1}, \dots, OP_{ak}\}$ )
    foreach ( $a_i \in a_j$ )
        counter := counter + 1
    end foreach
end foreach
end foreach

```

Алгоритм подсчёта операционных символов.

Вход: АТ-грамматика $G=(T, V, P, S0, A, OS)$

Выход: counter - количество операционных символов грамматики G

```

counter := 0
foreach ( $A_p \rightarrow X_{a1}, \dots, X_{an}, \{OP_{a1}, \dots, OP_{ak}\}, F\{f_1(a_1, \dots, a_n, a_{n+1}, \dots, a_k), \dots, f_n(a_1, \dots, a_n, a_{n+1}, \dots, a_k)\}$ )
    foreach ( $OP_{aj} \in \{OP_{a1}, \dots, OP_{ak}\}$ )
        counter := counter + 1
    end foreach
end foreach

```

Если преобразование выполнено верно, то должны выполняться **равенства**:

$C'_{op} - C_{op} = \sum |\{\forall f \in F: f - \text{некопирующее и не правило вычисления синтезируемого атрибута операционного символа}\}|$

$C'_{attr} - C_{attr} = (\sum |\{\forall f \in F': f - \text{некопирующее}\}| + 1) * 2 + (\sum |\{\forall f \in F: f - \text{некопирующее и правило вычисления синтезируемого атрибута операционного символа}\}| + 1) * 2$

Умножение на два делается потому что некопирующие функции являются бинарными.

$$C'_{op} = 3, C_{op} = 1, C'_{attr} = 22, C_{attr} = 16$$

$$3 - 1 = 2 \text{ — верно}$$

$$22 - 16 = 3 * 2 + 0 * 2 = 6 \text{ — верно}$$

Значит, преобразование выполнено верно.

7.3. Атрибутный перевод для LL(1)-грамматик

Расширим "1-предсказывающий" алгоритм разбора так, чтобы он мог выполнять атрибутный перевод, определяемый L-атрибутной транслирующей грамматикой, входной грамматикой которой является LL(1)-грамматика. Моделирование такого алгоритма можно осуществить с помощью атрибутного ДМП-преобразователя с концевым маркером.

Сначала рассмотрим проблему выполнения синтаксически управляемого перевода, определяемого транслирующей грамматикой цепочечного перевода, входной грамматикой которого является LL(1)-грамматика.

7.3.1 Реализация синтаксически управляемого перевода для транслирующей грамматики

Преобразуем "1-предсказывающий" алгоритм разбора для LL(1)-грамматик, включив в него действия, обеспечивающие перевод входной цепочки, порождаемой входной грамматикой, в цепочку операционных символов и запись этой цепочки на выходную ленту. Преобразованный таким образом алгоритм в дальнейшем будем называть нисходящим детерминированным процессором с магазинной памятью (нисходящий ДМП-процессор). При этом, если из контекста ясно, что речь идет о нисходящем методе анализа, слово "нисходящий" будем опускать.

В транслирующей грамматике множество терминальных символов разбито на множество входных символов $\sum_i$, и множество операционных символов $\sum_a$.

Расширим алфавит магазинных символов, добавив в него операционные символы. Тогда $V_p = \sum_i \cup \sum_a \cup N$. Затем доопределим управляющую таблицу M , которая для транслирующей грамматики цепочечного перевода задает отображение множества $(V_p \cup \{\perp\}) \times (\sum_i \cup \{\epsilon\})$ в множество, состоящее из следующих элементов:

(β, u) , где $\beta \in V_p^*$ — цепочка из правой части правила транслирующей грамматики $A \rightarrow u\beta$, а $u \in \sum_a^*$;

ВЫДАЧА (X) , где $X \in \sum_a$

ВЫБРОС;

ДОПУСК;

ОШИБКА.

Пусть $FIRST(x) = a$, где x — неиспользованная часть входной цепочки. Тогда работу ДМП-процессора в зависимости от элемента управляющей таблицы $M(X, a)$ можно определить следующим образом:

1. $(x, Xa, \pi) \vdash (x, \beta a)$, если $M(X, a) = (\beta, u)$. Верхний символ магазина X заменяется цепочкой $\beta \in V_p^*$, и в выходную цепочку дописывается цепочка операционных символов u . Входная головка при этом не сдвигается.

2. $(x, Xa, \pi) \vdash (x, a, \pi X)$, если $M(X, a) = \text{ВЫДАЧА } \{X\}$. Это означает, что если верхний символ магазина — операционный символ, то он выталкивается из магазина и записывается на выходную ленту. Входная головка не сдвигается.

Действия, соответствующие элементам управляющей таблицы: ВЫБРОС, ДОПУСК и ОШИБКА, остаются теми же, что и в "1-предсказывающем" алгоритме для LL(1)-грамматик.

Опишем алгоритм построения управляющей таблицы М.

Кр Алгоритм построения управляющей таблицы для транслирующей грамматики цепочечного перевода, входной грамматикой которой является LL(1)-грамматика.

Вход: Транслирующая грамматика цепочечного перевода $G^{Tb} = (T, \sum_i, \sum_a, P, S)$, входная грамматика которой является LL(1)- грамматика.

Выход : Корректная управляющая таблица М для грамматики G^{Tb} .

Описание алгоритма:

Управляющая таблица М определяется на множестве $(N \cup \sum_i \cup \sum_a \cup \{\perp\}) \times \sum_i \cup \{\epsilon\}$ по следующим правилам:

Если $A \rightarrow u\beta$ - правило вывода грамматики G_T , где $u \in \sum_a^*$, а β — либо ϵ , либо цепочка, начинающаяся с терминала или нетерминала, то $M\{A, a\} = (\beta, u)$ для всех $a \neq \epsilon$, принадлежащих множеству $FIRST(\beta)$.

Если $\epsilon \in FIRST(\beta)$, то $M(A, b) = (\beta, u)$ для всех $b \in FOLLOW(A)$.

Заметим, что при вычислении $FIRST(\beta)$ операционные символы, входящие в цепочку β , вычеркиваются.

$M(X, a) = \text{ВЫДАЧА}(X)$ для всех $x \in \sum_a$ и $a \in \sum_i \cup \{\epsilon\}$.

$M(a, a) = \text{ВЫБРОС}$ для всех $a \in \sum_i$.

$M(\perp, \epsilon) = \text{допуск}$

В остальных случаях $M(X, a) = \text{ОШИБКА}$ для $X \in (N \cup \sum_i \cup \sum_a \cup \{\perp\})$ и $a \in \sum_i \cup \{\epsilon\}$.

Построим управляющую таблицу для транслирующей грамматики, описывающей перевод инфиксных арифметических выражений в ПОЛИЗ. Эта транслирующая грамматика получена из входной LL(1)-грамматики G_1 путем включения в нее операционных символов $\{i\}$, $\{+\}$ и $\{*\}$:

$E \rightarrow E'$ (5) $T' \rightarrow * P\{*\} T'$

$E' \rightarrow + T\{+\} E'$ (6) $T' \rightarrow \epsilon$

$E' \rightarrow \epsilon$

$P \rightarrow i\{i\}$

$T \rightarrow PT'$

(8) $P \rightarrow (E)$

Управляющая таблица должна содержать 14 строк, помеченных символами из множества $N \cup \sum_i \cup \sum_a \cup \{\perp\}$, и 6 столбцов, помеченных символами из множества $\sum_i \cup \{\epsilon\}$.

Построение управляющей таблицы для строк, отмеченных символами из множества $N \cup \sum_i \cup \{\perp\}$, ничем не отличается от построения таблицы для соответствующей входной LL(1)-грамматики (табл. 1.1), а строки управляющей таблицы, отмеченные операционными символами, содержат значения $\text{ВЫДАЧА}\{X\}$, где $X \in \{i, +, *\}$. Заметим, что элементы таблицы, соответствующие строкам, помеченным операционными символами, для всех столбцов одинаковые, т. к. действия, выполняемые ДМП-процессором в случае,

когда верхним символом магазина является операционный символ, не зависят от входного символа.

Таблица 1.1. Управляющая таблица М для транслирующей грамматики, описывающей перевод инфиксных арифметических выражений в ПОЛИЗ

	i	(	)	+	*	E
E	TE', ,ε	TE',ε				
E'			ε, ε	*P{*} T', ε		ε, ε
T	PE', ε	PE',ε				
T'			ε, ε	ε, ε	*P{*} T', ε	ε, ε
P	i{i} ,ε	(E),ε				
i	ВЫБ РОС					
(		ВЫБРОС				
)			ВЫБРОС			
+				ВЫБРОС		
*					ВЫБРОС	
⊥						ДОПУСК
{i}	ВЫДАЧА({i})					
{+}	ВЫДАЧА({+})					
{*}	ВЫДАЧА({*})					
Начальное содержимое магазина - E⊥						

Начальное содержимое магазина — E⊥

Для входной цепочки i + i * i ДМП-процессор проделает следующую последовательность тактов:

$(i + i * i, E⊥, ε) \vdash (i + i * i, TE'⊥, ε)$

$\vdash (i + i * i, PT'E'⊥, ε)$

$\vdash (i + i * i, \{i\} T'E'⊥, ε)$

$\vdash (+ i * i, \{i\} T'E'⊥, ε)$

$\vdash (+ i * i, \{i\} T'E'⊥, \{i\})$

$\vdash (+ i * i, \{i\} E'⊥, \{i\})$

$\vdash (+ i * i, +T \{+\} E'⊥, \{i\})$

$\vdash (i * i, T \{+\} E'⊥, \{i\})$

$\vdash (i * i, PT' \{+\} E'⊥, \{i\})$

$\vdash (i * i, i \{i\} T' \{+\} E'⊥, \{i\})$

$\vdash (* i, \{i\} T' \{+\} E'⊥, \{i\})$

$\vdash (* i, T' \{+\} E'⊥, \{i\} \{i\})$

$\vdash (* i, *P \{*\} T' \{+\} E'⊥, \{i\} \{i\})$

$\vdash (i, P \{*\} T' \{+\} E'⊥, \{i\} \{i\})$

$\vdash (i, i \{i\} \{*\} T' \{+\} E'⊥, \{i\} \{i\})$

$\vdash (ε, \{i\} \{*\} T' \{+\} E'⊥, \{i\} \{i\})$

$$\begin{aligned} &\vdash (\varepsilon, \{i\}\{*\}T'\{+\}E'\perp, \{i\} \{i\}) \\ &\vdash (\varepsilon, T'\{+\}E'\perp, \{i\} \{i\}\{i\}\{*\}) \\ &\vdash (\varepsilon, \{+\}E'\perp, \{i\} \{i\}\{i\}\{*\}) \\ &\vdash (\varepsilon, E'\perp, \{i\} \{i\}\{i\}\{*\}\{+\}) \\ &\vdash (\varepsilon, \perp, \{i\} \{i\}\{i\}\{*\}\{+\}) \end{aligned}$$

ДМП-процессор можно использовать в качестве базового процессора для других видов синтаксически управляемого перевода, если операцию выдачи операционного символа в выходную ленту заменить операциями вызова соответствующих семантических процедур.

7.3.2 L-атрибутный ДМП-процессор

Процедура преобразования ДМП-процессора в атрибутный ДМП-процессор, которая описывается в данном пособии, требует, чтобы **АТ-грамматика, определяющая перевод, имела форму простого присваивания.**

Рассмотрим построение L-атрибутного ДМП-процессора, реализующего перевод, определяемый L-атрибутой транслирующей грамматикой в форме простого присваивания, входной грамматикой которого является LL(1)-грамматика.

В управляющую таблицу добавляются:

- операционные символы
- атрибуты
- операция “ВЫДАЧА”

При заполнении таблицы в ячейки записываются операционные символы вместо правил и их номеров. На выходную ленту идут операционные символы.

КР L-атрибутный ДМП-процессор: Сначала построим ДМП-процессор, реализующий цепочечный перевод, описываемый транслирующей грамматикой цепочечного перевода, которая получается из заданной L-атрибутой транслирующей грамматики после удаления из нее всех атрибутов. Затем расширим полученный таким образом ДМП-процессор, включив для каждого магазинного символа множество полей для представления атрибутов символа и дополнив управляющую таблицу действиями по вычислению атрибутов и записи их в соответствующие поля.

Магазинный символ с n атрибутами представляется в магазине **объектом** (n + 1)-ой ячейками, верхняя из которых содержит имя символа, а остальные — поля для атрибутов. Поля магазинного символа доступны для записи и извлечения атрибутов от момента вталкивания символа в магазин до момента выталкивания его из магазина.

В момент вталкивания символа в магазин в поле для каждого синтезированного атрибута и атрибута входного символа заносится указатель на связанный список полей, соответствующих унаследованным атрибутам, где этот атрибут должен запоминаться, а поле для каждого унаследованного атрибута

остаётся пустым. Содержимое полей синтезированных атрибутов и атрибутов входных символов остаётся неизменным в течение всего времени нахождения символа в магазине, а поля, соответствующие унаследованным атрибутам, приобретают значения атрибутов к моменту времени, когда магазинный символ окажется в верхушке магазина.

Пусть x — остаток входной цепочки, и $\text{FIRST}(x) = a$. Опишем действия, которые должен выполнять L-атрибутный ДМП-процессор в зависимости от элемента управляющей таблицы $M(X, o)$, где X — символ в верхушке магазина.

Начальная конфигурация. В магазине находится маркер дна и начальный символ грамматики. Поля начального символа грамматики, соответствующие унаследованным атрибутам, заполняются начальными значениями атрибутов, которые задаются L-атрибутной транслирующей грамматикой, а в поля, соответствующие синтезированным атрибутам, заносятся пустые указатели, которые служат маркерами конца списков.

$M(X, a) = \text{ВЫБРОС}$ (символ в верхушке магазина совпадает с текущим входным символом). В этом случае каждое поле верхнего магазинного символа содержит указатель на список полей магазина, в которых требуется поместить значение атрибута текущего входного символа. Операция ВЫБРОС расширяется таким образом, что каждый атрибут текущего входного символа копируется во все поля списка на который указывает соответствующее поле верхнего магазинного символа.

$M(X, a) = \text{ВЫДАЧА}(X)$ (в верхушке магазина находится операционный символ). Операция ВЫДАЧА(X) ДМП-процессора расширяется следующим образом:

унаследованных атрибутов извлекаются из соответствующих полей верхнего магазинного символа и используются затем при выдаче символа в выходную цепочку. При этом следует помнить, что если операционный символ появился в результате преобразования исходной атрибутной транслирующей грамматики в форму простого присваивания, то такой символ в выходную цепочку не выдается;

значение синтезированных атрибутов вычисляются по правилам вычисления атрибутов, связанным с данным операционным символом, после чего значение каждого синтезированного атрибута помещается во все поля списка, на который указывает соответствующее поле символа из верхушки магазина.

$M(X, a) = (\beta, y)$ (в верхушке магазина находится нетерминал). В этом случае L-атрибутный ДМП-процессор вталкивает в магазин цепочку символов β из правой части распознаваемого правила В DSLFTN WTGJXRE операционных символов y . При этом вычисляются атрибуты операционных символов, которые не вталкиваются в магазин, и заполняются поля атрибутов магазинных символов и символов цепочки β , вталкиваемых в магазин.

Источниками атрибутных правил, связанных с правилами вывода L-атрибутной транслирующей грамматики в форме простого присваивания, могут быть только константы, унаследованные атрибуты нетерминалов из левой части правил вывода, атрибуты входных и операционных символов, синтезированные атрибуты нетерминалов из правой части правил вывода. В табл. 1.2 приведены

значения источников копирующих правил в момент времени, когда верхним символом магазина является нетерминалы унаследованные атрибуты символов из правой части правил вывода и унаследованные атрибуты символов из правой части правил вывода. В табл. 1.3 приведены поля магазина, соответствующие приемникам атрибутных правил, которые необходимо заполнить во время перехода L-атрибутного ДМП-процессора при $M\{X, a\} = (\beta, y)$.

При выполнении перехода L-атрибутный ДМП-процессор выполняет следующие атрибутные действия:

Вычисляет значения синтезированных атрибутов операционных символов, которые не вталкиваются в магазин, и выдает цепочку операционных символов с их атрибутами в выходную цепочку, если эти символы не появились в результате преобразования грамматики в форму простого присваивания.

Если источник копирующего правила доступен, то значение источника помещается в соответствующее поле магазинного символа.

Если источник копирующего правила недоступен, то после вталкивания символа в магазин в соответствующие поля символа заносятся указатели на список полей, где будут храниться значения унаследованных атрибутов.

Действия L-атрибутного ДМП-процессора для элементов управляющей таблицы, имеющих значения ДОПУСК и ОШИБКА, остаются теми же самыми, что у ДМП-процессора.

Построим L-атрибутный ДМП-процессор для L-атрибутной транслирующей грамматики в форме простого присваивания.

На рис. 1.4 показано представление полей магазинных символов, а в табл. 1.5 приведена управляющая таблица для транслирующей грамматики, полученной из исходной L-атрибутной транслирующей грамматики путем вычеркивания из нее атрибутов.

Таблица 1.5. Управляющая таблица для транслирующей грамматики, полученной из исходной грамматики

S	$i := E \{ := \}, \varepsilon$				
E	iR, ε				
R			$+ i \{ + \} R, \varepsilon$	$* i \{ * \}$	ε, ε
/	ВЫБРОСС				
$:=$		ВЫБРОС			
+			ВЫБРОС		
*				ВЫБР	
$\perp$					ДОПУСК
$\{ := \}$	ВЫДАЧА($\{ := \}, p, q$)				
$\{ + \}$	ВЫДАЧА($\{ + \}, p, q, r$)				
$\{ * \}$	ВЫДАЧА($\{ * \}, p, q, r$)				

Начальное содержимое магазина - $S\perp$

Теоретические материалы

Элементы множеств нетерминальных, терминальных символов и операционных символов пара, состоят из номера правила и номера грамматического вхождения этого символа в правило.

Построим ДМП-процессор, реализующий цепочечный перевод, описываемой заданной L-атрибутной транслирующей грамматикой цепочечного перевода. Затем расширим полученный таким образом ДМП-процессор, дополнив операции ВЫБРОС и ВЫДАЧА управляющей таблицы.

Операция ВЫБРОС дополняется таким образом, что при её выполнении атрибуты входного символа копируются в атрибуты символа вершины стека.

Операция ВЫДАЧА дополняется таким образом, что при её выполнении на выходную ленту выдаётся не только операционный символ, но и его атрибуты.

В транслирующей грамматике множество терминальных символов разбито на множество входных символов $\sum_i$, и множество операционных символов $\sum_a$. То есть, исходное множество терминальных символов $T = \sum_i \cup \sum_a$.

Алгоритм построения L-атрибутного ДМП-процессора:

Все элементы множеств нетерминальных, терминальных символов и операционных символов будут представлять собой пару, состоящую из номера правила и номера грамматического вхождения этого символа в правую часть этого правила. Например, 1.2 означает, что символ входит в первое правило на второй позиции в правой части. Если символ входит в несколько правил, то пара дополняется альтернативами: 2.2|3.4|4.1 означает, что символ входит во второе, третье и четвертое правило на второй, четвертой и первой позиции в правой части соответственно.

Правила обрабатываются следующим образом:

Правила вида:

$$n. A \rightarrow A_1 A_2 \dots A_k$$

Преобразуются в правила следующего вида:

$$n. A \rightarrow n.1 \ n.2 \ \dots \ n.k$$

Например:

Правила вида:

$$\begin{array}{ll} 1. S & \rightarrow \ i_{p1} = E\{=\}_{p2} \\ 2. E & \rightarrow \ i_{p1} R_{p2} \end{array}$$

Преобразуются в правила следующего вида:

$$\begin{array}{ll} 1. S & \rightarrow \ 1.1 \ 1.2 \ 1.3 \ 1.4 \\ 2. E & \rightarrow \ 2.1 \ 2.2 \end{array}$$

Таким образом, для терминального символа i получаются следующие грамматические вхождения: 1.1|2.1, где символ | обозначает альтернативу.

Вход: L-атрибутная транслирующая грамматика цепочечного перевода

$$G^{AT} = \{N, \sum_i, \sum_a, P, S\}$$

Выход: Управляющая таблица M для грамматики G^{AT}

$$M = \text{matrix} ((N \cup \sum_i \cup \sum_a \cup \{\perp\}) * (\sum_i \cup \{\epsilon\}))$$

Шаг 1. Заполняем ячейки матрицы операцией ОШИБКА

for ($0 \leq i \leq |N \cup \sum_i \cup \sum_a \cup \{\perp\}|$, $i++$)

for ($0 \leq j \leq |\sum_i \cup \{\epsilon\}|$, $j++$)

$$M[i,j] = \text{ОШИБКА}$$

Шаг 2. Заполняем ячейки матрицы операцией ДОПУСК

$$M[\perp, \epsilon] = \text{ДОПУСК}$$

Шаг 3. Заполняем ячейки матрицы номером вставляемого правила

```
foreach (A → Xa1...Xan, {OP}ak, F(f1(a1,...,an, ak),...,fn(a1,...,an, ak)) ∈ P | y ∈ Σa* && (
FIRST(Xa1...Xan) ∈ {N ∪ Σi ∪ Σa} || Xa1...Xan = ε ))
    foreach (a ∈ FIRST(Xa1...Xan) | a ≠ ε)
        M[ID(A),a] = pair(NUM(A → Xa1...Xan), 1)
    foreach (b ∈ FOLLOW(A) | ε ∈ FIRST(Xa1...Xan))
        M[ID(A),b] = pair(NUM(A → Xa1...Xan), 1)
```

Шаг 4. Заполняем ячейки матрицы операцией ВЫДАЧА

```
foreach (X ∈ Σa)
    foreach (a ∈ Σi ∪ {ε})
        M[ID(X),a] = ВЫДАЧА(X)

foreach M[ID(Xa1,...,an), a] == ВЫДАЧА(X)
    for (0 ≤ i ≤ n, i++)
        ВЫДАЧА(αi)
```

Шаг 5. Заполняем ячейки матрицы операцией ВЫБРОС

```
foreach (a ∈ Σi)
    M[ID(a),a] = ВЫБРОС
```

```
foreach M[ID(aα1,...,aan), aβ1,...,βn]
    for (0 ≤ i ≤ n, i++)
        αi = βi
```

Функция ID(A) от символа A возвращает пару, состоящую из номера правила и номера грамматического вхождения этого символа в правую часть правила.

Функция NUM от правила возвращает номер правила.

Пример:

Грамматика $G^{AT} = \{N, \Sigma_i, \Sigma_a, P, S\}$

1. S → i_{p1} = E{=}p₂
p₂ ← p₁
2. E → i_{p1} R_{p2}
p₂ ← p₁
3. R_{p1} → +i_{q1} {+}p_{2,q2,r1} R_{r2}
r₁, r₂ ← COUNTER
p₂ ← p₁
q₂ ← q₁
4. R_p → *i_{q1} {*}p_{2,q2,r1} R_{r2}
r₁, r₂ ← COUNTER
p₂ ← p₁
q₂ ← q₁
5. R_{p1} → ε

Построение управляющей таблицы

	i	=	+	*	ε
1.3	2.1				
2.2 3.4 4.4			3.1	4.1	5.1

1.1 2.1 3.2 4.2	ВЫБРОС				
1.2		ВЫБРОС			
3.1			ВЫБРОС		
4.1				ВЫБРОС	
⊥					ДОПУСК
1.4	ВЫДАЧА(={ } p)				
3.3	ВЫДАЧА(={+} p q r)				
4.3	ВЫДАЧА(={*} p q r)				

Работа со стеком:

Шаг 1. Достается верхний элемент магазина, представляющий собой пару: номер правила и номер грамматического вхождения символа в правую часть этого правила.

Шаг 2. Получаем операцию из управляющей таблицы (нахождением нужной ячейки в управляющей таблице согласно верхнему элементу магазина и символу на входной ленте).

Шаг 2(а). Если в найденной ячейке управляющей таблицы операция ВЫБРОС, то управляющая головка передвигается на следующий символ входной ленты, при этом производятся действия над атрибутами. Затем инкрементируется значение второго элемента пары, и элемент вталкивается в магазин (т.е. если достается элемент 2.1, то вталкивается элемент 2.2). При этом, если после инкрементирования второе число пары стало больше, чем длина правой части правила, то такая пара в магазин не вталкивается.

Шаг 2(б). Если в найденной ячейке управляющей таблицы операция ВЫДАЧА, то операционный символ выводится на выходную ленту с соответствующими атрибутами. Затем инкрементируется значение второго элемента пары, и элемент вталкивается в магазин (т.е. если достается элемент 2.1, то вталкивается элемент 2.2). При этом, если после инкрементирования второе число пары стало больше, чем длина правой части правила, то такая пара в магазин не вталкивается.

Шаг 2(в). Если в найденной ячейке управляющей таблицы находится элемент в виде пары, то инкрементируется значение второго элемента пары, которую достали на 1 шаге и этот элемент вталкивается в магазин (т.е. если достается элемент 2.1, то вталкивается элемент 2.2). При этом, если после инкрементирования второе число пары стало больше, чем длина правой части правила, то такая пара в магазин не вталкивается. Затем вталкивается элемент, который находится в найденной ячейке управляющей таблицы.

Шаг 2(г). Если в найденной ячейке управляющей таблицы операция ДОПУСК, то разбор заканчивается успехом и выводится выходная лента.

Шаг 3. Повторять шаги 1 и 2 до тех пор, пока в найденной ячейке управляющей таблицы не будет находиться операция ДОПУСК или ОШИБКА.

Такты срабатывания для цепочки $i_7 = i_1 + i_2 * i_3$

$(1.1\perp, i_7 = i_1 + i_2 * i_3, \varepsilon) \vdash (1.2\perp, = i_1 + i_2 * i_3, \varepsilon) \vdash (1.3\perp, i_1 + i_2 * i_3, \varepsilon) \vdash$

$(2.1; 1.4\perp, i_1 + i_2 * i_3, \varepsilon) \vdash (2.2; 1.4\perp, + i_2 * i_3, \varepsilon) \vdash (3.1; 1.4\perp, + i_2 * i_3, \varepsilon) \vdash (3.2; 1.4\perp, i_2 * i_3, \varepsilon) \vdash$

$(3.3; 1.4\perp, * i_3, \varepsilon) \vdash (3.4; 1.4\perp, * i_3, \{+\}120) \vdash (4.1; 1.4\perp, * i_3, \{+\}120) \vdash (4.2; 1.4\perp, i_3, \{+\}120) \vdash$

$\vdash (4.3; 1.4\perp, \varepsilon, \{+\}120) \vdash (4.4; 1.4\perp, \varepsilon, \{+\}120\{*\}130) \vdash (5.1; 1.4\perp, \varepsilon, \{+\}120\{*\}130) \vdash$

$(1.4\perp, \varepsilon, \{+\}120\{*\}130) \vdash (\perp, \varepsilon, \{+\}120\{*\}130\{=\}7) \vdash \text{ДОПУСК}$

7.4. Атрибутный перевод методом рекурсивного спуска

Сначала рассмотрим, каким образом можно изменить процедуры для распознавания цепочек, порождаемых нетерминальными символами грамматики, для реализации перевода, описываемого транслирующей грамматикой цепочечного перевода.

В этом случае правила составления процедур дополняются следующим правилом: если текущим символом правой части правила вывода грамматики является операционный символ Y , ему соответствует вызов процедуры записи операционного символа в выходную цепочку `output` (Y).

В качестве примера реализации перевода с использованием метода рекурсивного спуска рассмотрим транслирующую грамматику, описывающую перевод инфиксных арифметических выражений в польскую инверсную запись. Эта грамматика построена на основе входной грамматики $G1$ и имеет следующие правила:

$$\begin{aligned} S &\rightarrow E \perp \\ E &\rightarrow TE' \\ E' &\rightarrow +T\{+\} E' \mid \varepsilon \\ T &\rightarrow PT' \\ T' &\rightarrow *P\{*\} T' \mid \varepsilon \\ P &\rightarrow (E) \mid i \{i\} \end{aligned}$$

Рассмотрение данного дерева происходит сверху – вниз и слева – направо.

Если данная вершина находится в квадрате, то мы перемещаемся по стрелочкам (сплошным), начиная с самой левой. Если же в ходе обхода встретился узел в круге, то это означает, что был найденный узел совпал с исследуемым элементом цепочки, дальше необходимо вернуться вверх по дереву по пунктирной стрелочке (в основном это на уровень самого первого разветвления, когда у узла больше 1 исходящей стрелки). Если же данный узел является самым правым в дереве, то необходимо подняться на два узла, имеющих более 1 стрелки, вверх.

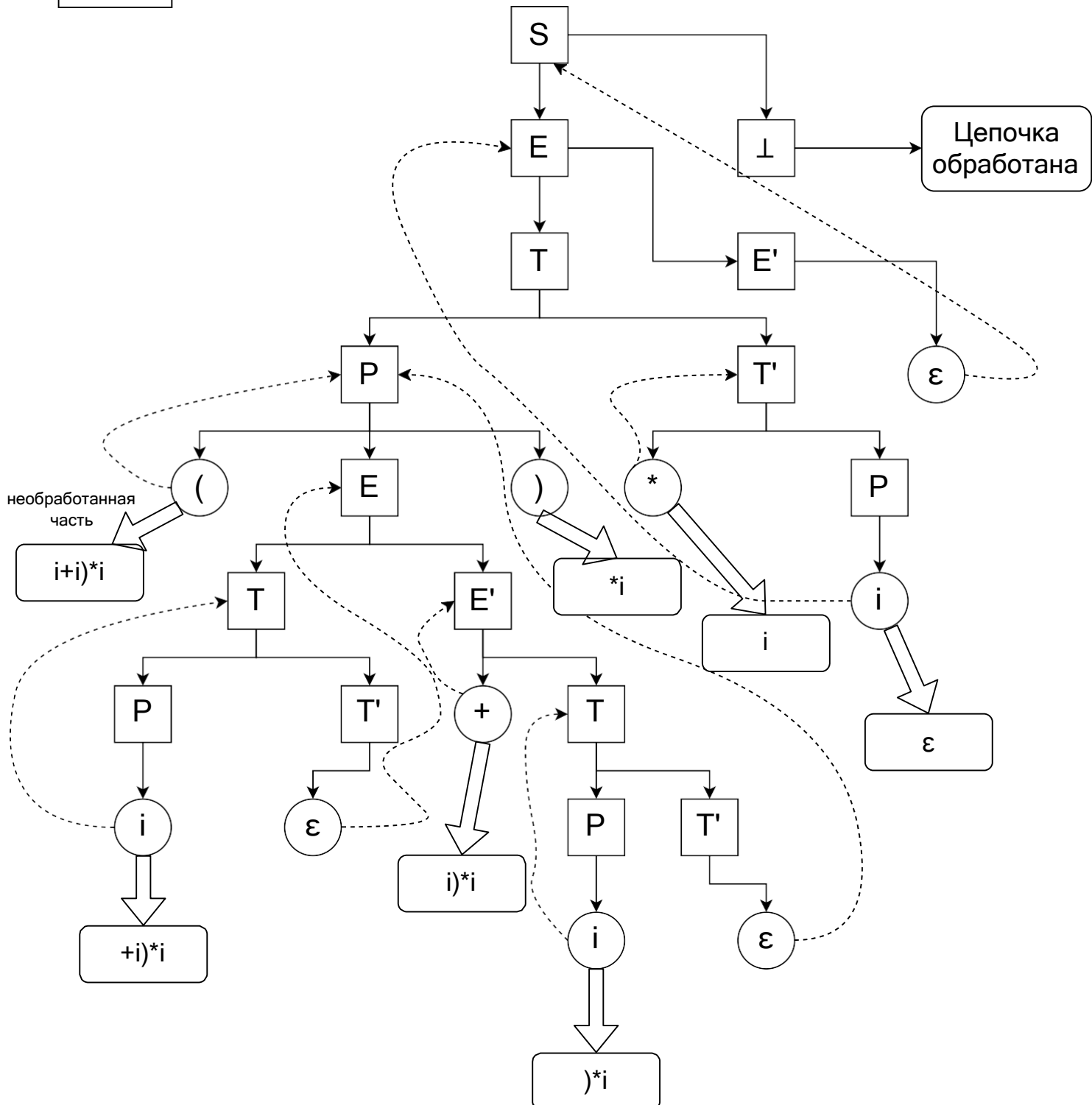
Дальше алгоритм продолжается рекурсивно.

В овальных прямоугольниках хранится оставшаяся часть исследуемой цепочки.

После того, как у нас прошел алгоритм по всему дереву, доходим до последнего узла и сравниваем исследуемую цепочку и узел сентинела. Если цепочка состоит из ε -строки, то цепочка принадлежит грамматике, иначе не принадлежит.

$$(i+i)^*i$$

Разбор цепочки



Для того чтобы процедуры обеспечивали правильную передачу параметров для унаследованных и синтезированных атрибутов, они должны быть написаны на языке программирования, который обеспечивает способы передачи параметров "вызов по значению" (для унаследованных атрибутов) и "вызов по ссылке" (для синтезированных атрибутов).

Замечание. Язык программирования Pascal поддерживает оба метода передачи параметров, а в языке С имеется единственный механизм передачи параметров — "вызов по значению" [36]. Эффект вызова по ссылке в языке С может быть получен, если использовать в качестве параметров указатели.

Поскольку имена атрибутов нетерминальных символов используются в качестве параметров процедур для распознавания этих нетерминалов, при именовании атрибутов необходимо выполнять дополнительное требование, заключающееся в том, что все вхождения некоторого нетерминала в левые части правил вывода АТ-грамматики должны иметь один и тот же список имен атрибутов. Например, в грамматике не может быть таких правил вывода:

$$\begin{aligned} R_{p_1, t_2} &\rightarrow + i_{q_1} \{*\}_{p_2, q_2, r_1} R_{r_1, t_1} \\ R_{p_1, t_2} &\rightarrow + i_{q_1} \{*\}_{p_2, q_2, r_1} R_{r_1, t_1} \\ R_{p_1, p_2} &\rightarrow \varepsilon \end{aligned}$$

т. к. первые два вхождения нетерминала R имеют атрибуты $p_1 t_2$, а последнее вхождение — p_1, p_2 . В этом случае необходимо выбрать какой-то один список имен атрибутов нетерминала R (например, p и t), использовать эти имена при описании типа атрибутов этого нетерминала (например, унаследованный p , синтезированный t) и переименовать его атрибуты соответствующим образом.

Указанные ограничения на имена атрибутов не распространяются на атрибуты символов из правых частей правил вывода грамматики.

После того как атрибуты в левых частях правил и в описании их типов переименованы, для упрощения или исключения некоторых правил вычисления атрибутов можно использовать новое соглашение об обозначениях атрибутов, которое формулируется следующим образом: "Если два атрибута получают одно и то же значение, то им можно дать одно и то же имя при условии, что для этого не нужно изменять имена атрибутов нетерминала в левой части правила вывода".

Рассмотрим несколько примеров.

$$\begin{aligned} A &\rightarrow B_x C_y D_z \\ y, z &\leftarrow x \end{aligned}$$

Атрибутам x , y и z присваивается одно и то же значение, поэтому им можно дать общее имя. Используя новое имя a , получим правило вывода грамматики ($A \rightarrow B_x C_y D_z$), которое не требует правила вычисления атрибута x .

$$\begin{aligned} A_x &\rightarrow B_y \{f\}_z \\ y, z &\leftarrow x \end{aligned}$$

Атрибутам можно дать одно имя, но оно должно быть x , для того чтобы не изменилось имя атрибута в левой части правила. Выполнив замену имен, получим правило вывода грамматики $A_x \rightarrow B_x \{f\}^*$, при этом отпадает необходимость в атрибутном правиле.

$$A_{x, y} \rightarrow a, B_z C_t$$
$$y, z, t \leftarrow x$$

Атрибуты y, z, t, x имеют одно и то же значение, но им нельзя дать одно имя, поскольку нетерминал в левой части правила вывода имеет два атрибута: x и y . В данном случае можно получить лишь частичное упрощение:

$$A_{x, y} \rightarrow a, B_y C_y$$
$$y \leftarrow x$$

Новый способ записи имен атрибутов позволяет непосредственно превращать списки атрибутов нетерминальных символов грамматики в списки параметров процедур для распознавания нетерминальных символов. Такой способ записи имеет недостаток, заключающийся в том, что из него не видно, каким образом между атрибутами передается информация. Например, из правила $A \rightarrow B C_x D_x$ не ясно, присваивается ли атрибут нетерминала C атрибуту нетерминала D или наоборот. Если атрибут нетерминала C присваивается атрибуту нетерминала D , то атрибутное правило является L-атрибутным и может использоваться при реализации перевода методом рекурсивного спуска. В противном случае метод рекурсивного спуска неприменим.

Обратившись к описанию типов атрибутов, можно определить порядок передачи информации между атрибутами (значение синтезированного атрибута должно присваиваться унаследованному атрибуту). Однако на практике более удобно использовать обычный способ именования атрибутов и переходить к новому способу записи только после того, как будет доказано, что исходная АТ-грамматика является L-атрибутной.

Для метода рекурсивного спуска не требуется, чтобы АТ-грамматика, описывающая перевод, имела форму простого присваивания.

Опишем детально, как необходимо расширить метод рекурсивного спуска, чтобы он выполнял атрибутный перевод.

Во-первых, изменим процедуру NextSymbol таким образом, чтобы она читала очередной символ входной цепочки (лексему) и присваивала класс текущего входного символа переменной ClassSymb, а значение лексемы (если оно есть) — переменной ValSymb.

Правила составления процедур при условии, что:

- АТ-грамматика, описывающая перевод, является L-атрибутной;
- левые части правил и описания нетерминалов используют одни и те же имена атрибутов;
- для атрибутов, имеющих одно и то же значение, можно использовать одинаковые имена;
- дополняются следующими правилами:
- формальные параметры. Список имен атрибутов, соответствующий вхождению нетерминала в левые части правил вывода, становится списком формальных параметров соответствующей процедуры;
- спецификации параметров. Спецификации атрибутов (УНАСЛЕДОВАННЫЙ или синтезированный) переводятся в спецификации формальных параметров по следующим правилам:
- тип унаследованный соответствует способу передачи параметров "вызов по значению";

- тип СИНТЕЗИРОВАННЫЙ соответствует способу передачи параметров "вызов по ссылке";
- локальные переменные. Все имена атрибутов символов данного правила грамматики, кроме тех, что связаны с символом из левой части, становятся локальными переменными соответствующей процедуры; обработка нетерминала из правой части правила. Для каждого вызова процедуры, соответствующего вхождению нетерминального символа в правую часть правила вывода, список атрибутов этого вхождения используется в качестве списка фактических параметров;
- обработка входного символа.

Для каждого вхождения входного символа в правую часть правила вывода грамматики перед вызовом процедуры NextSymb в процедуру включается фрагмент кода, который каждой переменной из списка атрибутов входного символа присваивает значение входного атрибута из переменной ValSymb;

1. обработка операционного символа. Для каждого вхождения операционного символа в правую часть правила вывода грамматики в процедуру включается фрагмент кода, который по соответствующим атрибутивным правилам вычисляет значения синтезированных атрибутов операционного символа и присваивает вычисленные значения переменным, соответствующим синтезированным атрибутам. Затем вызывается процедура выдачи операционного символа вместе с атрибутами в выходную строку;
2. обработка правил вычисления атрибутов. Для каждого правила вычисления атрибутов, сопоставленного правилу вывода грамматики, в процедуру включается фрагмент кода, который вычисляет значение атрибута и присваивает это значение каждой переменной из левой части атрибутивного правила (если соглашение об одинаковых именах выполнено, то в левой части атрибутивного правила будет только один атрибут). Фрагмент кода можно поместить в любом месте процедуры, которое находится:
 - после точки, где используемые в правиле атрибуты уже вычислены;
 - перед точкой, где впервые используется вычисленное значение атрибута;
 - головной модуль. Все имена синтезированных атрибутов начального символа грамматики становятся локальными переменными головного модуля. Список фактических параметров вызова процедуры распознавания начального символа грамматики содержит начальные значения унаследованных атрибутов и имена синтезированных атрибутов.

Пример 7.1. Рассмотрим L-атрибутную транслирующую грамматику в качестве примера реализации атрибутивного перевода с использованием метода рекурсивного спуска.

Для повышения наглядности переименуем атрибуты символов грамматики таким образом, чтобы всем вхождениям символов в правые части разных правил вывода соответствовали разные имена атрибутов, а также выберем одинаковые имена для атрибутов нетерминала R из левой части правил вывода с номерами (3), (4) и (5). Пусть p — унаследованный атрибут нетерминала R, а t — его синтезированный атрибут. После преобразования получим следующую грамматику:

Et синтезированный t

R_{p, t} унаследованный p синтезированный t

Атрибуты операционных символов унаследованные.

(0) S₀ → S ⊥

(1) S → I_a := E_b {:=} a, b

(2) E_c → I_d R_{d, c}

(3) R_{p, t} → I_{q1} {+} p, q₁, r₁ R_{r1, t}
r₁ ← GETNEW

(4) R_{p, t} → X I_{q2} {*} p, q₁, r₁ R_{r1, t}
r₂ ← GETNEW

(5) R_{p, t} → ε
t ← p

++ Кр: Атрибутный перевод операторов присваивания некоторого языка программирования в цепочку тетрад с кодами операций: СЛОЖИТЬ,

С учетом выполненных преобразований процедура **на языке Pascal**, реализующая атрибутный перевод операторов присваивания некоторого языка программирования в цепочку тетрад с кодами операций: СЛОЖИТЬ, УМНОЖИТЬ, ПРИСВОИТЬ методом рекурсивного спуска, приведена в листинге 7.2.2.

Листинг 7.2.2

Procedure Recurs_Method_ATG (List-Token: tList, List-Tetr: tListTetr);
{List-Token -входная цепочка, List-Tetr - цепочка тетрад}

```
Procedure Proc_S;  
begin  
  if ClassSymb = ClassId {текущий символ - идентификатор} then  
  begin  
    a := ValSymb;  
    NextSymb;  
    if ClassSymb = ':= ' then  
    begin  
      NextSymb;  
      Proc_E(b);  
      Output({:=}, a, b)  
    end  
  else  
    Error  
  end  
  else  
    Error  
end;  
{Proc_S}  
Procedure Proc_E (var c: integer);  
var d: integer; {локальная переменная Proc_E}
```

```

begin
if ClassSymb = ClassId {текущий символ - идентификатор} then
begin
d := ValSymb;
NextSymb;
Proc_R(d,c)
end
else
Error
end; {Proc_E}
Procedure Proc_R (p: integer, var t: integer);
Var q1, q2, r1, r2: integer; {локальные переменные Proc_R}
begin
  if ClassSymb = '+' then
    begin
      NextSymb;
      if ClassSymb = ClassId (текущий символ – идентификатор) then
        begin
          q2 := ValSymb;
          NextSymb;
          GetNew(r2);
          Output({+}, p, q2, r2);
          Proc_R (r2, t)
        end
      else
        Error;
      end
    else
      t := p
    end; {Proc_R}
  begin
    if ClassSymb = '*' then begin
      { В начале анализа переменная ClassSymb содержит класс первого символа
      входной цепочки, а переменная ValSymb - значение этого символа}
      Proc_S;
      if ClassSymb = '┐' then
        Access
      else
        Error
      end {Recurs_Method_ATG};
    end
  end

```

На рис. 1.3 приведен список имен процедур со значениями фактических параметров в порядке их вызова при переводе входной цепочки $i_5 = i_1 + i_2 * i_3 \perp$ в цепочку тетрад. Справа в строке, соответствующей вызову процедуры NextSymb, изображена непрочитанная часть входной цепочки (текущий символ находится слева) непосредственно после вызова этой процедуры.

Алгоритм построения дерева разбора входной цепочки по транслирующей грамматике.

Задача: разработать и реализовать алгоритм построения дерева разбора входной цепочки по транслирующей грамматике.

Входные данные: транслирующая грамматика.

Выходные данные: дерева разбора.

Для начала, построим по транслирующей грамматике управляющую таблицу. Управляющая таблица M определяется на множестве $(N \cup \sum i \cup \sum a \cup \{\perp\}) \times \sum i \cup \{\varepsilon\}$ по следующим правилам:

- 1) Если $A \rightarrow u\beta$ - правило вывода грамматики G^T , где $u \in \sum a^*$, а β — либо ε , либо цепочка, начинающаяся с терминала или нетерминала, то $M\{A, a\} = (\beta, u)$ для всех $a \neq \varepsilon$, принадлежащих множеству $FIRST(\beta)$.
- 2) Если $\varepsilon \in FIRST(\beta)$, то $M(A, b) = (\beta, u)$ для всех $b \in FOLLOW(A)$.

Заметим, что при вычислении $FIRST(\beta)$ операционные символы, входящие в цепочку β , вычеркиваются.

$M(X, a) = ВЫДАЧА(X)$ для всех $x \in \sum a$ И $a \in \sum i \cup \{\varepsilon\}$.

$M(a, a) = ВЫБРОС$ для всех $a \in \sum i$.

$M(\perp, \varepsilon) = ДОПУСК$

В остальных случаях $M(X, a) = ОШИБКА$ для $X \in (N \cup \sum i \cup \sum a \cup \{\perp\})$ и $a \in \sum i \cup \{\varepsilon\}$.

Затем, по управляющей таблице проведем разбор входной цепочки и построим дерево разбора. В дереве разбора будут содержаться только нетерминальные символы и операционные символы. Дерево разбора строится во время разбора выражения, в зависимости от того, какое правило содержится в ячейке управляющей таблицы. Затем, элементы этого правила добавляются в детей от предшествующей вершины.

Для текущего символа входной цепочки $a \in \sum i$ и текущего символа стека $b \in (N \cup \sum i \cup \sum a \cup \{\perp\})$ определяем правило $P \in V$, соответствующее значению ячейки управляющей таблицы $M(b, a)$. Затем, определяем набор символов, входящих в правило P . Если текущий символ ($c \in P$) $\in (N \cup \sum a)$, добавляем его в дерево разбора как вершину дерева.

1 часть – построение управляющей таблицы

Вход: транслирующая грамматика $G^T = (V, \sum i, \sum a, P, S)$.

Выход: управляющая таблица (table)

table = $\emptyset$

$T = \sum i \in G$

$O = \sum a \in G$

$T = T \cup \varepsilon$

$P = P \in G$

foreach ($t \in T$)

 table(" ", t) = table(" ", t) $\cup t$

end foreach

$V = V \in G$

```

foreach (v ∈ V)
    table(v,"")=table(v,"") ∪ v.
    foreach (p ∈ P)
        currentFirstSet = ∅
        pWithoutOperSymb = ∅
        pWithoutOperSymb = pWithoutOperSymb ∪ p \ O
        currentFirstSet = currentFirstSet ∪ FIRST(pWithoutOperSymb)
        foreach( firstSymbol ∈ currentFirstSet)
            if (firstSymbol ∉ ε)
                table(v,firstSymbol) = table(v,firstSymbol) ∪ p
            end if
            else
                currentFollowSet = ∅
                currentFollowSet = currentFollowSet ∪ FOLLOW(p)
                foreach(followSymbol ∈ currentFollowSet)
                    table(v, followSymbol) = table(v,followSymbol) ∪ p
                end foreach
            end else
        end foreach
    end foreach
end foreach
H = O ∪ { ⊥ } ∪ T \ { ε }
foreach (h ∈ H)
    table(h,"") =table(h,"") ∪ h
    if (h ∈ O)
        foreach(symbols ∈ T)
            table(h,symbol) = ВЫДАЧА({h})
        end foreach
    end if
    else
        if (h ∈ {⊥})
            table(h, {ε}) = ДОПУСК
        endif
        else
            table(h,h)= ВЫБРОС
        end else
    end else
end foreach

```

2 часть – построение дерева разбора

Вход: управляющая таблица (table), входная цепочка (inputChain)

Выход: дерево разбора входной цепочки в виде массива (treeArray)

table

inputChain

treeArray = ∅

```

treeNode =  $\emptyset$ 
stack =  $\emptyset$ 
stack = stack  $\cup$  E  $\cup$   $\perp$ 
treeNode = treeNode  $\cup$  E
treeArray = treeArray  $\cup$  treeNode
i=0
while (stack  $\notin$  { $\perp$ } ) do
    rule =  $\emptyset$ 
    rule = rule  $\cup$  table(pop(stack),inputChain(i))
    if (rule  $\in$  {«ДОПУСК», «ВЫБРОС»,«ВЫДАЧА({i})»,«ВЫДАЧА
    ({+})»,«ВЫДАЧА({*})»})
        i++
    end if
    else if (rule  $\notin$  {ОШИБКА}):
        stack = stack  $\cup$  rule
        rule = rule / T
        foreach (ruleSymbol  $\in$  rule)
            treeNode = treeNode  $\cup$  ruleSymbol
            treeArray = treeArray  $\cup$  treeNode
        end foreach
    end else if
end
end

```


Дерево разбора было построено для следующей транслирующей грамматики:
 $G^T = (V, \Sigma_i, \Sigma_a, P, S)$, $V \in \{E, T, E', P, T'\}$, $\Sigma_i \in \{+, *, i\}$, $\Sigma_a \in \{+, *, i\}$, $S = E$, $P \in \{$

1) $E \rightarrow TE'$

2) $T \rightarrow PE'$

3) $E' \rightarrow +P\{+\}T'$

4) $T' \rightarrow \varepsilon$

5) $E' \rightarrow \varepsilon$

6) $T' \rightarrow *P\{*\}T'$

7) $P \rightarrow i\{i\}$

}

И для следующей входной цепочки: $i+i*i$.

Полученная в ходе выполнения алгоритма управляющая таблица:

	i	+	*	ε
E	TE'			
E'		$+P\{+\}T'$		ε
T	PE'			
T'			$*P\{*\}T'$	ε
P	$i\{i\}$			
i	ВЫБРОС			
+		ВЫБРОС		
*			ВЫБРОС	
$\perp$				ДОПУСК
{i}	ВЫДАЧА({i})			
{+}	ВЫДАЧА({+})			
{*}	ВЫДАЧА({*})			

Продemonстрируем пару шагов алгоритма построения дерева разбора входной цепочки по управляющей таблице.

В начале алгоритма в стеке лежит $E\perp$, первый символ входной цепочки – i .

E
$\perp$

$i+i*i$

Стек Входная цепочка

Добавляем корень дерева – нода с символом E .



Извлекаем верхний символ стека – E .

E
$\perp$

Этому набору данных соответствует ячейке $M(E, i) - TE'$.
Заносим эти символы в вершину стека.

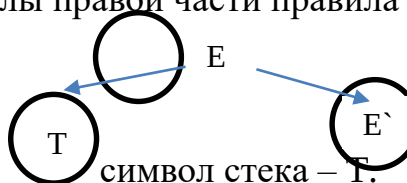
	i
E	TE'

Ячейка
управляющей
таблицы

T
E'
$\perp$

Так как в правой части правила отсутствуют терминальные символы, то добавляем все символы правой части правила в дерево потомков ноды E .

E



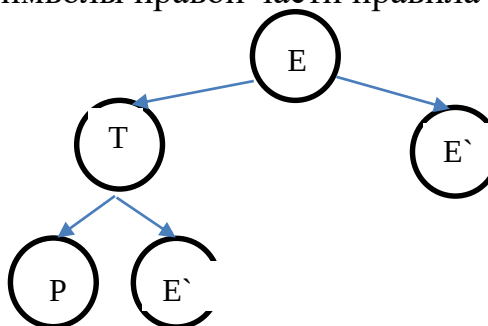
Извлекаем верхний

T
E'
$\perp$

Распознанный символ цепочки все еще i . Этому набору данных соответствует ячейке $M(T, i) - PE'$. Заносим эти символы в вершину стека.

P
E'
E'
$\perp$

Так как в правой части правила отсутствуют терминальные символы, то добавляем все символы правой части правила в дерево потомков ноды T .



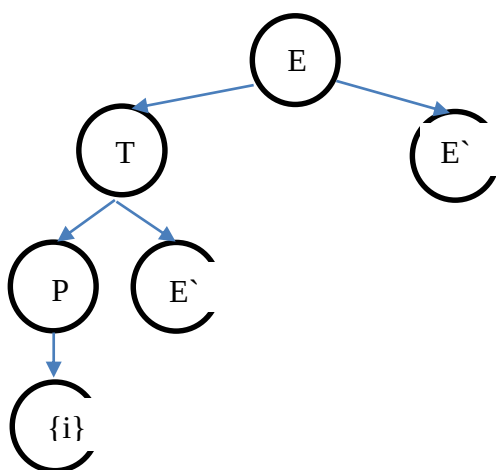
Извлекаем верхний символ стека – P .

P
E`
E`
⊥

Распознанный символ цепочки все еще i . Этому набору данных соответствует ячейке $M(P, i) - i \{i\}$. Заносим эти символы в вершину стека.

i
$\{i\}$
E`
E`
⊥

Так как в правой части правила присутствуют терминальные символы, то удаляем их из правой части правила и добавляем их дерево в потомков ноды P.



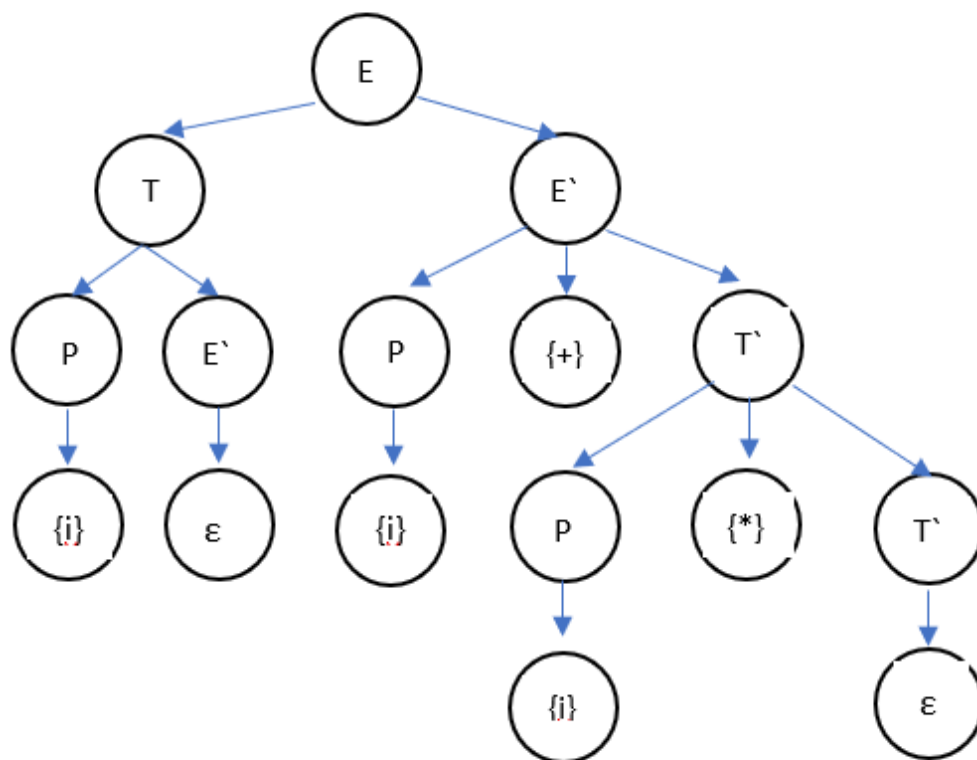
Далее, извлекаем верхний символ стека – i . Распознанный символ цепочки все еще i . Этому набору данных соответствует ячейке $M(i, i) - \text{ВЫБРОС}$.

Переходим к распознаванию следующего символа входной цепочки – $+$.

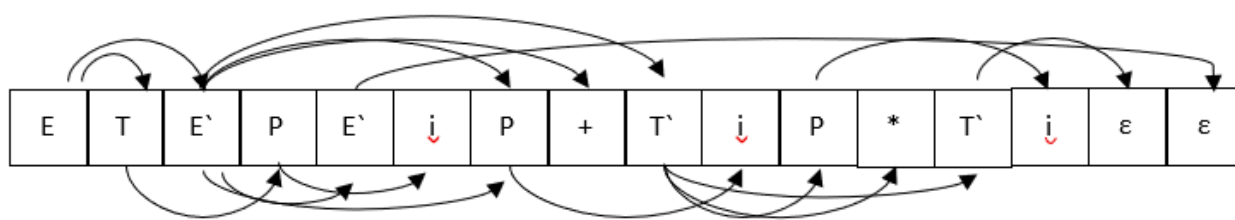
$i+i*i$

Далее, продолжаем выполнение действий алгоритма до тех пор, пока не достигнем маркера дна стека ($\perp$), или пока не встретим в ячейке ОШИБКА.

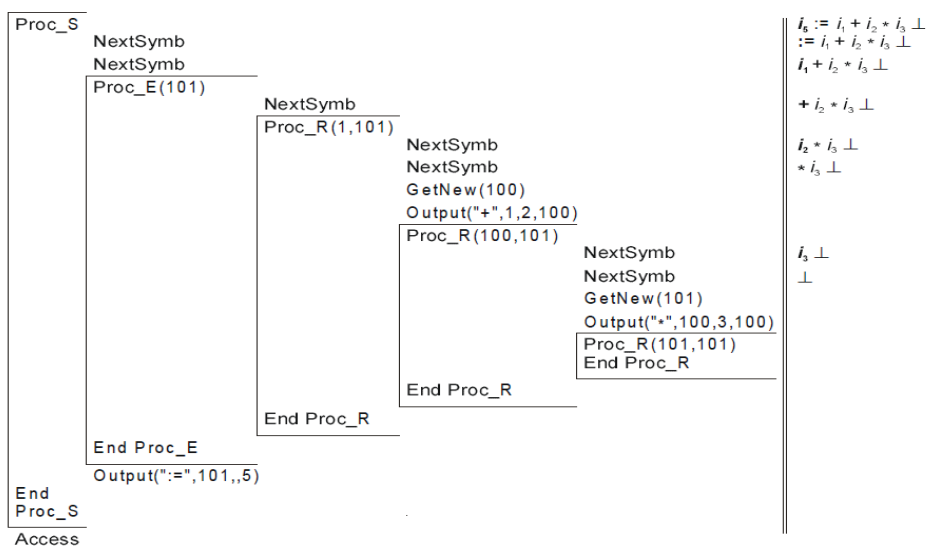
Полученное в ходе выполнения алгоритма разбора входной цепочки дерево:



Полученный в ходе разбора цепочки массив вершин дерева:



S-атрибутный ДМП-процессор



КР Математическая модель восходящего ДМП-процессора

Для любого восходящего синтаксического анализатора, последовательность операций **переноса и свертки**, выполняемых при обработке допустимых входных цепочек, можно описать с помощью транслирующей грамматики.

+++ end

Входной для этой транслирующей грамматики является грамматика, на основе которой построен анализатор. Для получения транслирующей грамматики в самую крайнюю правую позицию каждого i -го правила входной грамматики вставляется операционный символ $\{\text{СВЕРТКА}, i\}$. Например, транслирующая грамматика, построенная по входной грамматике $G_q = (\{E, T \setminus P\}, \{i, +, *, (,)\}, P, E)$, где $P = \{E \rightarrow E + T, E \rightarrow T, T \rightarrow T * P, P, P \rightarrow i, P \rightarrow (E)\}$, будет выглядеть следующим образом:

$E \rightarrow E + T \{\text{СВЕРТКА}, 1\}$

$E \rightarrow T \{\text{СВЕРТКА}, 2\}$

$T \rightarrow T * P \{\text{СВЕРТКА}, 3\}$

$T \rightarrow P \{\text{СВЕРТКА}, 4\}$

$P \rightarrow i \{\text{СВЕРТКА}, 5\}$

$P \rightarrow (E) \{\text{СВЕРТКА}, 6\}$

Если каждый входной символ в активной цепочке интерпретировать как представление операции ПЕРЕНОС, выполняемой в момент времени, когда этот символ является текущим входным символом, то активная цепочка в точности описывает последовательность операций переноса и свертки, выполняемых при обработке входной цепочки. Каждая операция свертки выполняется сразу же после того, как локализована соответствующая основа (или первичная фраза), т. е. когда завершается обработка последнего символа в правой части правила вывода. Например, для входной цепочки $i + i * i$, разбор которой рассматривался в разд. 9.4, активная цепочка, порождаемая рассмотренной ранее транслирующей грамматикой, имеет вид:

$i \{\text{СВЕРТКА}_6\} + i \{\text{СВЕРТКА}_6\} * i \{\text{СВЕРТКА}_6\} \{\text{СВЕРТКА}_3\} \{\text{СВЕРТКА}_1\}$,

что полностью соответствует последовательности операций переноса и свертки, выполняемых при обработке входной цепочки.

Восходящий анализатор можно расширить действиями по выполнению перевода, если перевод определяется постфиксной транслирующей грамматикой. Модификация анализатора заключается в том, что операция свертки расширяется действиями, определяемыми операционными символами соответствующего правила грамматики. Это можно сделать, т. к. при обработке входной цепочки момент времени выполнения свертки для каждого правила грамматики совпадает с моментом выполнения действий по переводу для этого правила. Например, для постфиксной транслирующей грамматики цепочечного перевода:

$E \rightarrow E + T \{+\}$

$E \rightarrow T$

$T \rightarrow T * P \{*\}$

$T \rightarrow P$

$P \rightarrow i\{i\}$

$P \rightarrow (E)$

операции свертки расширяются следующим образом: СВЕРТКА_1 будет обеспечивать выдачу операционного символа $\{+\}$ в выходную строку, СВЕРТКА 3 - выдачу операционного символа $\{*\}$, а СВЕРТКА 6 — выдачу операционного символа $\{i\}$.

Определение. Восходящий ДМП-процессор - это синтаксический анализатор, дополненный формальными действиями по выполнению перевода.

7.5. Реализация S-атрибутного ДМП-процессора

Задача. Преобразовать восходящий ДМП-процессор в S-атрибутный ДМП-процессор. Каждый магазинный символ будет представляться как объект, хранящий атрибуты.

Теоретическая часть.

На вход программы подаётся уже готовый восходящий ДМП-процессор для S-атрибутной АТ-грамматики. Обозначим свойства этой грамматики.

На любую S-атрибутную грамматику накладывается три условия:

1) Она является L-атрибутной:

- а. Аргументами правила вычисления значения унаследованного атрибута символа из правой части правила вывода могут быть только унаследованные атрибуты символа из левой части и произвольные атрибуты символов из правой части, расположенные левее рассматриваемого символа.
- б. Аргументами правила вычисления значения синтезированного атрибута символа из левой части правила вывода являются унаследованные атрибуты этого символа или произвольные атрибуты символов из правой части.
- с. Аргументами правила вычисления значения синтезированного атрибута операционного символа могут быть только унаследованные атрибуты этого символа.

2) Все атрибуты нетерминалов - синтезированные

Изначально для АТ-грамматики G^{AT} строится восходящий ДМП-процессор. Сначала её сделаем пополненной, обозначив как G' , разделяя для каждого символа $\alpha \in NU \sum_i$ индексы вхождений i, j и атрибуты $a_1 \dots a_n$ символом «|» ($\alpha_{ij|a_1, \dots, a_n}$).

Затем строится управляющая таблица M для данной грамматики без учёта её атрибутов и операционных символов, т.е. как для LR(k) грамматики. Для атрибутной грамматики её нужно модифицировать, дополнив символы v_{ij} матрицы переходов $g(x)$ их атрибутами $a_1 \dots a_n$ таким образом: $v_{ij|a_1, \dots, a_n}$.

Алгоритм модификации управляющей таблицы М:

Вход: управляющая таблица М, дополненная атрибутивная грамматика

$G' = \{N, \sum_i, \sum_a, P, S\}$, Γ - множество грамматических вхождений.

Выход: модифицированная управляющая таблица M' , матрица переходов $g(x)$

```
foreach ( $X_{ij|a_1, \dots, a_n} \in \Gamma$ )
    foreach ( $Y_{kl|b_1, \dots, b_n} \in \Gamma \mid g[X_{ij}, \text{symbol}(Y_{kl|b_1, \dots, b_n})] \neq \emptyset$ )
         $g[X_{ij}, \text{symbol}(Y_{kl})] := Y_{kl|b_1, \dots, b_n}$  //дополняем символ атрибутами  $b_1, \dots, b_n$ 
    end
end
```

В отличие от ДМП-автомата для КС-грамматики, S-атрибутивная АТ-грамматика требует хранить в магазине вместе с символами значения их атрибутов. Для этого изменим структуру магазина, теперь он будет хранить объекты $O_1, \dots, O_n$ символов грамматического вхождения. Каждый объект O состоит из поля имени символа X_{ij} и поля множества n пар атрибута a_k и его значения v_k . В общем виде $O = \{X_{ij} \in \Gamma, A = \{ \langle a_k, v_k \rangle \mid k = 1, \dots, n \} \}$ В дальнейшем будем обозначать создание нового объекта как: $\text{ATSymbol}(\alpha_{ij|a_1, \dots, a_n}, v_1, \dots, v_n) = \{X_{ij} \in \Gamma, A = \{ \langle a_k, v_k \rangle \mid k = 1, \dots, n \} \}$

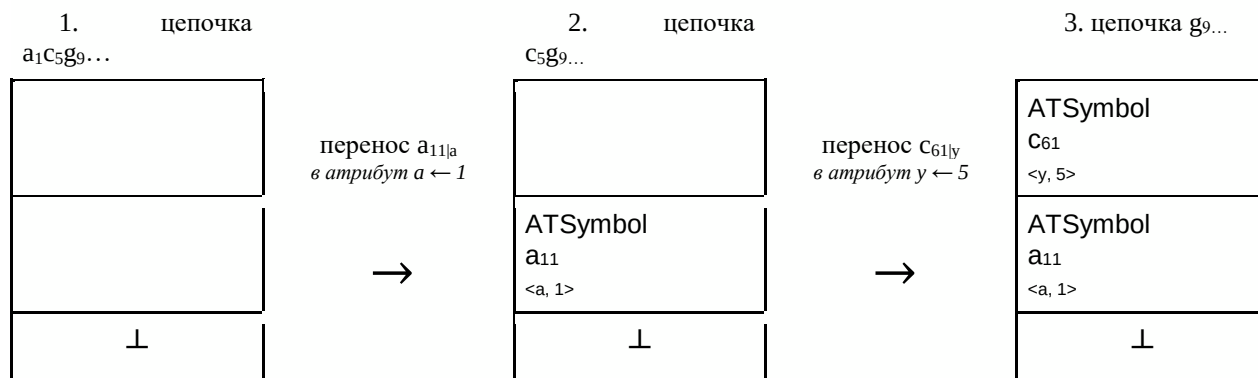
Пример. Для символа $b_{11|p, f}$ при считывании из входной строки символа $b_{4,5}$ в магазин будет вставлен объект $O = \{b_{11}, A = \{ \langle p, 4 \rangle, \langle f, 5 \rangle \} \}$. Значения атрибутов 4, 5 создали с именами атрибутов p, f массив пар A соответственно. В поле имени символа вставлен $b_{11|p, f}$ без имён атрибутов.

Следующим шагом является переопределение операции ПЕРЕНОС и СВЁРТКА в восходящем ДМП-автомате. В операции переноса по управляющей таблице M определяется какой символ $\alpha_{ij|a_1, \dots, a_n}$ вталкивается в магазин и какие атрибуты $a_1 \dots a_n$ у него есть. Создаём объект $\text{ATSymbol}(\alpha_{ij|a_1, \dots, a_n}, v_1, \dots, v_n)$ данного символа и поля атрибутов заполняем соответствующими значениями $v_1, \dots, v_n$ считанного из входной строки символа $\square_{v_1, \dots, v_n}$. Только потом мы вставляем объект ATSymbol в магазин и не проводим никаких операций над его атрибутами, пока не будет произведена СВЁРТКА.

Для операции СВЁРТКА будет проведена следующая модификация. Атрибуты удаляемых из магазина объектов и их значения расширенная операция использует для вычисления всех атрибутов операционных символов и для атрибутов нетерминального символа из левой части грамматики. Вычисленные атрибуты нетерминального символа будут записаны в поля объекта, соответствующего нетерминальному символу, который будет вставлен в магазин. После вычисления атрибутов операционных символов, создаётся объект операционного символа с заполненными значениями атрибутов и он выводится в выходную ленту. Чтобы это реализовать, каждое правило вывода имеет набор правил вычислений атрибутов. Проводя операцию СВЁРТКА по i -ому правилу, мы также получаем набор правил вычислений атрибутов, которые мы должны произвести над атрибутами.

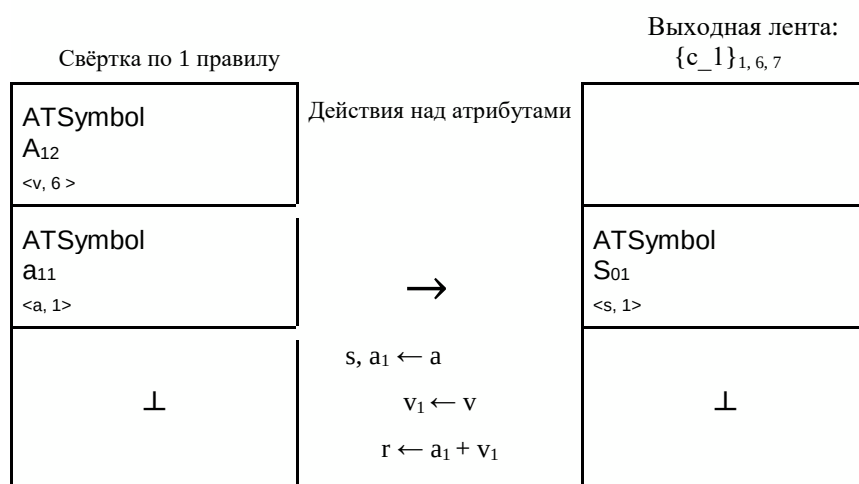
Пример операций ПЕРЕНОС и СВЁРТКА:

ПЕРЕНОС



При переносе мы считывали символ и значения его атрибутов и по таблице М вставляли соответствующий объект символа с заполненными парами <атрибут, значение>

СВЁРТКА



При выполнении операции СВЁРТКА мы берём n верхних объектов в магазине (n - длина правой цепочки правила), создаём объект терминала из левой части правила по таблице М и по правилам рассчитываем его атрибуты. В данном случае атрибут v символа S_{01} был скопирован из атрибута a объекта a_{11} . Если в изначальной грамматике есть операционный символ в правиле, то мы на выходную ленту выдаём объект этого символа с так же посчитанными атрибутами. Например атрибут r был синтезирован из атрибутов a и v , после чего в выходную ленту был выведен операционный символ $\{c_1\}_{1,6,7}$.

Пример. Пусть определена входная S-атрибутная АТ-грамматика

$$G^{AT} = \{N, \sum_i, \sum_a, P, S\} =$$

$\{$
 $N = \{S, A, B, C, D, K, C'\},$
 $\Sigma_i = \{a, b, c, d, f, g, \},$
 $\Sigma_a = \{\{c_1\}, \{c_2\}, \{c_3\}, \{c_4\}, \{c_5\}, \{c_6\}, \{c_7\}\},$
 $P = \{$
1. $S_s \rightarrow a_a A_v \{c_1\}_{a1, v1, r}$
 $s, a_1 \leftarrow a$
 $v_1 \leftarrow v$
 $r \leftarrow a_1$ (синтезирован из унаследованных)
2. $A_v \rightarrow B_b C_c D_d K_k \{c_2\}_{b1, c1, d1, k1}$
 $v \leftarrow b$
 $c \leftarrow d$
 $b_1 \leftarrow b$
 $c_1 \leftarrow c$
 $d_1 \leftarrow d$
 $k_1 \leftarrow k$
3. $C_c \rightarrow d_q C'_g \{c_3\}_{q1, g1}$
 $c \leftarrow g$
 $q_1 \leftarrow q$
 $g_1 \leftarrow g$
4. $D_d \rightarrow g_l \{c_4\}_{l1}$
 $d \leftarrow l$
 $l_1 \leftarrow l$
5. $C'_g \leftarrow f_f B_b \{c_5\}_{f1, b1}$
 $g \leftarrow f$
 $f_1 \leftarrow f$
 $b_1 \leftarrow b$
6. $B_b \rightarrow c_y D_d \{c_6\}_{y1, d1}$
 $b \leftarrow d$
 $y_1 \leftarrow y$
 $d_1 \leftarrow d$
7. $K_k \leftarrow b_z \{c_7\}_{z1}$
 $k \leftarrow z$
 $z_1 \leftarrow z$
 $\}$
 $\}$

Шаг 1. Создание пополненной грамматики.

Для данной грамматики была выведена пополненная грамматика G' с правилами P' .

$P' = \{$

0. $S'_0 \rightarrow S_{01|s}$

1. $S_{1|s} \rightarrow a_{11|a} A_{12|v} \{c_1\}_{a1, v1, r}$

2. $A_{2|v} \rightarrow B_{21|b} C_{22|c} D_{23|d} K_{24|k} \{c_2\}_{b1, c1, d1, k1}$

3. $C_{3|c} \rightarrow d_{31|q} C'_{32|g} \{c_3\}_{q1, g1}$

4. $D_{4|d} \rightarrow g_{41|l} \{c_4\}_{l1}$

5. $C'_{5|g} \rightarrow f_{51|f} B_{52|b} \{c_5\}_{f1, b1}$

6. $B_{6|b} \rightarrow c_{61|y} D_{62|d} \{c_6\}_{y1, d1}$

7. $K_{7|k} \rightarrow b_{71|z} \{c_7\}_{z1}$

$\}$

Знак разделителя обозначает - вхождения грамматики | атрибуты символов.

Шаг 2. Создание управляющей таблицы M и её модификация до M' .

Модифицированная управляющая таблица M' :

	F(u)							g(x)												
	a	c	d	g	b	f	ε	S	a	A	c	B	C	d	D	g	K	b	C`	f
⊥	Π							S _{01 s}	a _{11 a}											
S ₀₁							Д													
a ₁₁		Π								A _{12 v}	c _{61 y}	B _{21 b}								
A ₁₂	C, 1	C, 1	C, 1	C, 1	C, 1	C, 1	C, 1													
B ₂₁			Π										C _{22 c}	d _{31 q}						
C ₂₂				Π											D _{23 d}	g _{41 l}				
D ₂₃					Π												K _{24 k}	b _{71 z}		
K ₂₄	C, 2	C, 2	C, 2	C, 2	C, 2	C, 2	C, 2													
d ₃₁						Π													C` _{32 g}	f _{51 f}
C` _{3 2}	C, 3	C, 3	C, 3	C, 3	C, 3	C, 3	C, 3													
g ₄₁	C, 4	C, 4	C, 4	C, 4	C, 4	C, 4	C, 4													
f ₅₁		Π									c _{61 y}	B _{52 b}								

B ₅₂	C, 5	C, 5	C, 5	C, 5	C, 5	C, 5	C, 5											
C ₆₁				П										D ₆₂ d	g ₄₁			
D ₆₂	C, 6	C, 6	C, 6	C, 6	C, 6	C, 6	C, 6											
b ₇₁	C, 7	C, 7	C, 7	C, 7	C, 7	C, 7	C, 7											

Пример. Такты автомата для грамматики G^{AT} и цепочки $a_4c_8g_3d_7f_1c_2g_8g_9b_2$:

$(\perp, a_4c_8g_3d_7f_1c_2g_8g_9b_2, \varepsilon) \vdash \Pi$

$(\perp_{a_{11|4}}, c_8g_3d_7f_1c_2g_8g_9b_2, \varepsilon) \vdash \Pi$

$(\perp_{a_{11|4}} c_{61|8}, g_3d_7f_1c_2g_8g_9b_2, \varepsilon) \vdash \Pi$

$(\perp_{a_{11|4}} c_{61|8} g_{41|3}, d_7f_1c_2g_8g_9b_2, \varepsilon) \vdash c$

$(\perp_{a_{11|4}} c_{61|8} D_{62|3}, d_7f_1c_2g_8g_9b_2, \{c_4\}_3) \vdash c$

$(\perp_{a_{11|4}} B_{21|3}, d_7f_1c_2g_8g_9b_2, \{c_4\}_3 \{c_6\}_{8,3}) \vdash \Pi$

$(\perp_{a_{11|4}} B_{21|3} d_{31|7}, f_1c_2g_8g_9b_2, \{c_4\}_3 \{c_6\}_{8,3}) \vdash \Pi$

$(\perp_{a_{11|4}} B_{21|3} d_{31|7} f_{51|1}, c_2g_8g_9b_2, \{c_4\}_3 \{c_6\}_{8,3}) \vdash \Pi$

$(\perp_{a_{11|4}} B_{21|3} d_{31|7} f_{51|1} c_{61|2}, g_8g_9b_2, \{c_4\}_3 \{c_6\}_{8,3}) \vdash \Pi$

$(\perp_{a_{11|4}} B_{21|3} d_{31|7} f_{51|1} c_{61|2} g_{41|8}, g_9b_2, \{c_4\}_3 \{c_6\}_{8,3}) \vdash c$

$(\perp_{a_{11|4}} B_{21|3} d_{31|7} f_{51|1} c_{61|2} D_{62|8}, g_9b_2, \{c_4\}_3 \{c_6\}_{8,3} \{c_4\}_8) \vdash c$

$(\perp_{a_{11|4}} B_{21|3} d_{31|7} f_{51|1} B_{52|8}, g_9b_2, \{c_4\}_3 \{c_6\}_{8,3} \{c_4\}_8 \{c_6\}_{2,8}) \vdash c$

$(\perp_{a_{11|4}} B_{21|3} d_{31|7} C'_{32|1}, g_9b_2, \{c_4\}_3 \{c_6\}_{8,3} \{c_4\}_8 \{c_6\}_{2,8} \{c_5\}_{1,8}) \vdash c$

$(\perp_{a_{11|4}} B_{21|3} C_{22|1}, g_9b_2, \{c_4\}_3 \{c_6\}_{8,3} \{c_4\}_8 \{c_6\}_{2,8} \{c_5\}_{1,8} \{c_3\}_{7,1}) \vdash \Pi$

$(\perp_{a_{11|4}} B_{21|3} C_{22|1} g_{41|9}, b_2, \{c_4\}_3 \{c_6\}_{8,3} \{c_4\}_8 \{c_6\}_{2,8} \{c_5\}_{1,8} \{c_3\}_{7,1}) \vdash c$

$(\perp_{a_{11|4}} B_{21|3} C_{22|1} D_{23|9}, b_2, \{c_4\}_3 \{c_6\}_{8,3} \{c_4\}_8 \{c_6\}_{2,8} \{c_5\}_{1,8} \{c_3\}_{7,1} \{c_4\}_9) \vdash \Pi$

$(\perp_{a_{11|4}} B_{21|3} C_{22|1} D_{23|9} b_{71|2}, \varepsilon, \{c_4\}_3 \{c_6\}_{8,3} \{c_4\}_8 \{c_6\}_{2,8} \{c_5\}_{1,8} \{c_3\}_{7,1} \{c_4\}_9) \vdash c$

$(\perp_{a_{11|4}} B_{21|3} C_{22|1} D_{23|9} K_{24|2}, \varepsilon, \{c_4\}_3 \{c_6\}_{8,3} \{c_4\}_8 \{c_6\}_{2,8} \{c_5\}_{1,8} \{c_3\}_{7,1} \{c_4\}_9 \{c_7\}_2) \vdash c$

$(\perp_{a_{11|4}} A_{12|3}, \varepsilon, \{c_4\}_3 \{c_6\}_{8,3} \{c_4\}_8 \{c_6\}_{2,8} \{c_5\}_{1,8} \{c_3\}_{7,1} \{c_4\}_9 \{c_7\}_2 \{c_2\}_{3,1,9,2}) \vdash c$

$(\perp_{S_{01|4}}, \varepsilon, \{c_4\}_3 \{c_6\}_{8,3} \{c_4\}_8 \{c_6\}_{2,8} \{c_5\}_{1,8} \{c_3\}_{7,1} \{c_4\}_9 \{c_7\}_2 \{c_2\}_{3,1,9,2})$

Библиографический список

1. Ахо А., Ульман Д. Теория синтаксического анализа перевода и компиляции // Пер. с англ. - М.: Мир, 1978.
2. Ульман, Джеффри, Д. Компиляторы: Принципы, технологии, инструменты. Пер. с англ. — М.: Издательский дом "Вильямс", 2008 и 2003.
3. Р. Хантер. Проектирование и конструирование компиляторов. Пер. с англ. — М.: Финансы и статистика, 1984.
4. Н. Вирт. Алгоритмы + структуры данных = программы. Пер. с англ. — М.: МИР, 1985.
5. Д. Грис. Конструирование компиляторов для цифровых вычислительных машин. — М.: Мир, 1975.
6. Ф. Льюис, Д. Розенкранц, Р. Стирнз. Теоретические основы проектирования компиляторов. — М.: Мир, 1979. А. Ахо, Дж. Ульман. Теория синтаксического анализа, перевода и компиляции. — Т.1,2. — М.: Мир, 1979.
7. Ф. Вайнгартен. Трансляция языков программирования. — М.: Мир, 1977.
8. А. Ахо., Р. Сети, Дж. Ульман. Компиляторы: принципы, технологии, инструменты. — М.: «Вильямс», 2001.
9. А. Ахо, М. Лам, Р. Сети, Дж. Ульман. Компиляторы: принципы, технологии и инструментарий. — М.: «Вильямс», 2008.
10. О
Опалева Э. А., Самойленко В.П. Языки программирования и методы трансляции. - СПб.: БХВ- Петербург, 2005.
11. А
А.С. Семенов. Построение класса фрактальных систем по шаблону на примере дерева Фибоначчи // РАН. Информационные технологии и Вычислительные системы. — М.: N2, 2005 стр.10-17.